Функции расстояния



Измеряй измеримое

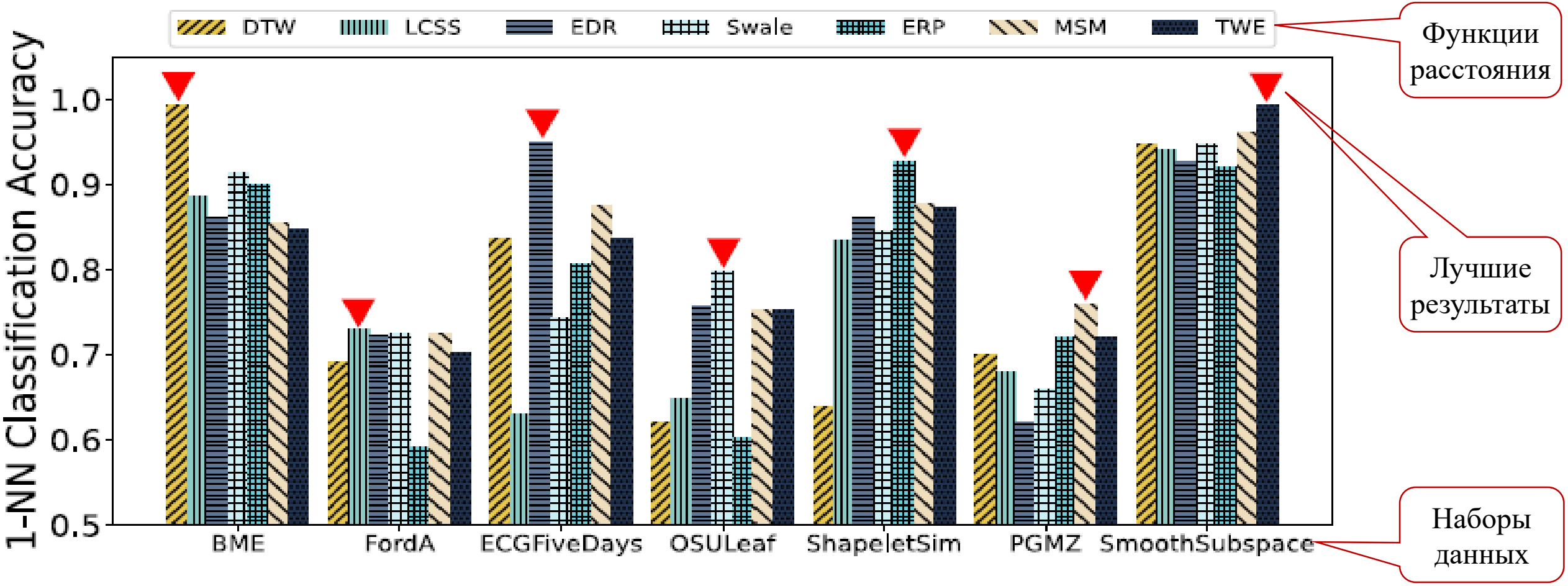
и делай неизмеримое измеримым.

Г. Галилей

Содержание

- Иерархия функций расстояния
- Евклидово расстояние
- Динамическая трансформация времени (DTW)

Важность выбора функции расстояния



Paparrizos, J. *et al.* Accelerating similarity search for elastic measures: A study and new generalization of lower bounding distances. Proc. VLDB Endow. 16(8), 2019–2032 (2023). DOI 10.14778/3594512.3594530. https://www.vldb.org/pvldb/vol16/p2019-paparrizos.pdf

Аксиомы метрики: Dist: $M \times M \to \mathbb{R}$: $\forall x, y, z \in M$

- Аксиома тождества:
$$\mathrm{Dist}(x, x) = 0$$
- Аксиома положительности:
$$\mathrm{Dist}(x, y) \geq 0$$
- Аксиома симметричности:
$$\mathrm{Dist}(x, y) = \mathrm{Dist}(y, x)$$
- Аксиома (неравенство) треугольника:
$$\mathrm{Dist}(x, z) \leq \mathrm{Dist}(x, y) + \mathrm{Dist}(y, z)$$

Метрика

Функция расстояния

Ослабленная функция расстояния

Иерархия функций расстояния



Содержание

- Иерархия функций расстояния
- **Евклидово расстояние**
  - Определение
  - Важность неравенства треугольника
  - Z-нормализованное евклидово расстояние
  - Преимущества и недостатки
- Динамическая трансформация времени (DTW)

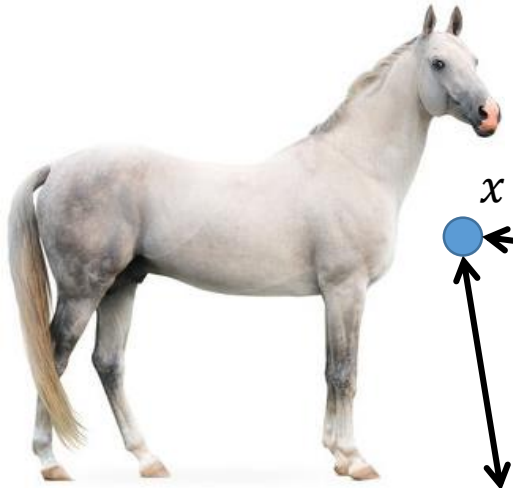
Евклидово расстояние*: $\mathrm{ED}(Q,C)=\sqrt{\sum_{i=1}^{m}(q_i-c_i)^2}$

- Интуитивно понятное расстояние
  - геометрический смысл – удаленность объектов друг от друга
  - сравнение равных по длине рядов по принципу «один-к-одному» без учета их формы
- Вычислительная сложность $O(m)$
- Ускорение вычислений: $\mathrm{ED}^2(Q,C)=\sum_{i=1}^{m}(q_i-c_i)^2$
  - Вычисляется быстрее, но не метрика (не выполняется неравенство треугольника)
  - Усиление несхожести
  - Усложнение интерпретации (квадраты единиц)



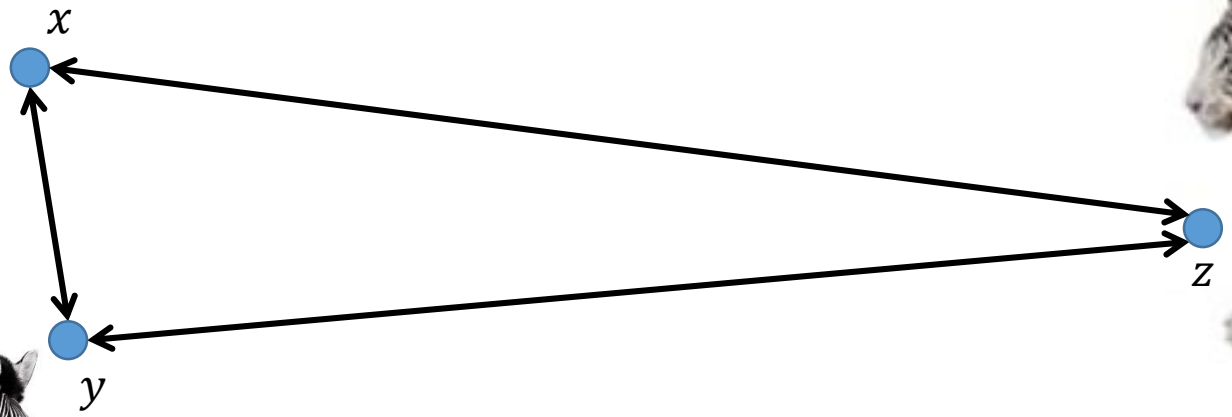
Евклид*
325-265 д.н.э.

Неравенство треугольника выполнено





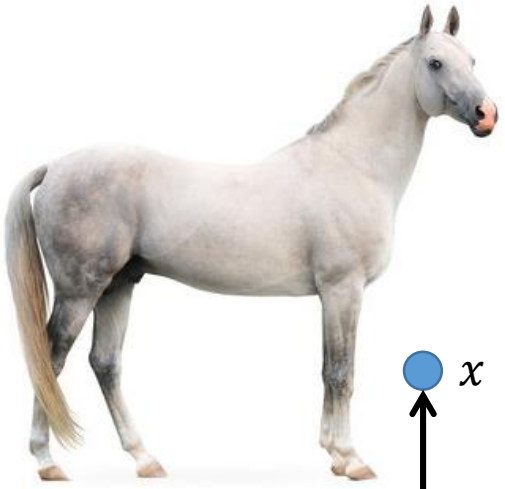




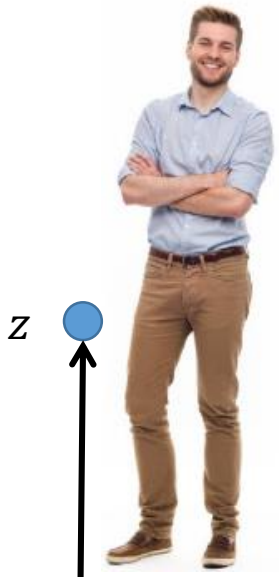
$$\forall x, y, z$$

$$\mathrm{Dist}(x, z) \leq \mathrm{Dist}(x, y) + \mathrm{Dist}(y, z)$$

Неравенство треугольника нарушено: ОНО РАБОТАЕТ, НО НЕ ВСЕГДА









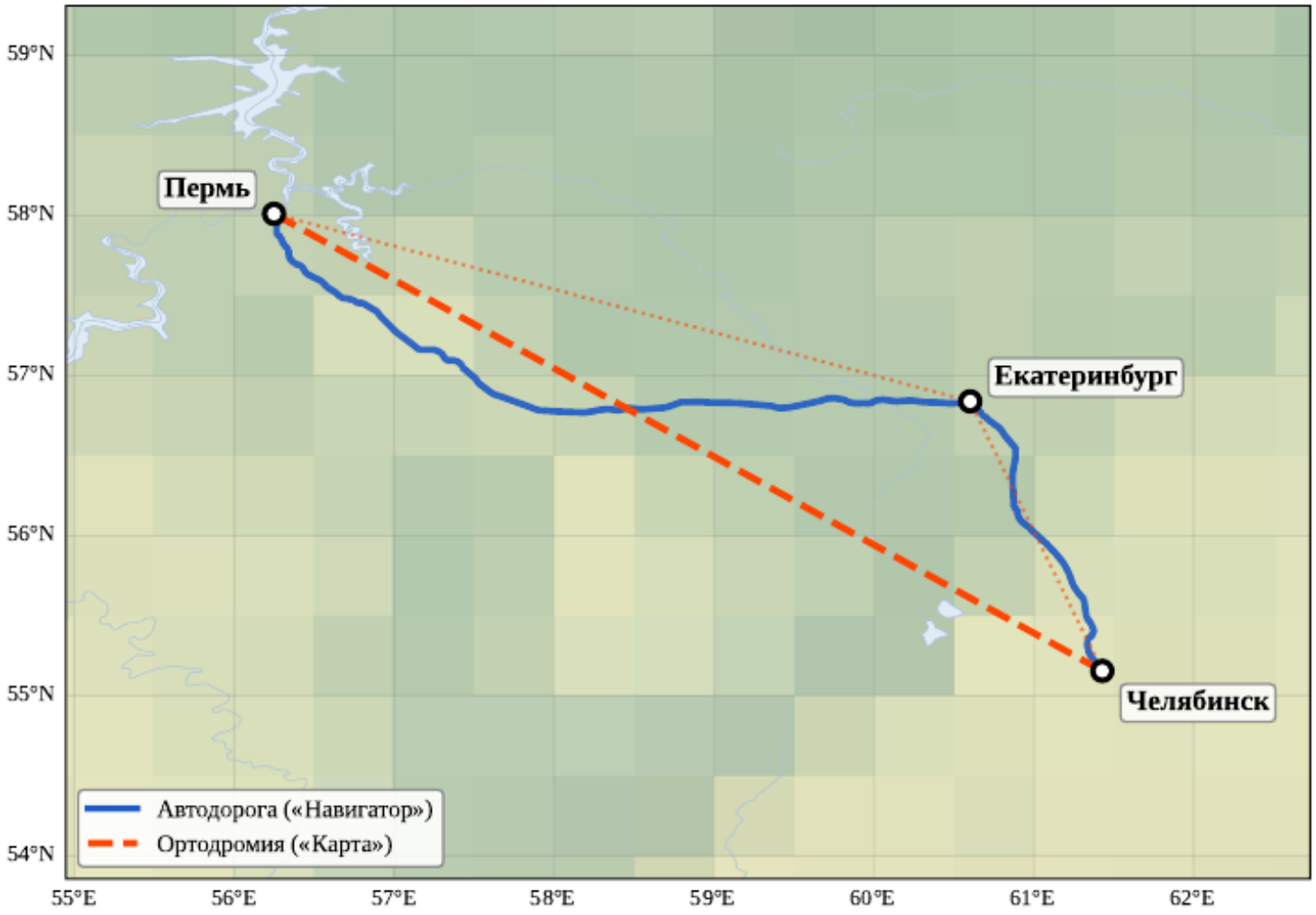


$$\neg(\forall x, y, z \; \mathrm{Dist}(x, z) \leq \mathrm{Dist}(x, y) + \mathrm{Dist}(y, z))$$

$$\Downarrow$$

$$\exists\, x, y, z \; \mathrm{Dist}(x, z) > \mathrm{Dist}(x, y) + \mathrm{Dist}(y, z)$$

Нарушение неравенства треугольника – не повод отвергнуть функцию расстояния



$$\text{Карта}(\text{Чел}, \text{Прм}) \leq \text{Карта}(\text{Чел}, \text{Екб}) + \text{Карта}(\text{Екб}, \text{Прм})$$

$$\text{Навигатор}(\text{Чел}, \text{Прм}) > \text{Навигатор}(\text{Чел}, \text{Екб}) + \text{Навигатор}(\text{Екб}, \text{Прм})$$

Выполнение неравенства треугольника – возможность ускорить поиск ближайших соседей

- Неравенство треугольника:

$$\forall x, y, z\ \mathrm{Dist}(x, z) \leq \mathrm{Dist}(x, y) + \mathrm{Dist}(y, z)$$

- Обратное неравенство треугольника

$$\forall x, y, z\ |\mathrm{Dist}(x, y) - \mathrm{Dist}(x, z)| \leq \mathrm{Dist}(y, z)$$

- Нижняя граница расстояния

Если x фиксирован, y, z – произвольные, то LB – нижняя граница $\mathrm{Dist}(y, z)$

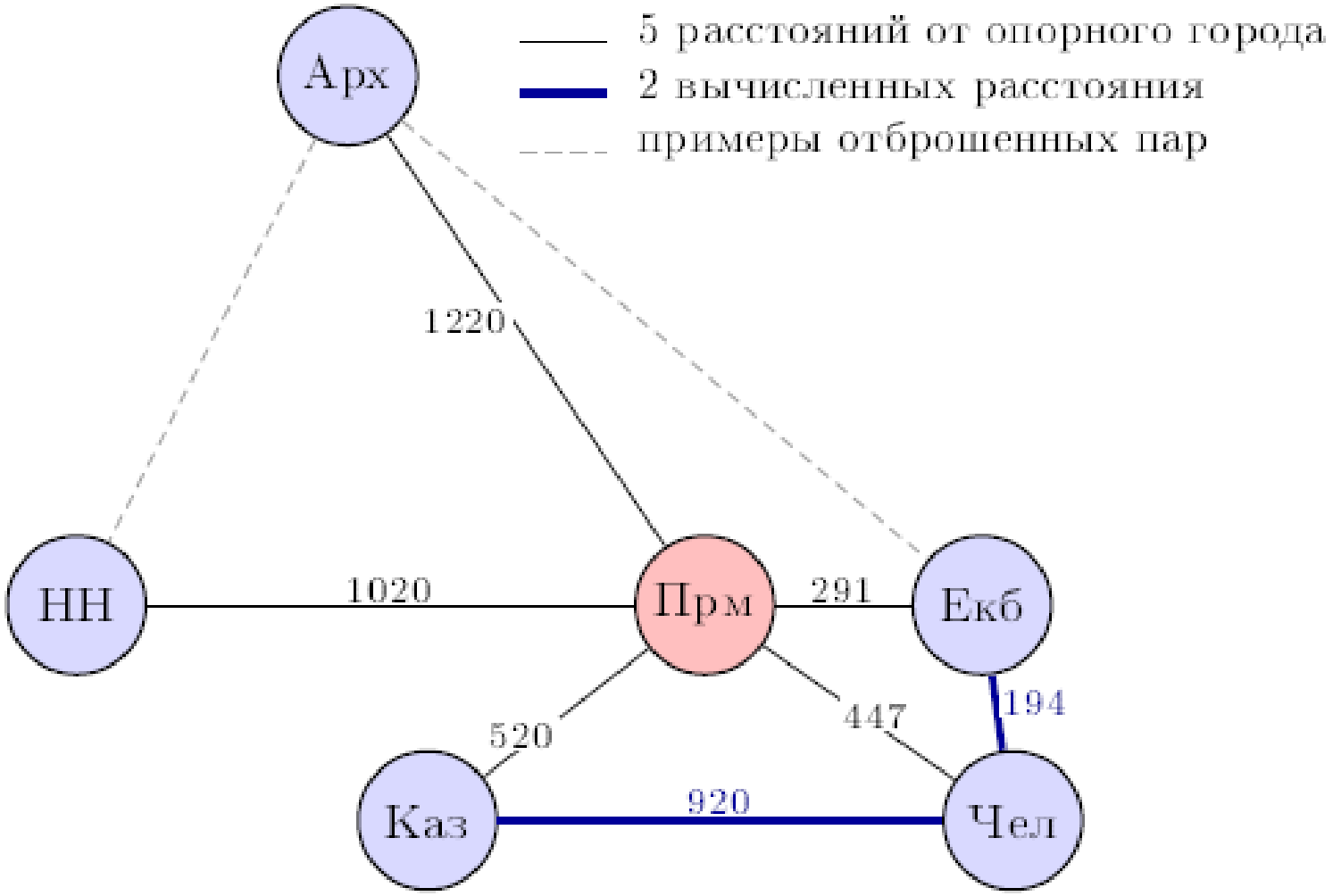
$$\mathrm{LB}(y, z) = |\mathrm{Dist}(x, y) - \mathrm{Dist}(x, z)|$$

- Контрапозиция обратного неравенства треугольника
  - bsf – текущий минимум расстояния между перебираемыми парами
  - Если $\mathrm{LB}(y, z) > bsf$, то $\mathrm{Dist}(y, z) > bsf$. Тогда пару (y, z) можно отбросить без вычисления $\mathrm{Dist}(y, z)$

Неравенство треугольника ускоряет поиск ближайших соседей

Карта(·, ·)	Арх	Екб	Каз	НН	Прм	Чел
Арх	0	1690	1360	1160	1220	1960
Екб	1690	0	800	1260	291	194
Каз	1360	800	0	390	520	920
НН	1160	1260	390	0	1020	1400
Прм	1220	291	520	1020	0	447
Чел	1960	194	920	1400	447	0

Алгоритм	Количество операций
Неравенство треугольника, просмотр по возрастанию	**7**
Неравенство треугольника, просмотр по убыванию	8
Неравенство треугольника, просмотр в случайном порядке	9
Полный перебор	15



Z-нормализация: сравнение рядов без учета разницы амплитуд

$$\hat{T} = (\hat{t}_1, \dots, \hat{t}_m), \qquad \hat{t}_i = \frac{t_i - \mu}{\sigma}$$

$$\mu = \frac{1}{m}\sum_{i=1}^{m} t_i, \qquad \sigma = \sqrt{\frac{1}{m}\sum_{i=1}^{m} t_i^2 - \mu^2}$$

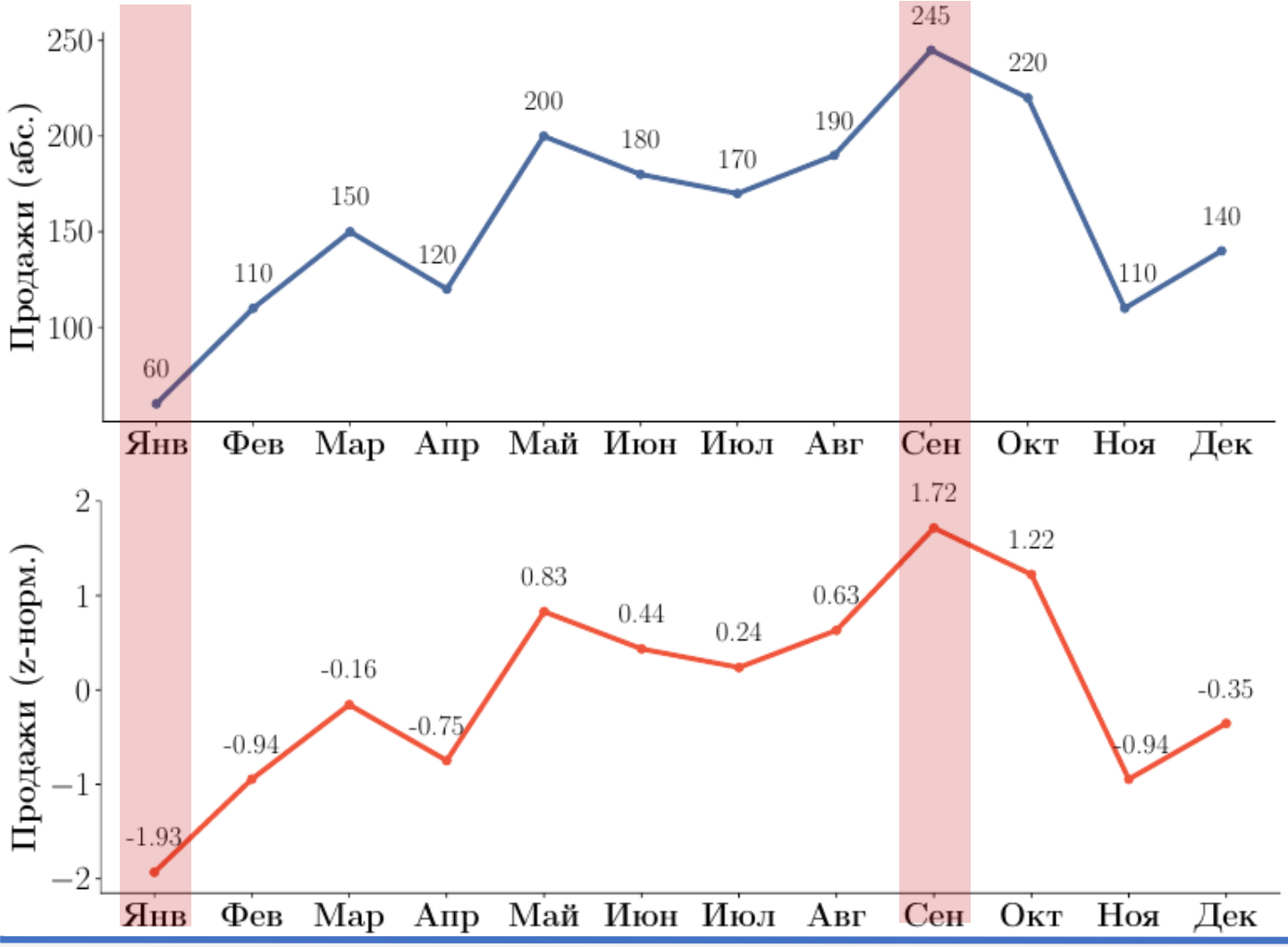
$\mu_{\hat{T}}=0$

$\sigma_{\hat{T}}=1$

$$\widehat{\mathrm{ED}}(Q, C) = \mathrm{ED}(\hat{Q}, \hat{C})$$

- Возможность сравнения рядов (подпоследовательностей) с разными масштабами и единицами измерения
- Простота интерпретации
- Улучшение обучения ML-моделей

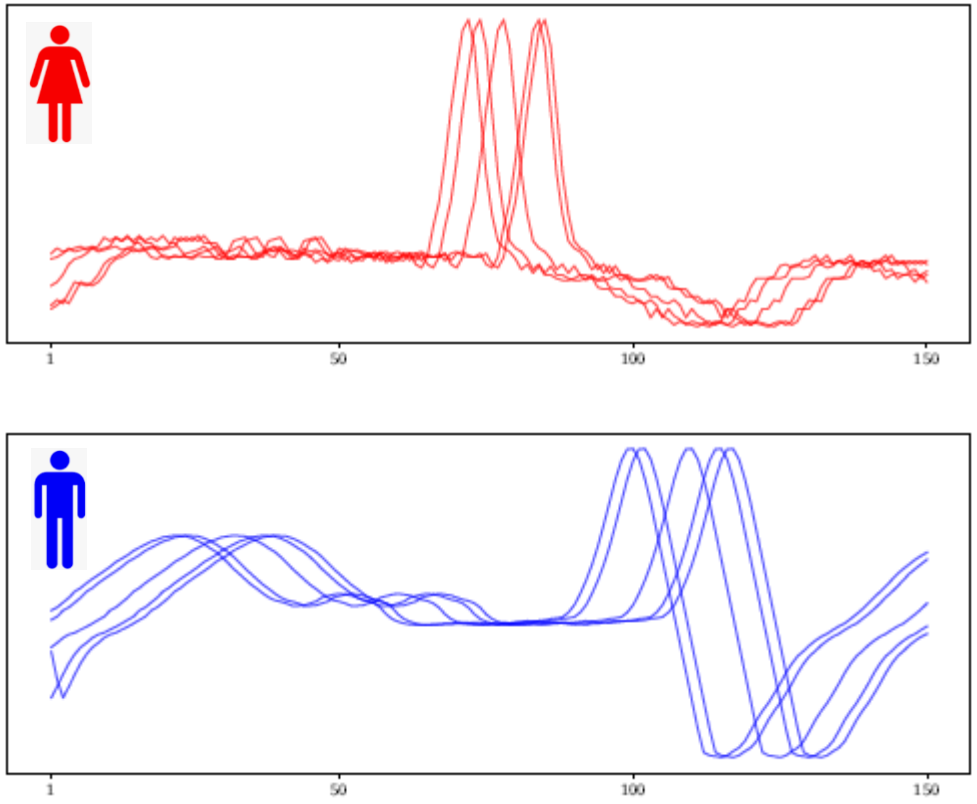
Пример z-нормализации



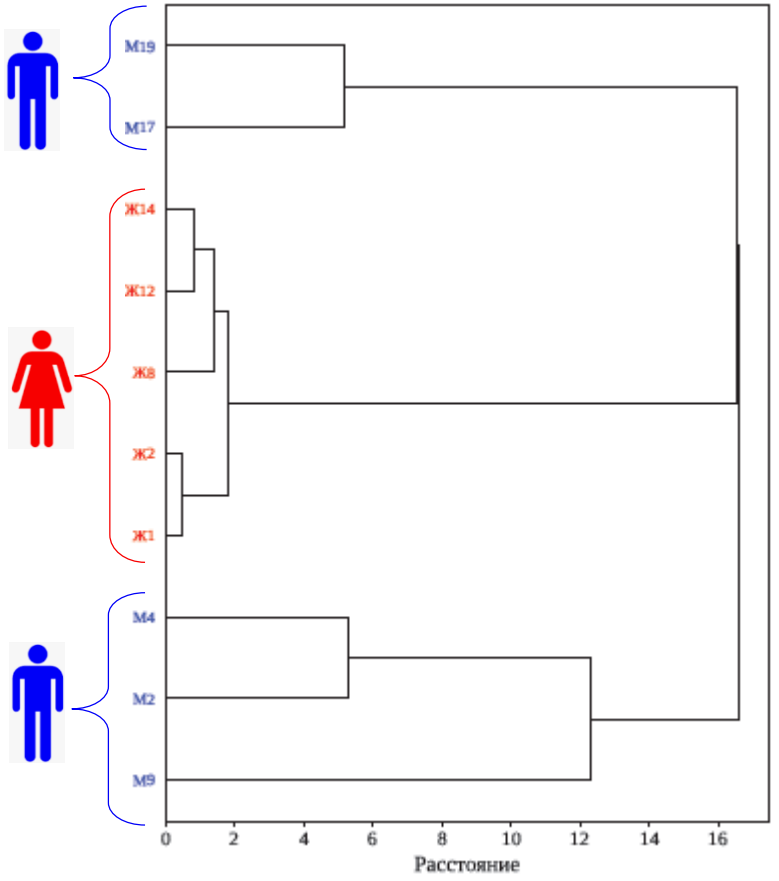
- Шкала меняется, но форма остается
- В январе продажи на 1.93 стандартных отклонения ниже среднего (значительно ниже обычного)
- В сентябре продажи на 1.72 стандартных отклонения выше среднего (значительно выше обычного)

Важность z-нормализации подпоследовательностей

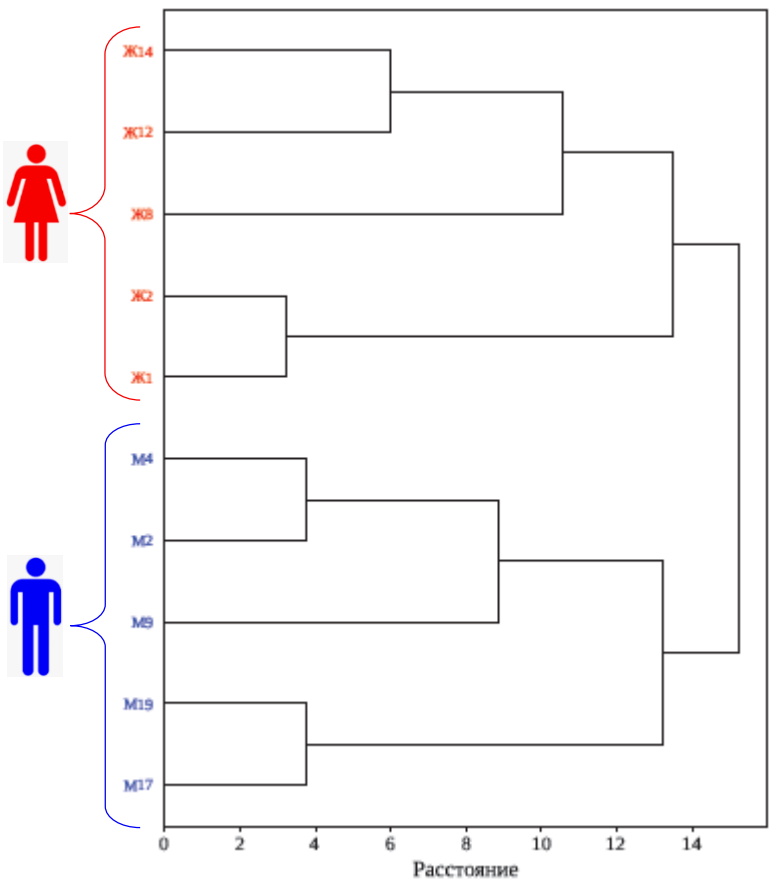
BIDMC Congestive Heart Failure Database*



Не нормализованные



Z-нормализованные

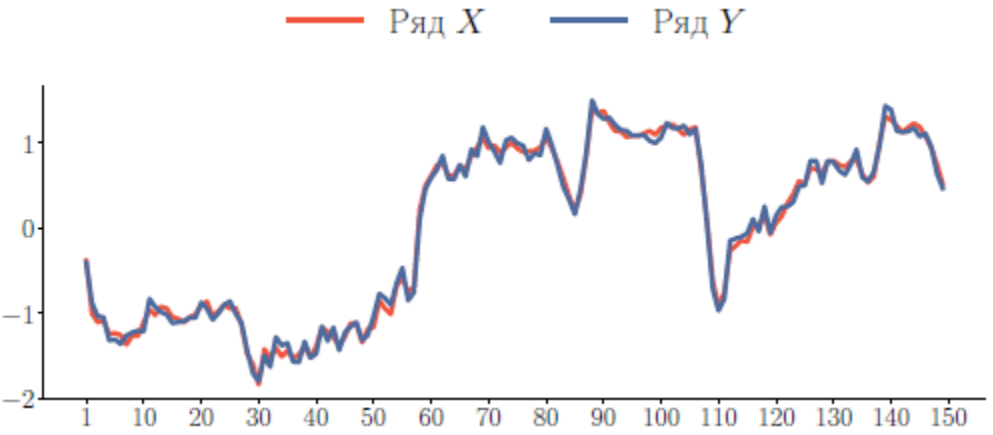


* BIDMC Congestive Heart Failure Database. URL: https://www.kaggle.com/datasets/shymammoth/bidmc-congestive-heart-failure

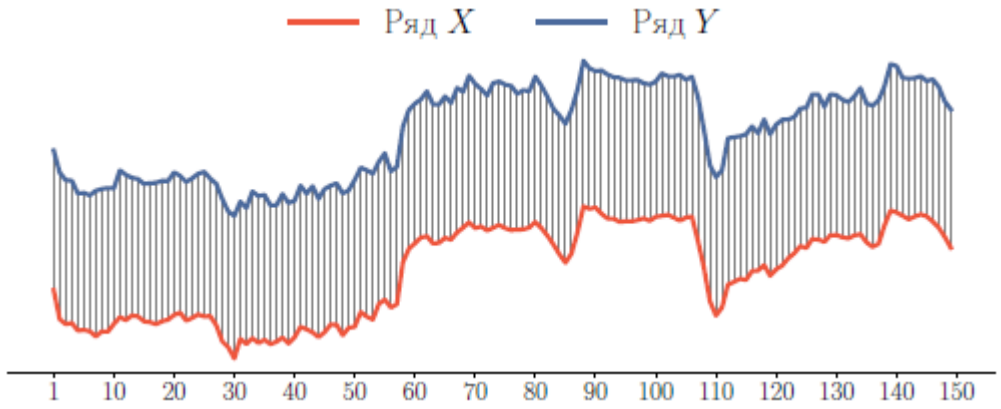
Евклидово расстояние: обсуждение

- Преимущества
  - Одна из наиболее распространенных метрик
  - Естественность (геометрическая интерпретация)
  - Выполнение неравенства треугольника (нижние границы)
- Недостатки
  - Чувствительность к выбросам
  - Чувствительность к масштабу
  - Только для рядов равной длины
  - Не учитывает сдвиги по времени

Евклидово расстояние не учитывает сдвиги по времени



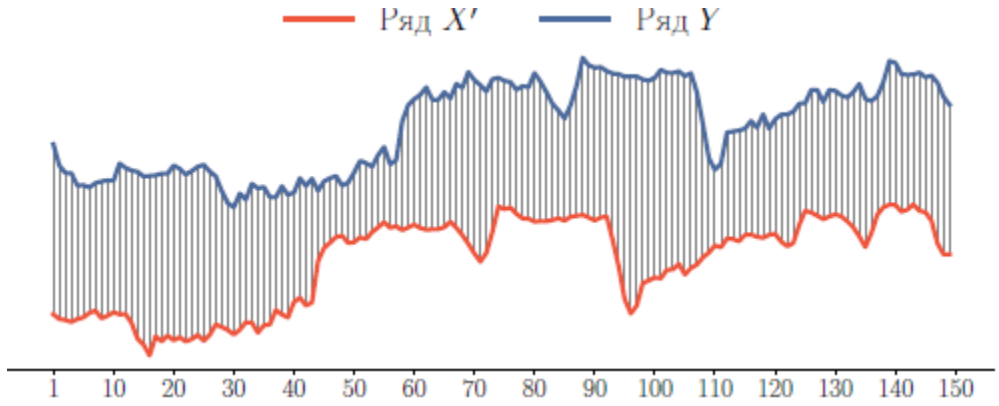
$$\mathrm{ED}(X, Y) = \mathbf{0.925}$$



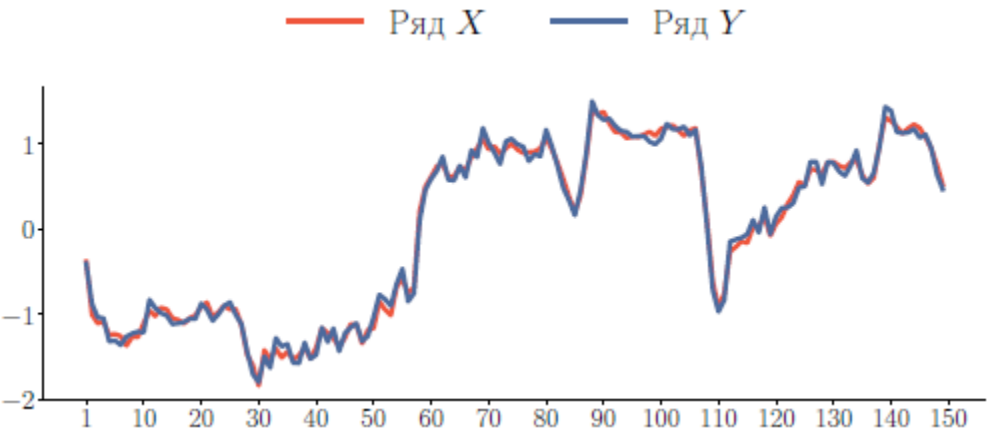
Ряд X' получен сдвигом ряда X



$$\mathrm{ED}(X', Y) = \mathbf{9.513}$$

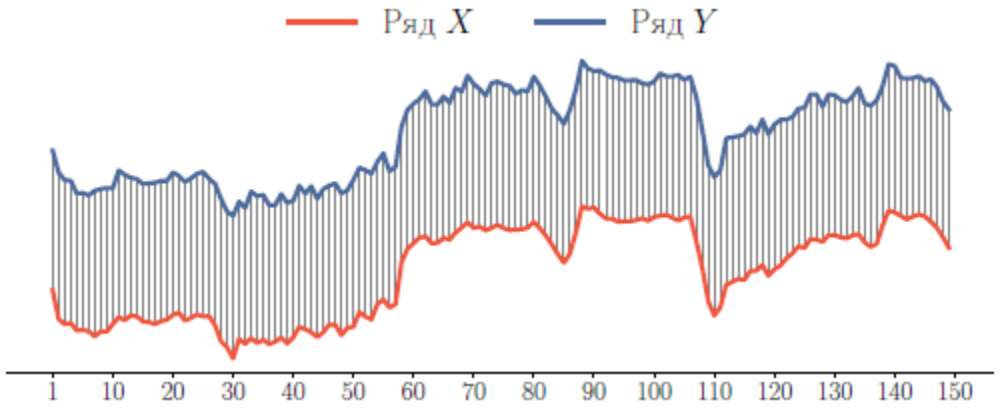


DTW-расстояние учитывает сдвиги по времени



$\mathrm{DTW}(X, Y) = \mathbf{0.838}$

$\mathrm{ED}(X, Y) = \mathbf{0.925}$

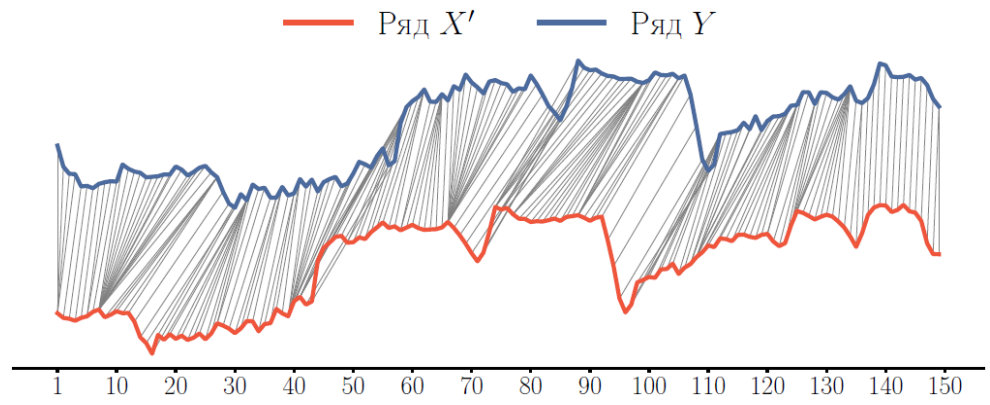


Ряд X' получен сдвигом ряда X



$\mathrm{DTW}(X', Y) = \mathbf{1.941}$

$\mathrm{ED}(X', Y) = \mathbf{9.513}$



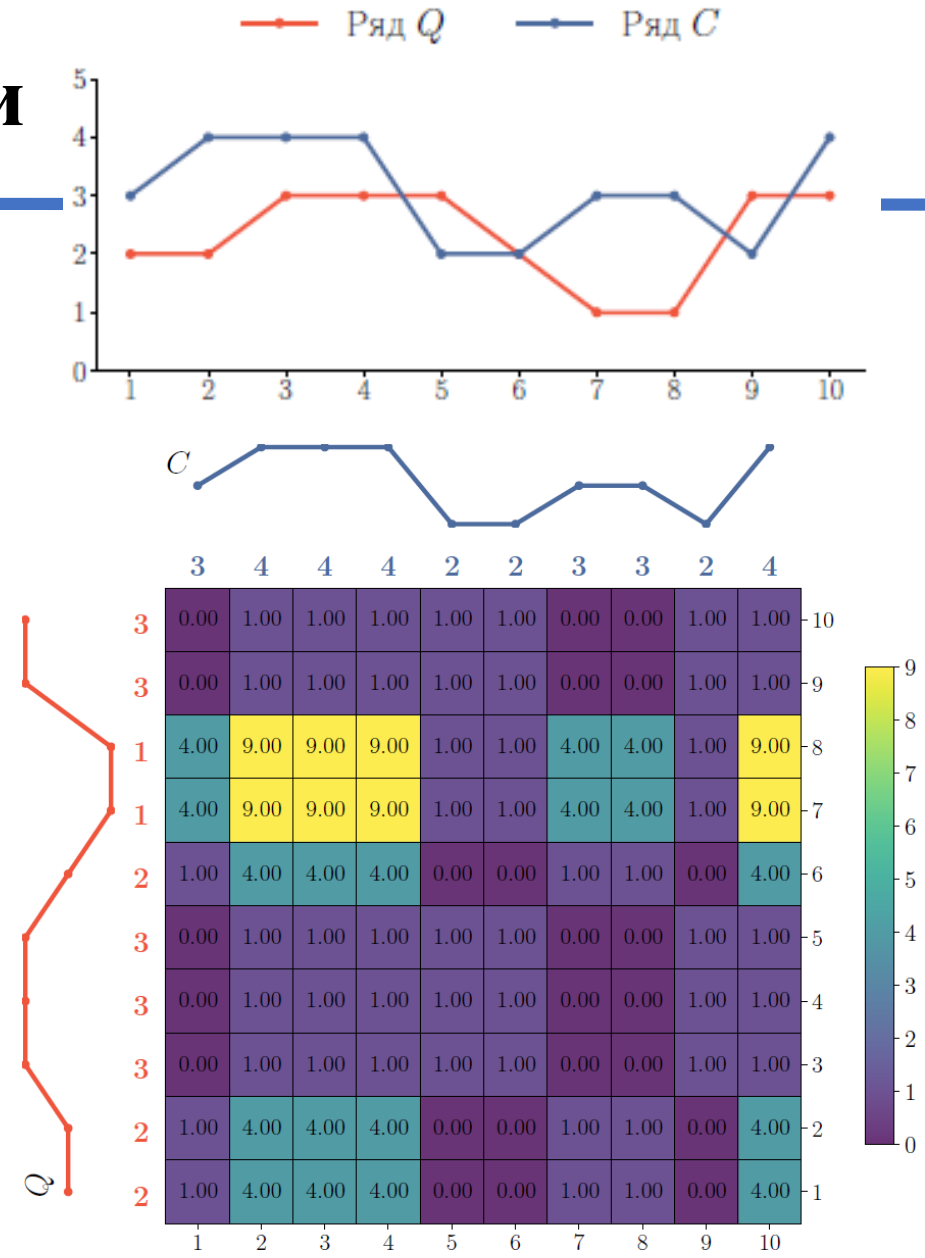
Содержание

- Иерархия функций расстояния
- Евклидово расстояние
- **Динамическая трансформация времени (DTW)**
  - Формальное определение
  - Ограничения пути трансформации

Динамическая трансформация времени, DTW (Dynamic Time Warping)

- Как DTW измеряет сходство
  - манера «один-ко-много»
  - трансформируется ось времени одного или обоих рядов так, чтобы достигнуть их оптимального взаимного выравнивания
  - вычисляется не расстояние между рядами в геометрическом смысле, а стоимость их оптимального взаимного выравнивания
- DTW – не формула, а алгоритм со многими гиперпараметрами
  - функция локальной стоимости,
  - функция стоимости пути трансформации,
  - параметры ограничений пути трансформации,
  - шаблон шага пути трансформации
  - …

Функция и матрица локальной стоимости



- *Функция локальной стоимости* измеряет стоимость выравнивания (расстояние) между точками ряда – гиперпараметр DTW

$$\text{Cost}(q_i, c_j) = \text{ED}^2(q_i, c_j) = (q_i - c_j)^2$$

- *Матрица локальной стоимости* (матрица расстояний) показывает стоимость выравнивания между всеми возможными парами точек рядов Q и C

$$d(i,j) = \text{Cost}(q_i, c_j)$$

Путь трансформации

- Последовательность шагов по ячейкам матрицы локальной стоимости, шаг – пара (строка, столбец)

  $W = \{w_k\}_{k=1}^{K}, w_k = (i, j), m \le K < 2m - 1$

- Ограничения пути трансформации
  1. **Граничные условия** (содержит все точки рядов)

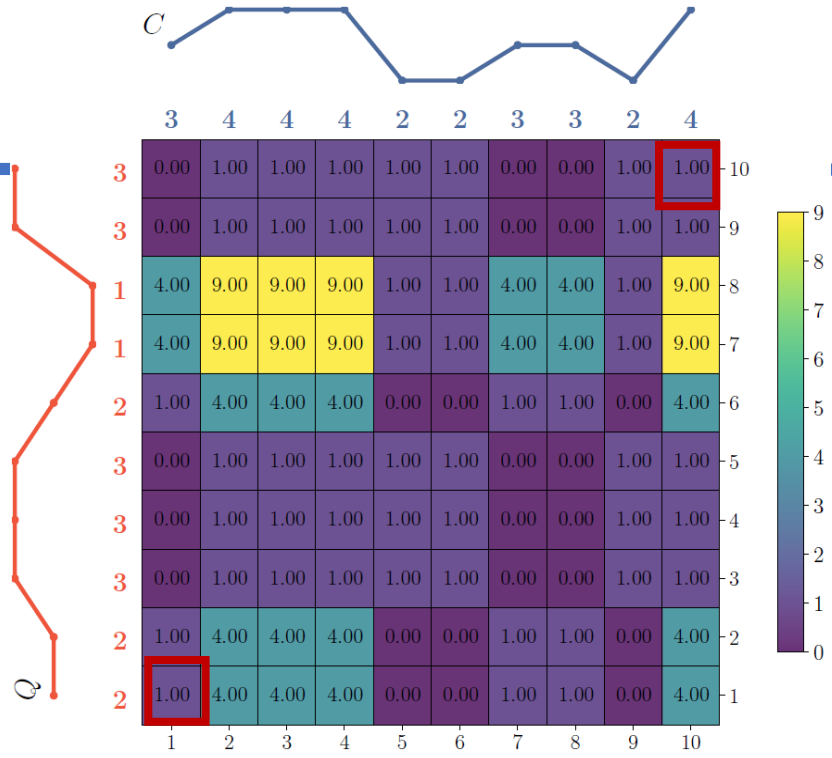
     $w_1 = (1,1), w_K = (m, m)$

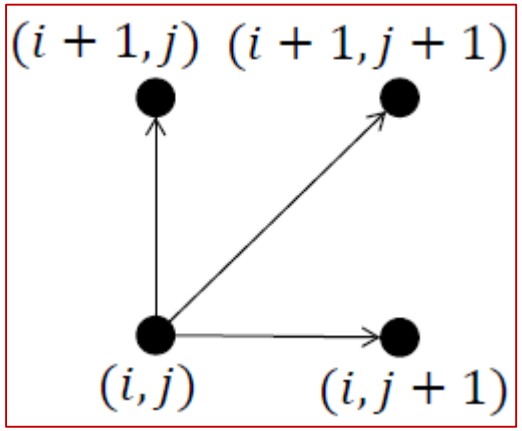
  2. **Монотонность** (без возврата к пройденному)

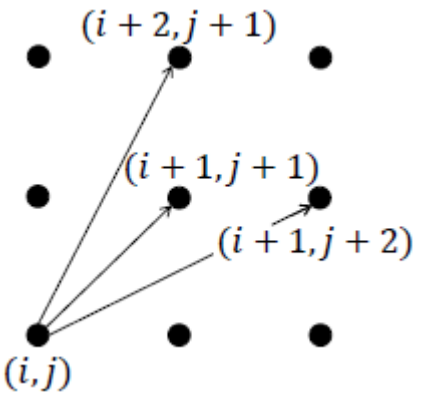
     $\forall\, w_k = (i, j)$ и $w_{k+1} = (p, q)$:

     $i - p \ge 0$ и $q - j \ge 0$

  3. **Непрерывность** (каждый шаг по шаблону)







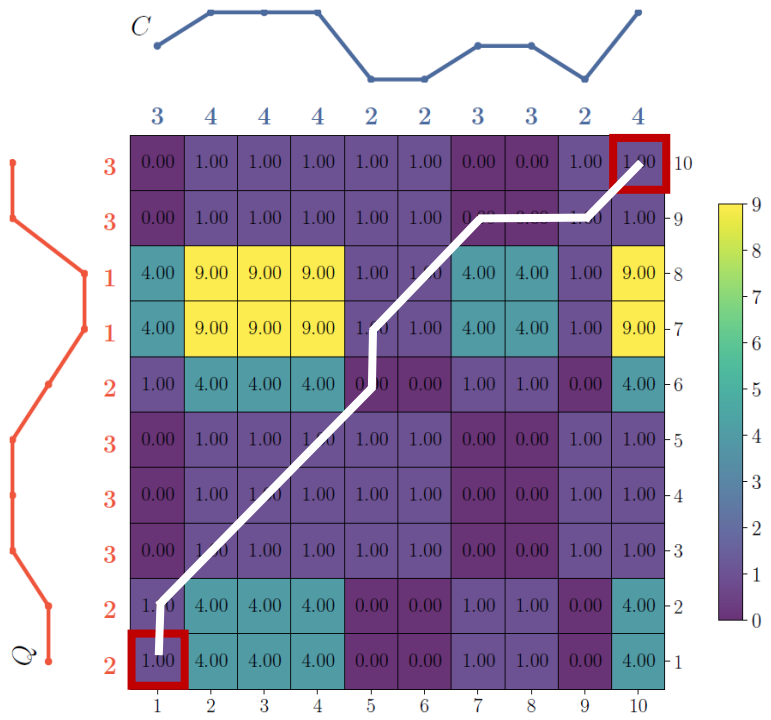
Функция стоимости пути трансформации

- Стоимость трудоемкости взаимного выравнивания рядов, квадратный корень из суммы локальных стоимостей выравнивания всех точек, входящих в путь

$$\mathrm{Cost}_W(Q,C) = \sqrt{\sum_{k=1}^{K} d(w_k)}$$

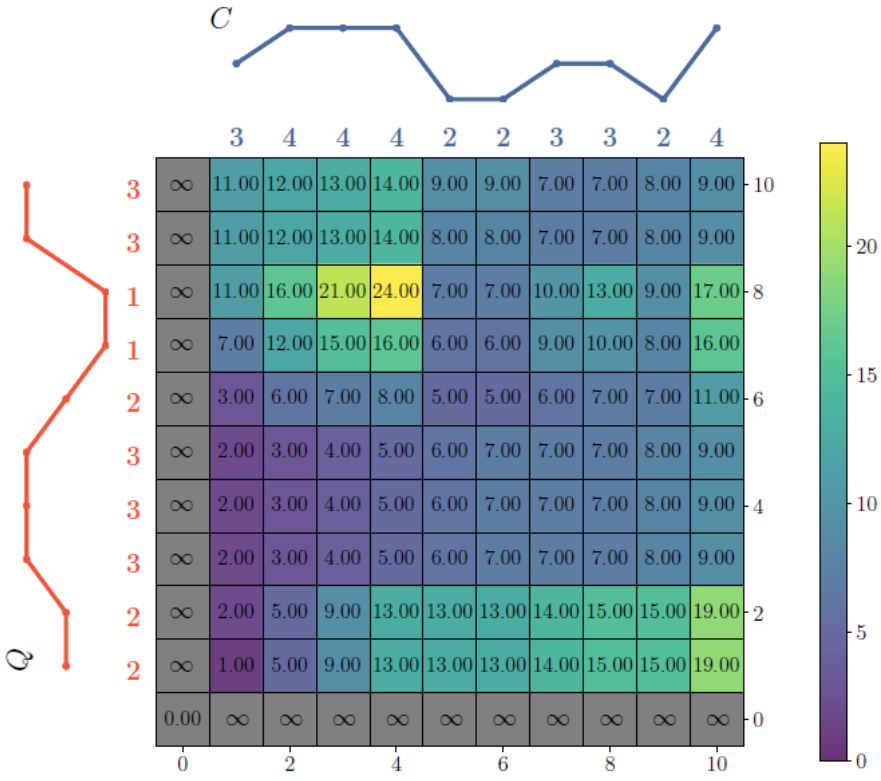
- DTW – стоимость оптимального пути трансформации W^*, имеющего минимальную стоимость

$$\mathrm{DTW}(Q,C) = \mathrm{Cost}_{W^*}(Q,C) = \min_W \sqrt{\sum_{k=1}^{K} d(w_k)}$$

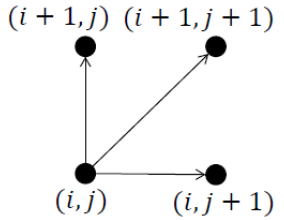


Матрица глобальной стоимости (матрица трансформации)

- Поиск оптимального пути в матрице локальной стоимости невозможен из-за экспоненциального роста числа путей-кандидатов
- Поиск оптимального пути выполняется в *матрице глобальной стоимости* методом динамического программирования



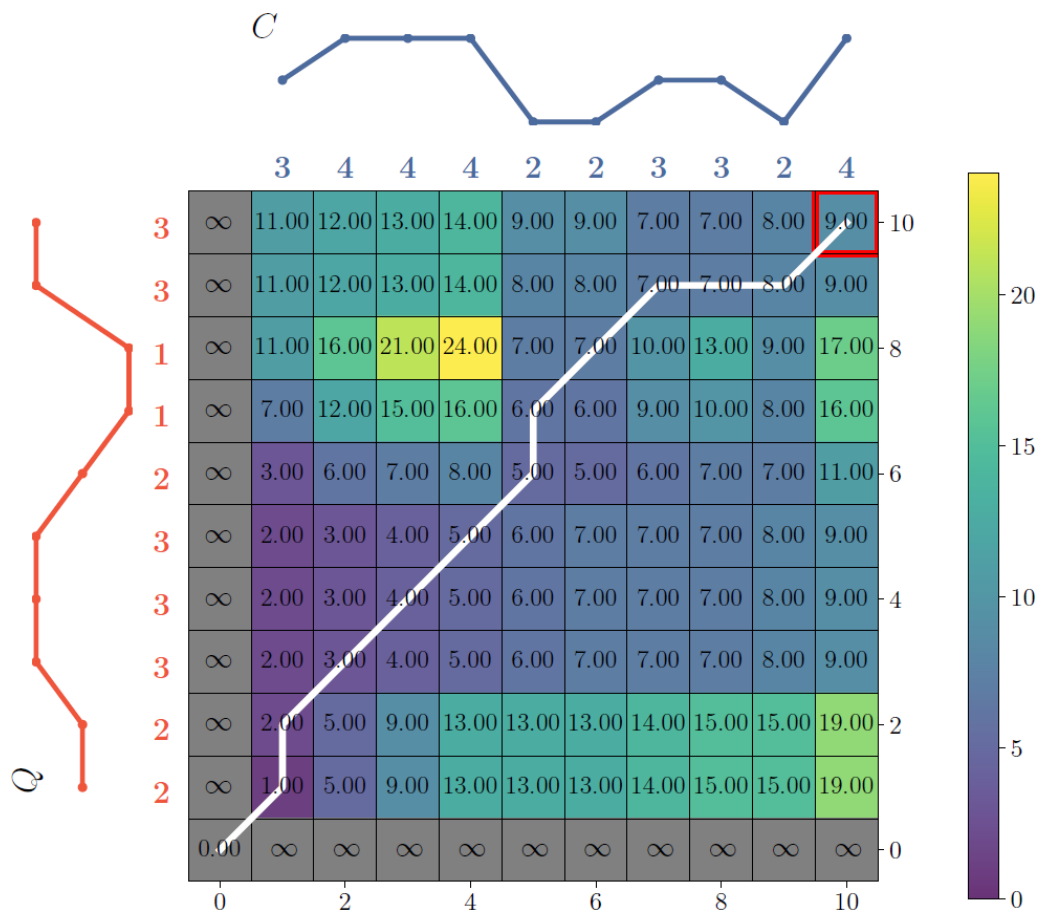
$$D(i,0) = D(0,j) = +\infty, D(0,0) = 0,$$
$$D(i,j) = d(i,j) + \min(D(i-1,j-1), D(i-1,j), D(i,j-1))$$



DTW

- Расстояние DTW между заданными рядами – это квадратный корень из значения в ячейке матрицы глобальной стоимости в финальной точке оптимального пути трансформации

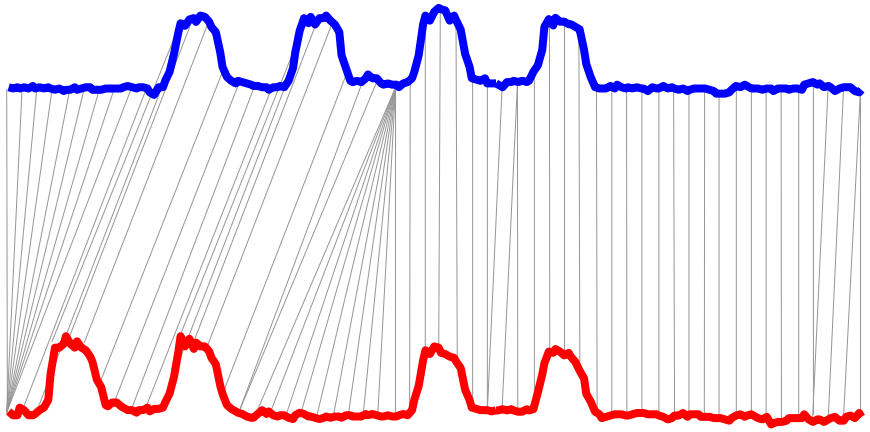
$$\mathrm{DTW}(Q, C) = \sqrt{D(m, m)}$$



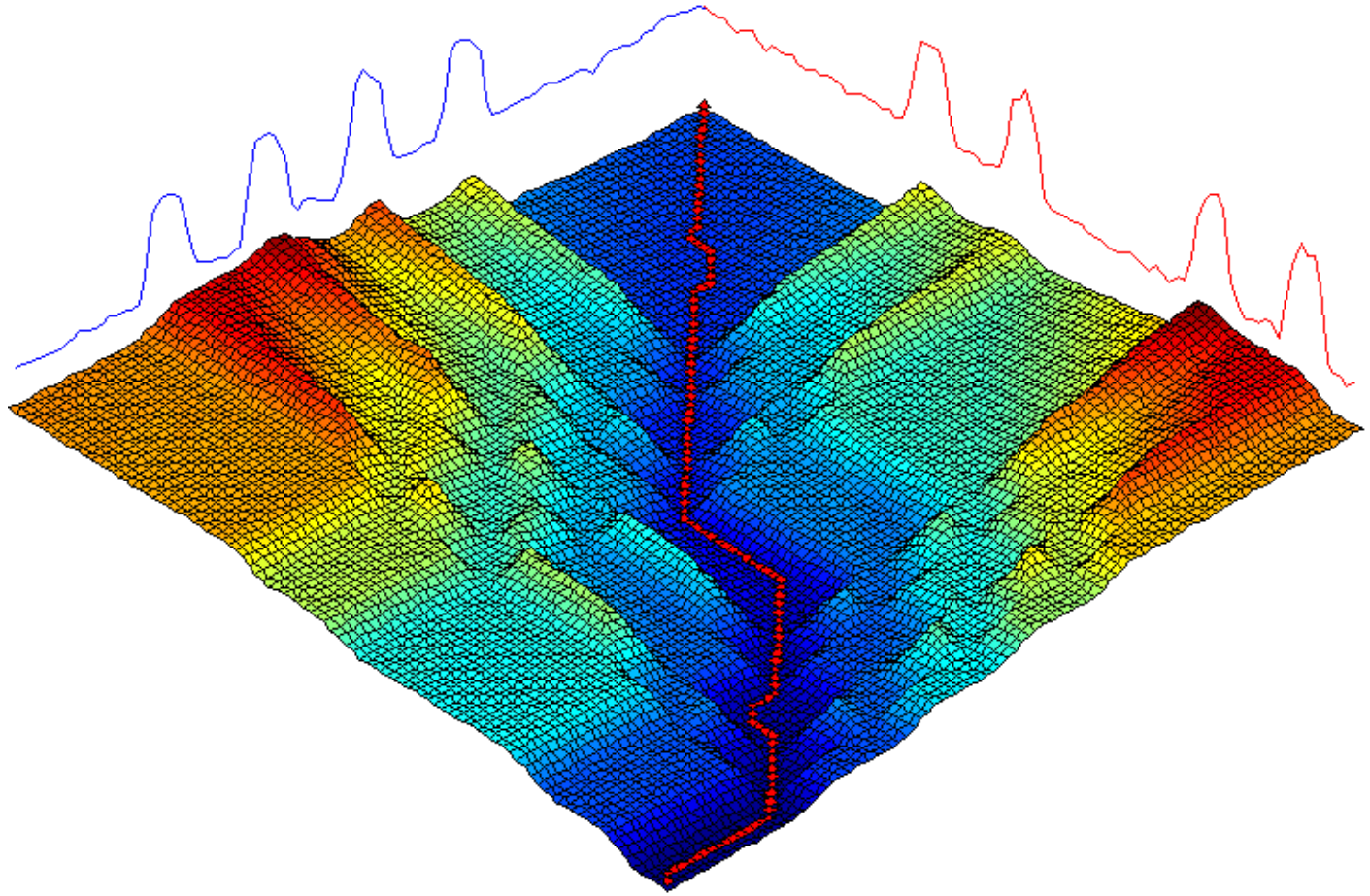
Пример оптимального пути трансформации

Недельное энергопотребление вычислительного центра (Голландия, 1997)*

C: 4-дневная рабочая неделя,
Понедельник – выходной



Q: 4-дневная рабочая неделя,
Среда – выходной



* van Wijk J.J., van Selow R.R. Cluster and calendar based visualization of time series data. INFOVIS 1999: 4-9. DOI: 10.1109/INFVIS.1999.801851

Алгоритм вычисления DTW

Вход: ряды Q, C длины m

Выход: расстояние DTW от Q до C

Данные: матрица локальной стоимости $d \in \mathbb{R}^{m \times m}$,
матрица глобальной стоимости $D \in \mathbb{R}^{(1+m) \times (1+m)}$

1: $D(0, \cdot) \leftarrow +\infty_{1+m}$; $D(\cdot, 0) \leftarrow +\infty_{1+m}$; $D(0, 0) \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** m **do**
3: **for** $j \leftarrow 1$ **to** m **do**
4: $d(i, j) \leftarrow \mathrm{Cost}(q_i, c_j)$
5: $D(i, j) \leftarrow d(i, j) + \min\big(D(i-1, j-1),\ D(i-1, j),\ D(i, j-1)\big)$
6: **end for**
7: **end for**
8: **return** $\sqrt{D(m, m)}$

Алгоритм вычисления DTW

Вход: ряды Q, C длины m

Выход: расстояние DTW от Q до C

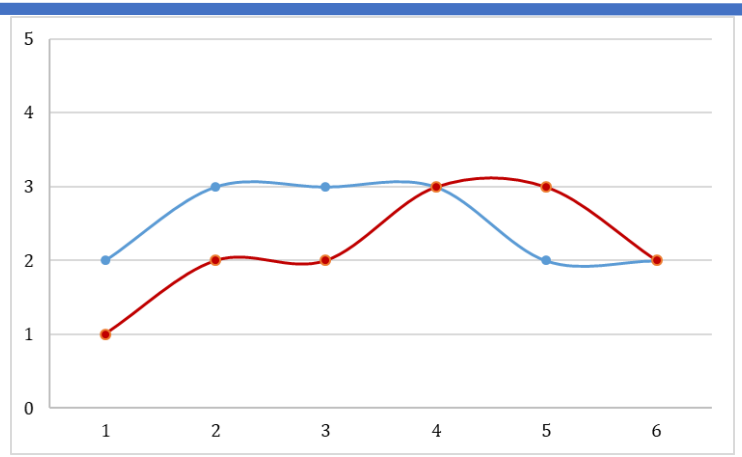
Данные: матрица локальной стоимости $d \in \mathbb{R}^{m \times m}$,
матрица глобальной стоимости $D \in \mathbb{R}^{(1+m)\times(1+m)}$

Пространственная сложность $\boldsymbol{O(m^2)}$

1: $D(0, \cdot) \leftarrow +\infty_{1+m}$; $D(\cdot, 0) \leftarrow +\infty_{1+m}$; $D(0, 0) \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** m **do**
3: **for** $j \leftarrow 1$ **to** m **do**
4: $d(i, j) \leftarrow \text{Cost}(q_i, c_j)$
5: $D(i, j) \leftarrow d(i, j) + \min\big(D(i-1, j-1), D(i-1, j), D(i, j-1)\big)$
6: **end for**
7: **end for**
8: **return** $\sqrt{D(m, m)}$

Вычислительная сложность $\boldsymbol{O(m^2)}$

Пример вычисления DTW



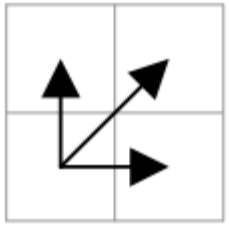
$$DTW(Q,C) = D(m,m)$$

$$D(i,j) = (q_i - c_j)^2 + \min\begin{cases} D(i-1,j) \\ D(i,j-1) \\ D(i-1,j-1) \end{cases}$$

$$D(0,0) = 0, \qquad D(i,0) = D(0,j) = +\infty$$

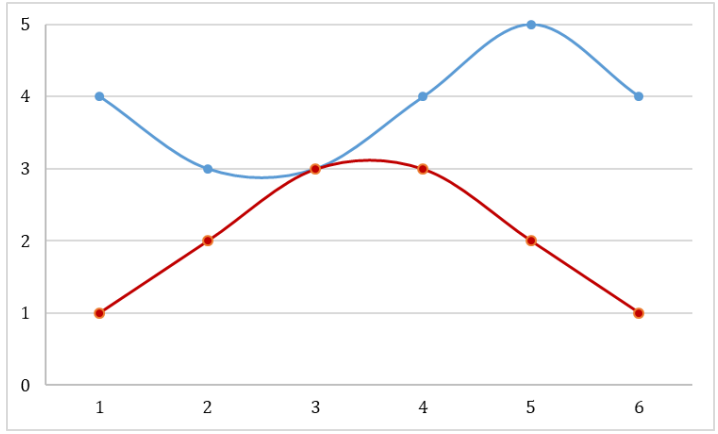
$$C = (2,3,3,3,2,2), Q = (1,2,2,3,3,2)$$

d	2	3	3	3	2	2	
2	0	1	1	1	0	0	6
3	1	0	0	0	1	1	5
3	1	0	0	0	1	1	4
2	0	1	1	1	0	0	3
2	0	1	1	1	0	0	2
1	1	4	4	4	1	1	1
→ j	1	2	3	4	5	6	↑ i



D	2	3	3	3	2	2	
2	+∞	3	2	2	2	1	**1**	6
3	+∞	3	1	1	1	2	3	5
3	+∞	2	1	1	1	2	3	4
2	+∞	1	2	3	4	4	4	3
2	+∞	1	2	3	4	4	4	2
1	+∞	1	5	9	13	14	15	1
	0	+∞	+∞	+∞	+∞	+∞	+∞	0
→ j	0	1	2	3	4	5	6	↑ i

Пример вычисления DTW



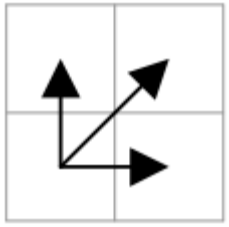
$$\mathrm{DTW}(Q, C) = D(m, m)$$

$$D(i,j) = (q_i - c_j)^2 + \min \begin{cases} D(i-1,j) \\ D(i,j-1) \\ D(i-1,j-1) \end{cases}$$

$$D(0,0) = 0, \qquad D(i,0) = D(0,j) = +\infty$$

$$C = (4,3,3,4,5,4), Q = (1,2,3,3,2,1)$$

d	4	3	3	4	5	4	
1	9	4	4	9	16	9	6
2	4	1	1	4	9	4	5
3	1	0	0	1	4	1	4
3	1	0	0	1	4	1	3
2	4	1	1	4	9	4	2
1	9	4	4	9	16	9	1
$\rightarrow j$	1	2	3	4	5	6	$\uparrow i$



D	4	3	3	4	5	4	
1	$+\infty$	28	15	15	20	30	**28**	6
2	$+\infty$	19	11	11	14	20	19	5
3	$+\infty$	15	10	10	11	15	16	4
3	$+\infty$	14	10	10	11	15	16	3
2	$+\infty$	13	10	11	15	24	28	2
1	$+\infty$	9	13	17	26	42	51	1
	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	0
$\rightarrow j$	0	1	2	3	4	5	6	$\uparrow i$

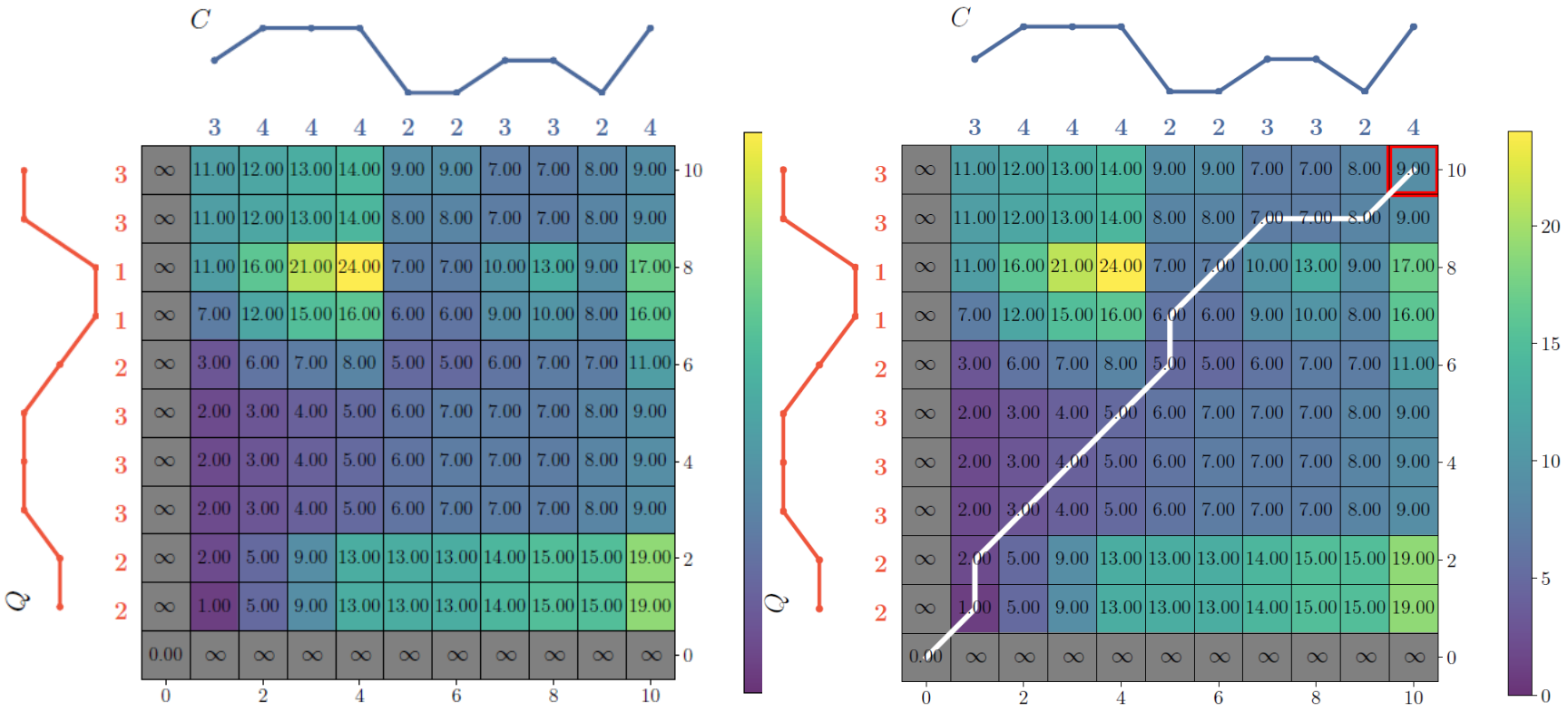
Построение оптимального пути трансформации

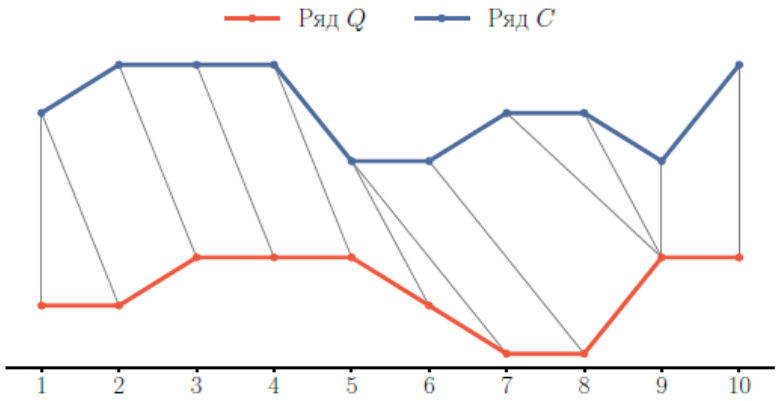
Вход: ряды Q, C длины m, матрица глобальной стоимости $D \in \mathbb{R}^{(1+m)\times(1+m)}$
Выход: оптимальный путь трансформации W^*

$i \leftarrow m$; $j \leftarrow m$; $k \leftarrow 0$; $W^* \leftarrow \varnothing$
while $i > 1$ **or** $j > 1$ **do**
  $k \leftarrow k + 1$; $w_k^* \leftarrow (i, j)$ ▷ Добавить текущую ячейку
  $diagCost \leftarrow +\infty$; $downCost \leftarrow +\infty$; $leftCost \leftarrow +\infty$
  ▷ Выбрать направление движения
  if $i > 1$ **and** $j > 1$ **then**
    $diagCost \leftarrow D(i-1, j-1)$
  end if
  if $i > 1$ **then**
    $downCost \leftarrow D(i-1, j)$
  end if
  if $j > 1$ **then**
    $leftCost \leftarrow D(i, j-1)$
  end if
  if $diagCost \leqslant downCost$ **and** $diagCost \leqslant leftCost$ **then**
    $i \leftarrow i - 1$; $j \leftarrow j - 1$
  else
    if $downCost \leqslant leftCost$ **then**
      $i \leftarrow i - 1$
    else
      $j \leftarrow j - 1$
    end if
  end if
end while
$k \leftarrow k + 1$; $w_k^* \leftarrow (1, 1)$ ▷ Добавить первую ячейку
$W^* \leftarrow \textsc{Reverse}(W^*)$ ▷ Обратить построенный путь
return W^*

Оптимальный путь восстанавливается из матрицы глобальной стоимости движением в порядке, обратном заполнению матрицы – от (m, m) к $(1, 1)$ – и затем обращается

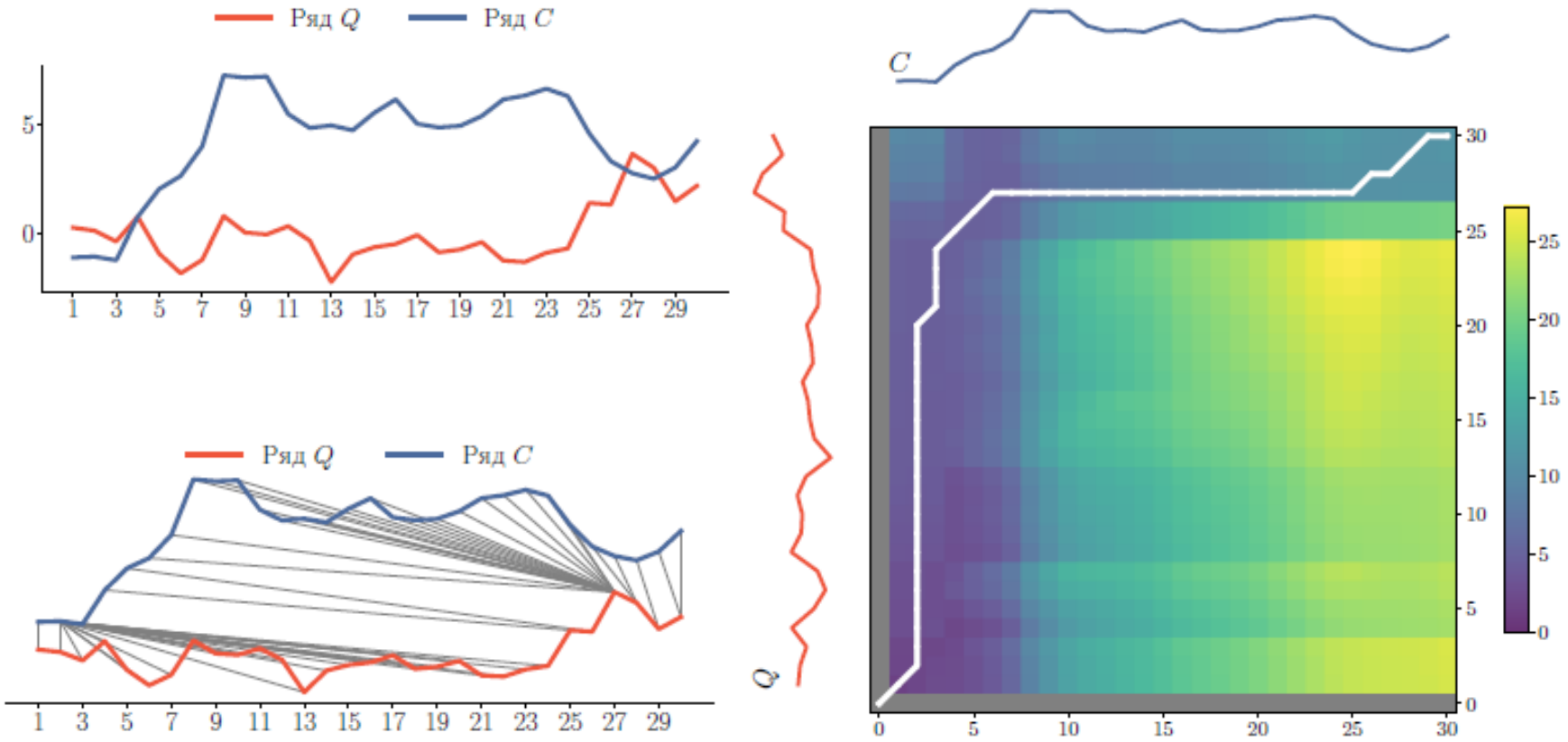
Построение оптимального пути





Патологическое выравнивание рядов

- DTW может полагать оптимальными интуитивно непонятные и нелогичные пути трансформации: вычисленное расстояние не будет отражать реальную схожесть

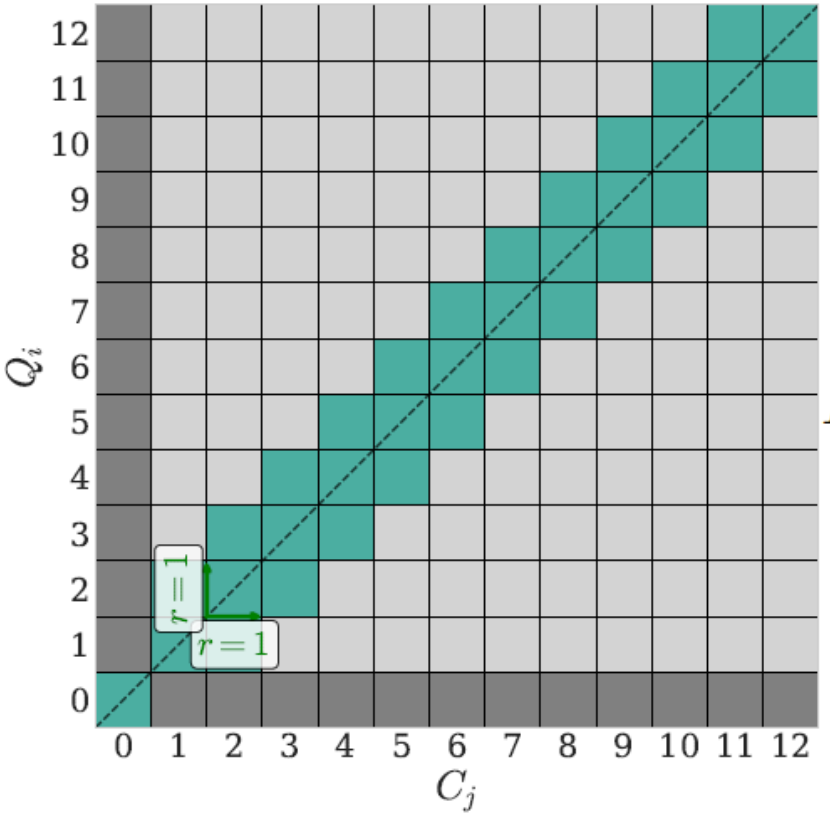


Предотвращение патологического выравнивания

- Полоса Сакоэ—Тибы
- Параллелограмм Итакуры
- Нормализация стоимости пути

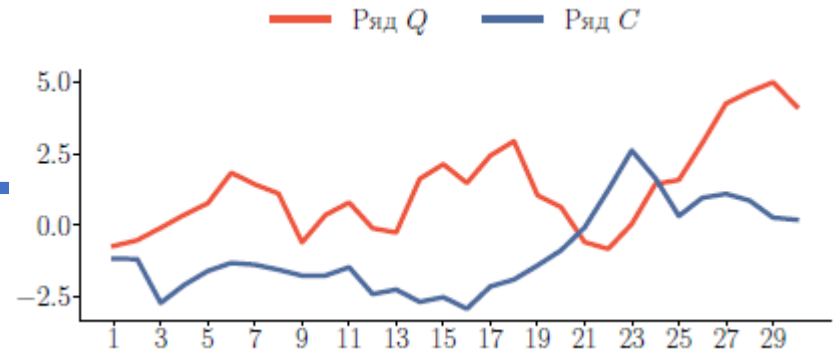
Полоса Сакоэ—Тибы

- Запрещает отклонение пути трансформации более чем на r ячеек $(0 \leq r \leq m-1)$ от главной диагонали матрицы глобальной стоимости

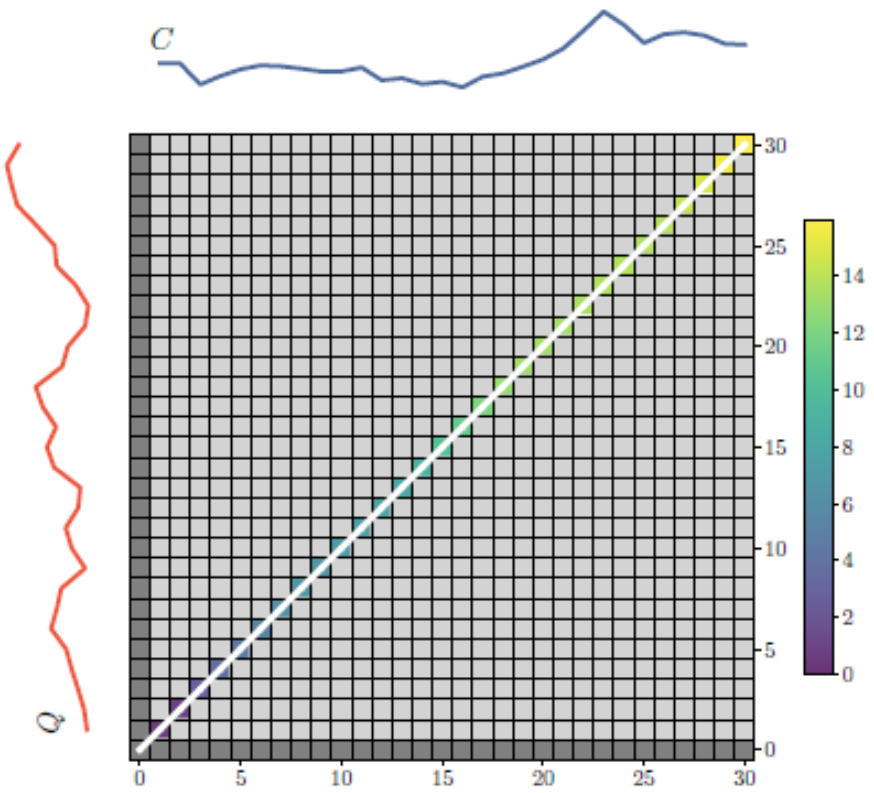


$$D(i,\,j) = \begin{cases} +\infty, & 1 \leqslant i \leqslant m, \\ & \min(m, i+r) < j < \max(1, i-r) \\ d(i,j) + \min\big(D(i-1, j-1), & \\ \qquad D(i-1, j), D(i, j-1)\big), & \text{иначе} \end{cases}$$

Вычисление DTW с полосой Сакоэ—Тибы

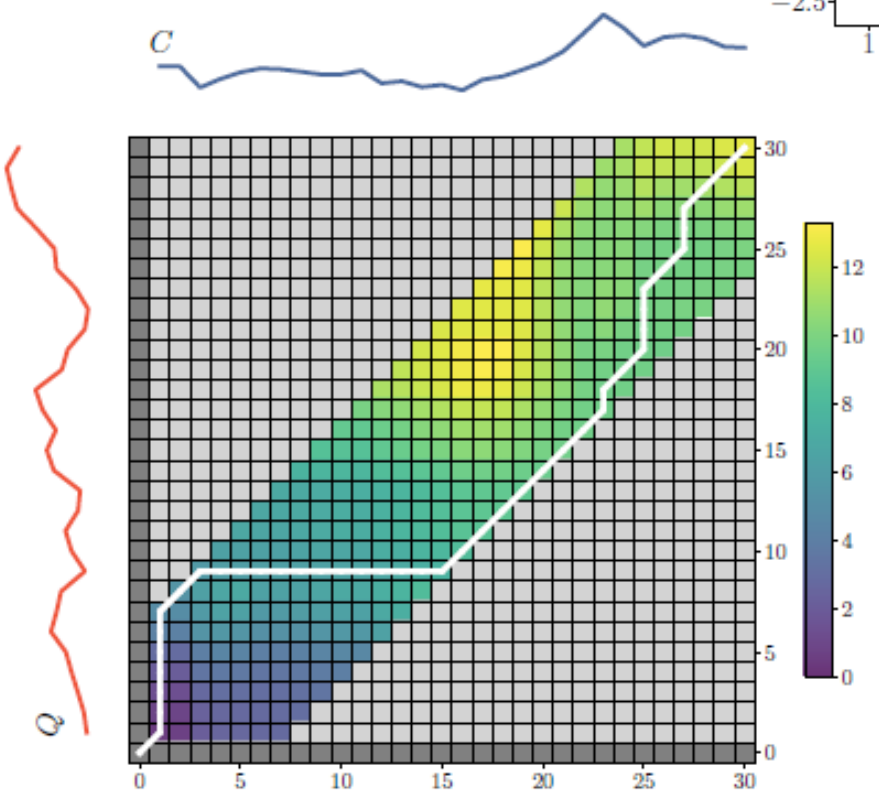


$r = 0$



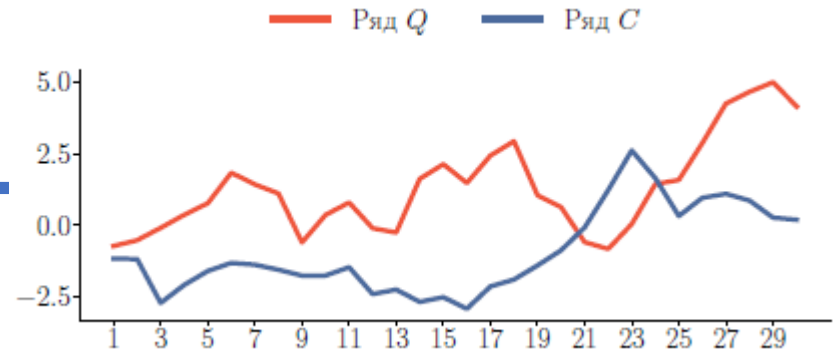
$\mathrm{DTW}(Q, C) = 15.961$

$r = 0.25m$

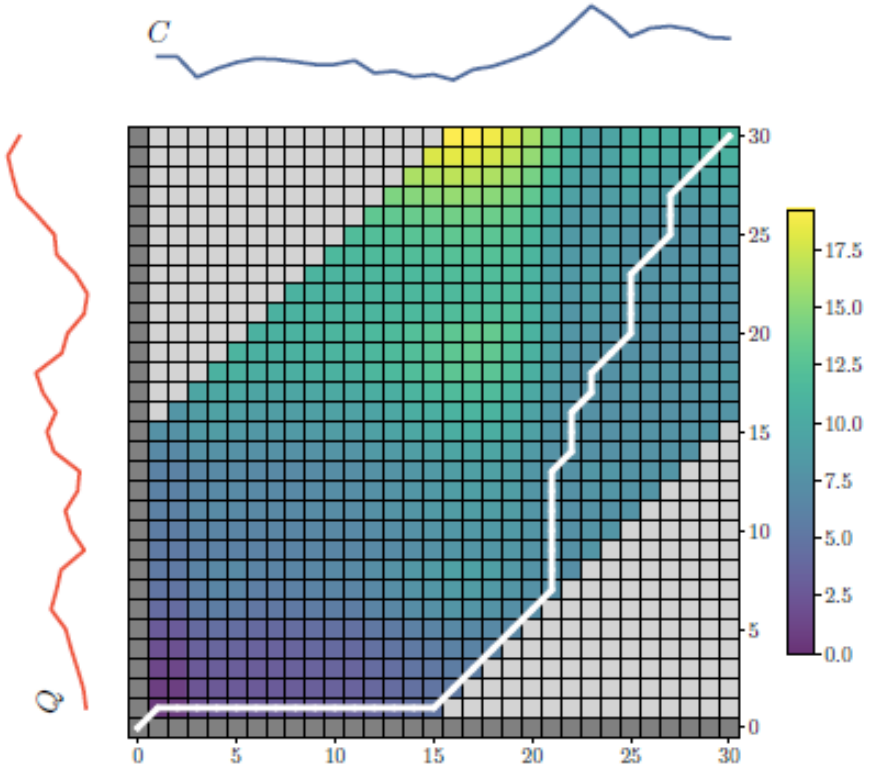


$\mathrm{DTW}(Q, C) = 12.644$

Вычисление DTW с полосой Сакоэ—Тибы

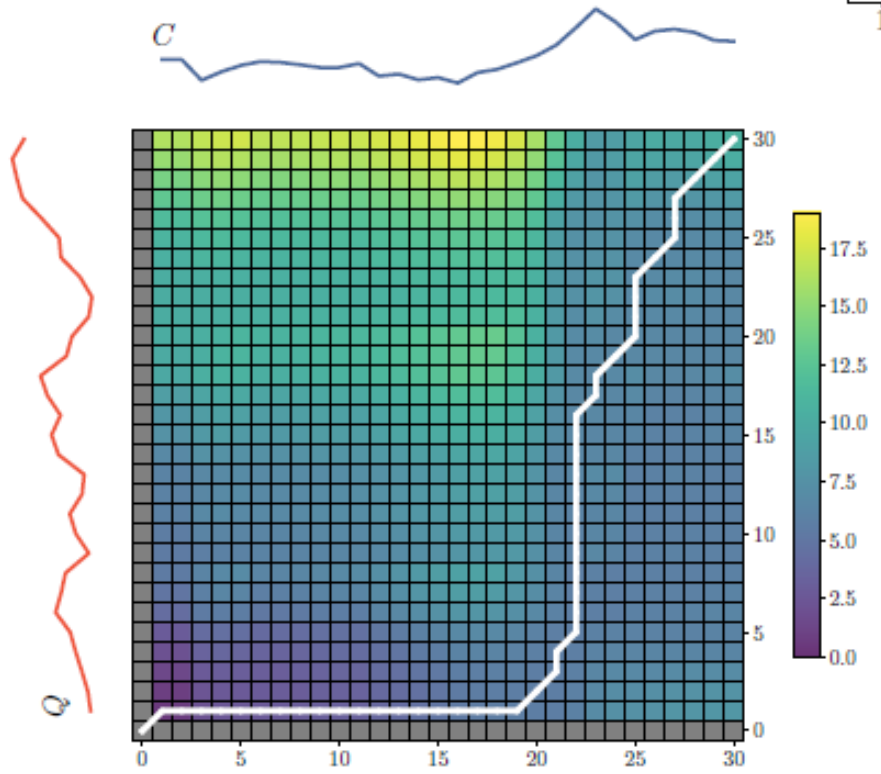


$r = 0.5m$



$\mathrm{DTW}(Q, C) = 11.182$

$r = m - 1$



$\mathrm{DTW}(Q, C) = 10.528$

Алгоритм cDTW (constraint DTW)

Вход: ряды Q, C длины m, ограничение r; $0 \leqslant r \leqslant m-1$
Выход: расстояние DTW от Q до C
Данные: матрица локальной стоимости $d \in \mathbb{R}^{m \times m}$,
матрица глобальной стоимости $D \in \mathbb{R}^{(1+m)\times(1+m)}$

1: $D(0, \cdot) \leftarrow +\infty_{1+m}$; $D(\cdot, 0) \leftarrow +\infty_{1+m}$; $D(0, 0) \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** m **do**
3: **for** $j \leftarrow \max(1, i-r)$ **to** $\min(m, i+r)$ **do**
4: $d(i, j) \leftarrow \mathrm{Cost}(q_i, c_j)$
5: $D(i, j) \leftarrow d(i, j) + \min\big(D(i-1, j-1),\ D(i-1, j),\ D(i, j-1)\big)$
6: **end for**
7: **end for**
8: **return** $\sqrt{D(m, m)}$

Алгоритм cDTW (constraint DTW)

Вход: ряды Q, C длины m, ограничение r; $0 \leqslant r \leqslant m-1$

Выход: расстояние DTW от Q до C

Данные: матрица локальной стоимости $d \in \mathbb{R}^{m \times m}$,
матрица глобальной стоимости $D \in \mathbb{R}^{(1+m)\times(1+m)}$

Пространственная сложность $\boldsymbol{O(m^2)}$

1: $D(0, \cdot) \leftarrow +\infty_{1+m}$; $D(\cdot, 0) \leftarrow +\infty_{1+m}$; $D(0, 0) \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** m **do**
3: **for** $j \leftarrow \max(1, i-r)$ **to** $\min(m, i+r)$ **do**
4: $d(i, j) \leftarrow \mathrm{Cost}(q_i, c_j)$
5: $D(i, j) \leftarrow d(i, j) + \min\big(D(i-1, j-1),\ D(i-1, j),\ D(i, j-1)\big)$
6: **end for**
7: **end for**
8: **return** $\sqrt{D(m, m)}$

Вычислительная сложность $\boldsymbol{O(rm)}$

Алгоритм cDTW (constraint DTW)

Вход: ряды Q, C длины m, ограничение r; $0 \leqslant r \leqslant m-1$
Выход: расстояние DTW от Q до C
Данные: матрица локальной стоимости $d \in \mathbb{R}^{m \times m}$,
матрица глобальной стоимости $D \in \mathbb{R}^{(1+m)\times(1+m)}$

1: $D(0, \cdot) \leftarrow +\infty_{1+m}$; $D(\cdot, 0) \leftarrow +\infty_{1+m}$; $D(0, 0) \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** m **do**
3: **for** $j \leftarrow \max(1, i-r)$ **to** $\min(m, i+r)$ **do**
4: $d(i, j) \leftarrow \mathrm{Cost}(q_i, c_j)$
5: $D(i, j) \leftarrow d(i, j) + \min\big(D(i-1, j-1), D(i-1, j), D(i, j-1)\big)$
6: **end for**
7: **end for**
8: **return** $\sqrt{D(m, m)}$

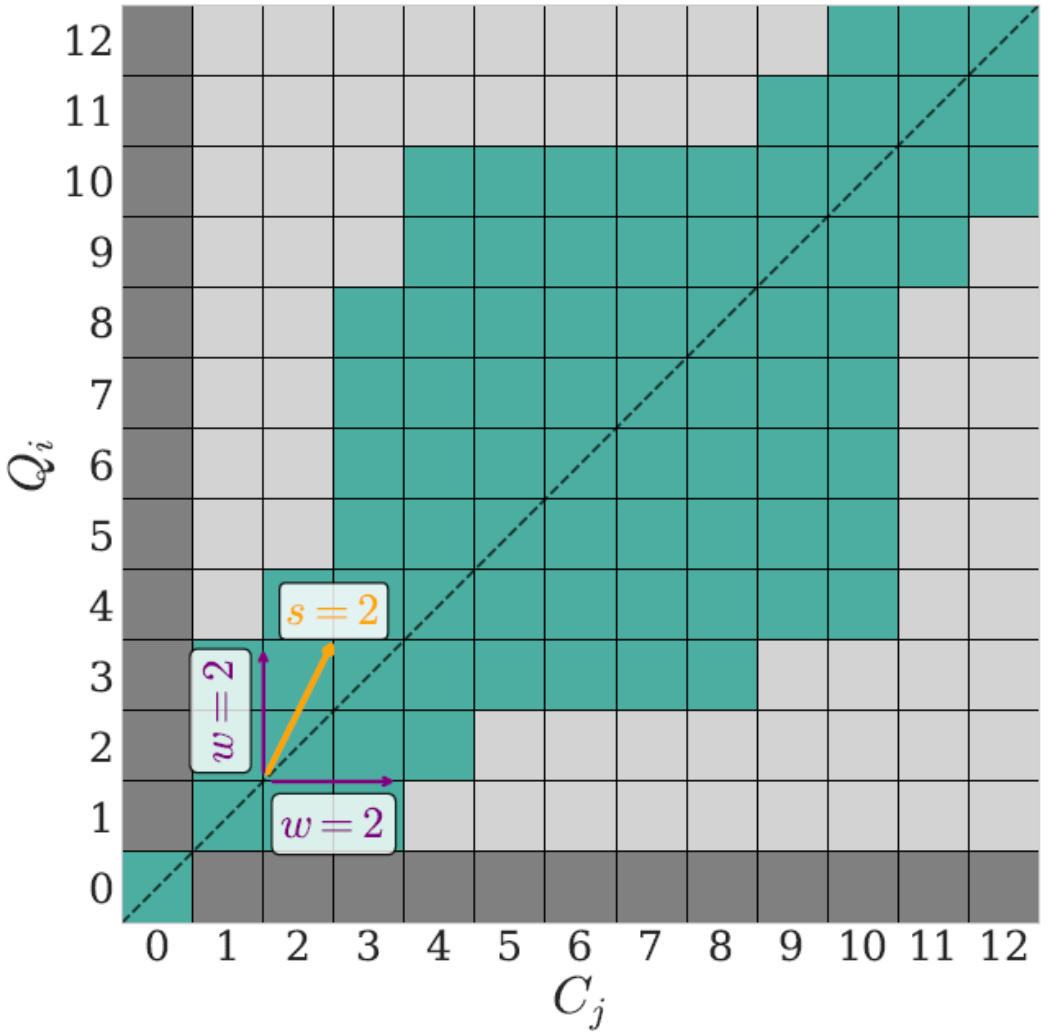
Параметр r подбирается экспериментально, нач. значение – 5-10% от m

Немного выравнивания – это хорошо, но слишком много выравнивания – это плохо*

* Ratanamahatana C.A., Keogh E.J. Three myths about Dynamic Time Warping data mining. SDM 2005. https://doi.org/10.1137/1.9781611972757.50

Параллелограмм Итакуры

- Формируется вокруг главной диагонали матрицы глобальной стоимости
  - наклон s $(1 \leq s < m)$ задает максимальное отклонение пути трансформации от диагонали по оси i или j на единицу изменения по другой оси
  - окно w $(0 \leq w < m)$ задает, насколько далеко в начале или конце может отходить путь трансформации от диагонали11

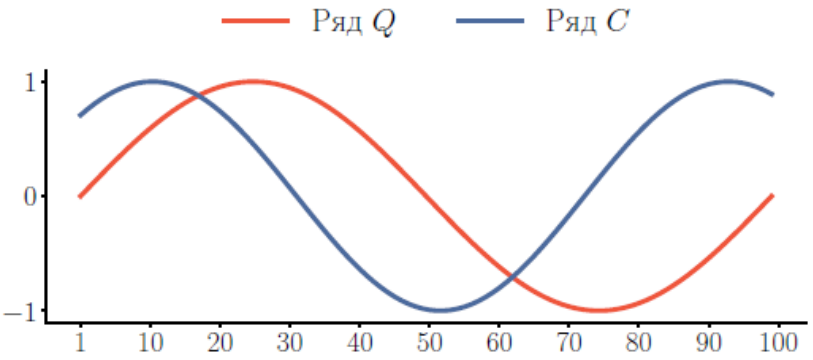


Нормализация стоимости пути

- Формула $\mathrm{Cost}_W(Q,C) = \sqrt{\sum_{k=1}^{K} d(w_k)}$ дает зависимость стоимости пути от его длины: более длинные пути стоят больше, но отвергаются, хотя могут дать лучшее выравнивание
- Формула с нормализацией стоимости относительно длины пути («средняя стоимость выравнивания рядов» на одну точку):

$$\mathrm{Cost}_W(Q,C) = \frac{1}{K}\sqrt{\sum_{k=1}^{K} d(w_k)}$$

Пример выгоды от нормализации стоимости пути



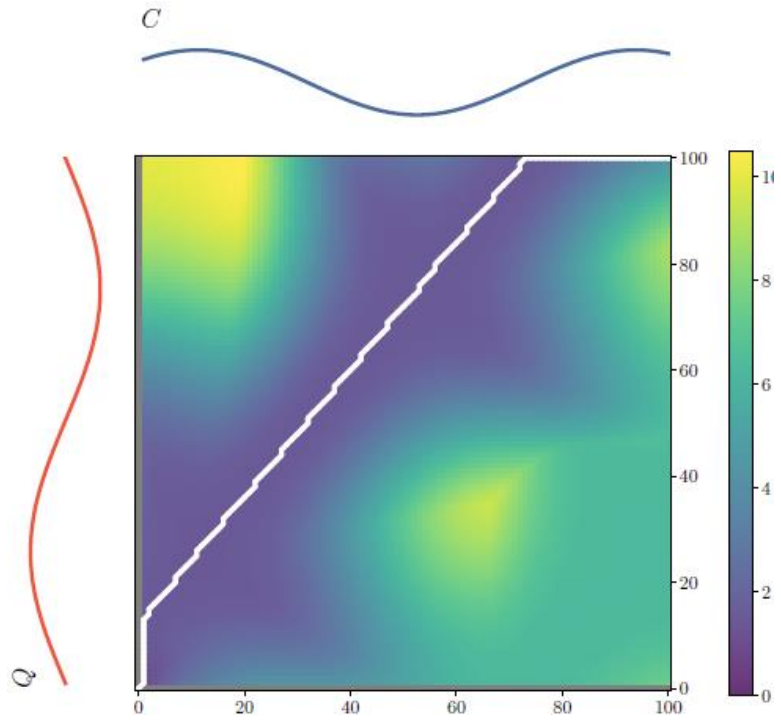
$$\mathrm{Cost}_W(Q,C) = \sqrt{\sum_{k=1}^{K} d(w_k)}$$

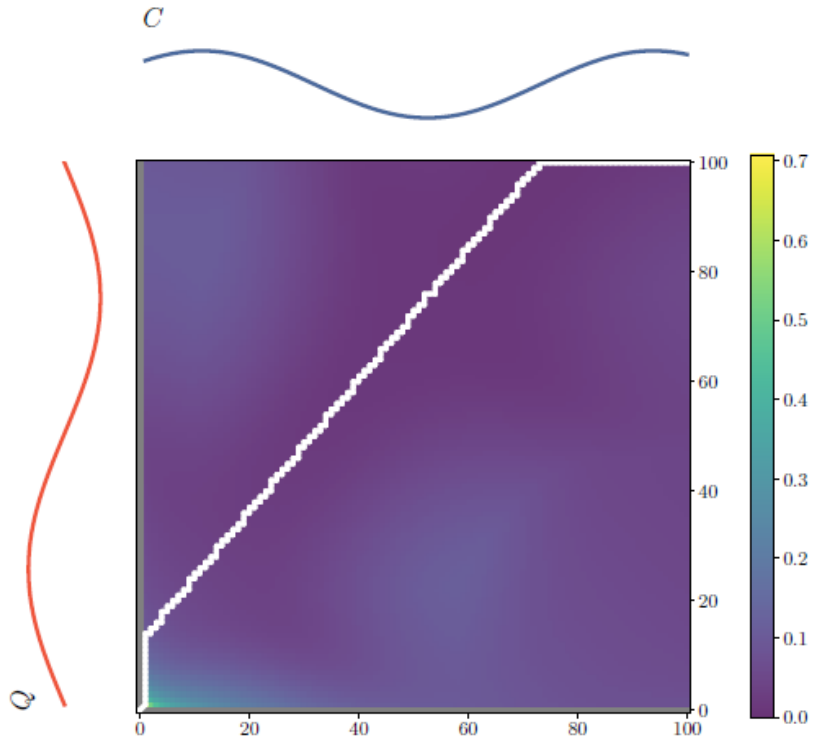
$$\mathrm{DTW}(Q,C) = \mathbf{4.318}, |W^*| = \mathbf{128}$$

$$\mathrm{Cost}_W(Q,C) = \frac{1}{K}\sqrt{\sum_{k=1}^{K} d(w_k)}$$

$$\mathrm{DTW}(Q,C) = 0.022, |W^*| = \mathbf{199}$$

$$m = 100$$
$$q_i = \sin t_i$$
$$c_i = \sin(1.2t_i + \pi/4)$$
$$t_i = 2\pi/99$$





Литература

Цымблер М.Л. Интеллектуальный анализ временных рядов: учебное пособие. Челябинск: Издательский центр ЮУрГУ, 2026. 312 с. ISBN 978-5-696-05766-8

Глава 2. Функции расстояния



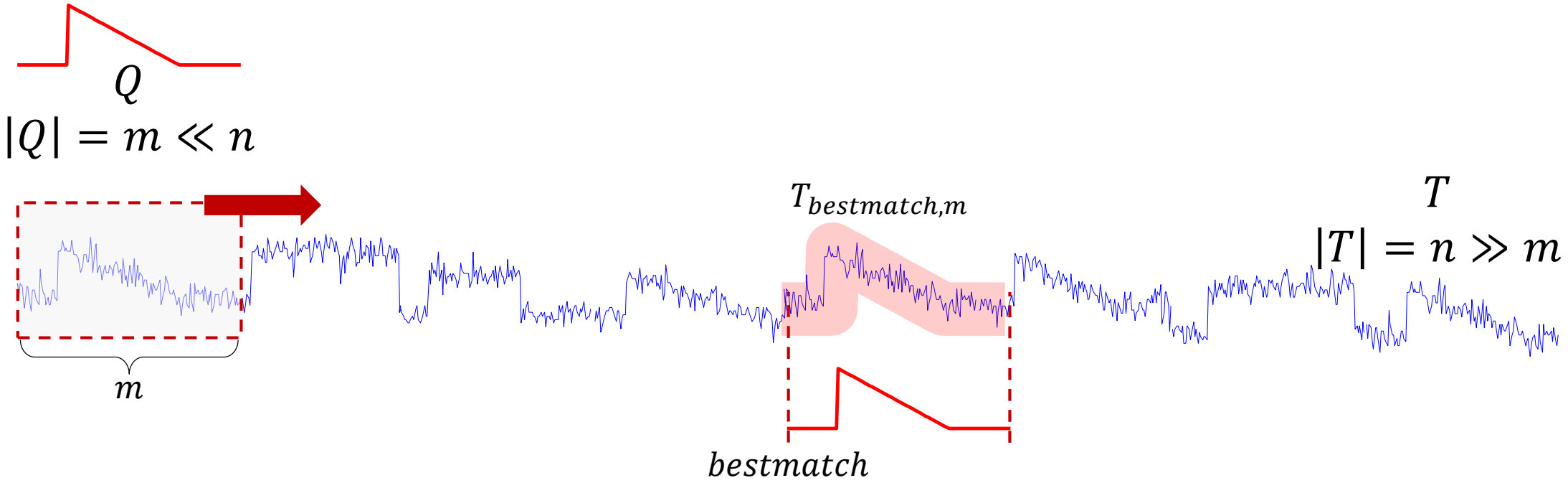
Поиск по образцу на основе евклидова расстояния



Возьми себе в образец героя древних времен, наблюдай его, иди за ним вслед, поравняйся, обгони – слава тебе!

А.В. Суворов

Поиск по образцу (subsequence matching/similarity search)



В ряде T найти подпоследовательность $T_{bestmatch,m}$, наиболее похожую на запрос Q:

$$\forall T_{i,m} \in S_T^m \;\; \mathrm{Dist}\left(T_{bestmatch,m}, Q\right) \leq \mathrm{Dist}\left(T_{i,m}, Q\right)$$

Алгоритм MASS (Mueen's Algorithm for Similarity Search)

- MASS* – в настоящее время наиболее быстрый алгоритм вычисления *z-нормализованных евклидовых расстояний* от подпоследовательности-запроса до каждой подпоследовательности временного ряда
- Применение MASS
  - Поиск шаблонов в ряде
  - Вычисление матричного профиля и поиск мотивов ряда
  - Классификация и кластеризация рядов
  - Предсказание значений ряда
  - Поиск аномалий в ряде



Абдулла Муин (Abdullah Mueen)
Проф. Университета Нью-Мексико, США

* Zhong S., Mueen A. MASS: distance profile of a query over a time series. Data Min. Knowl. Discov. 2024. Vol. 38. P. 1466–1492. DOI: 10.1007/s10618-024-01005-2.

Наивный алгоритм

Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$

Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$; $Q \leftarrow \mathrm{zNorm}(Q)$
2: **for** $i \leftarrow 1$ **to** $n - m + 1$ **do**
3: $\quad T_{i,m} \leftarrow \mathrm{zNorm}(T_{i,m})$
4: $\quad D_Q(i) \leftarrow \sqrt{\sum_{j=1}^{m}(q_j - t_{i+j-1})^2}$
5: **end for**
6: **return** D_Q

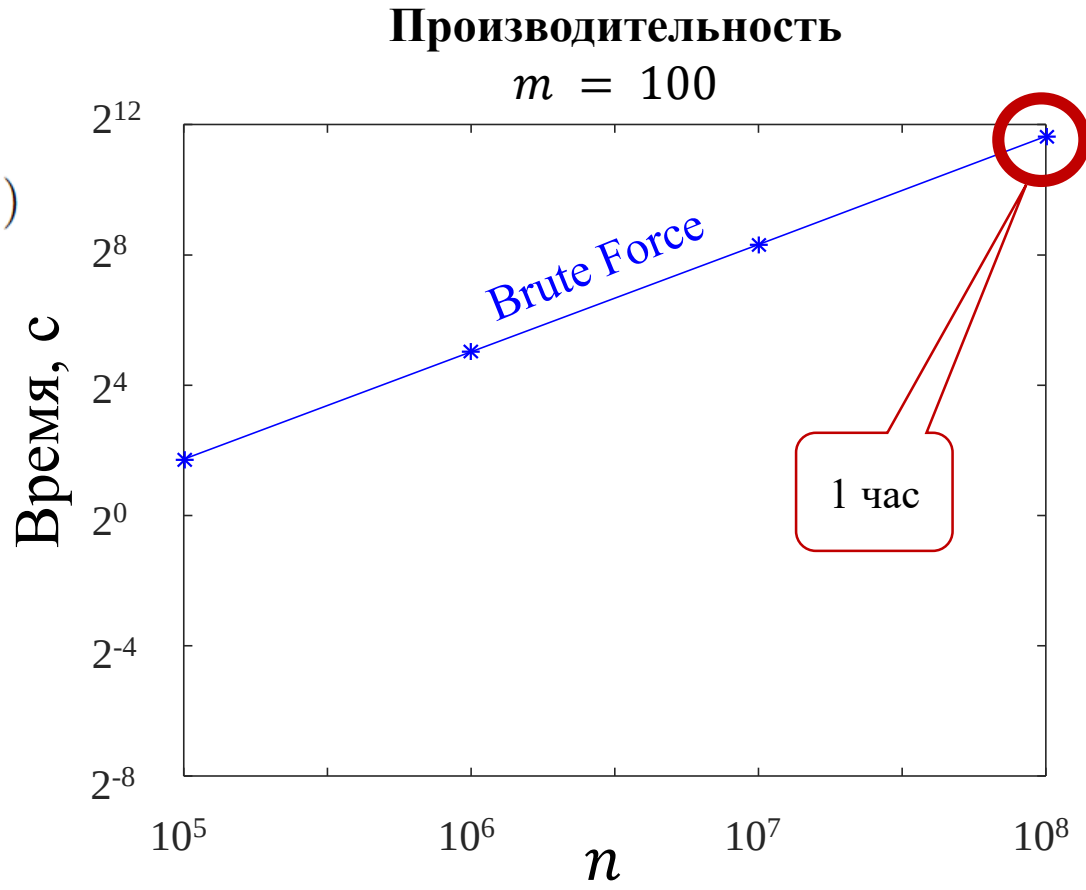
Сложность $O(nm)$,

на каждой итерации нужно $2m$ арифметических операций, т.к. подпоследовательность посещается дважды: для нормализации и для вычисления расстояния

Наивный алгоритм

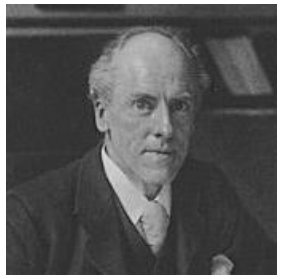
Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$
Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$; $Q \leftarrow \textsc{zNorm}(Q)$
2: **for** $i \leftarrow 1$ **to** $n - m + 1$ **do**
3: $\quad T_{i,m} \leftarrow \textsc{zNorm}(T_{i,m})$
4: $\quad D_Q(i) \leftarrow \sqrt{\sum_{j=1}^{m}(q_j - t_{i+j-1})^2}$
5: **end for**
6: **return** D_Q



Корреляция Пирсона как мера схожести

- $\mathrm{corr}(C,Q) = \frac{(\mathrm{M}_C - \mu_C)(\mathrm{M}_Q - \mu_Q)}{\sigma_C \sigma_Q} = \frac{\sum_{i=1}^{m} c_i q_i - m\mu_C\mu_Q}{m\sigma_C\sigma_Q} = \frac{\langle C,Q\rangle - m\mu_C\mu_Q}{m\sigma_C\sigma_Q}$
- $\mathrm{corr}(\cdot,\cdot) \in [-1,1]$ и потому не является метрикой
- *Достаточная статистика*
  - $\sum_{i=1}^{m} c_i q_i \quad \sum_{i=1}^{m} c_i \quad \sum_{i=1}^{m} q_i \quad \sum_{i=1}^{m} c_i^2 \quad \sum_{i=1}^{m} q_i^2$
  - Может быть вычислена за один проход
  - При наличии вычисленной достаточной статистики $\mathrm{corr}(\cdot,\cdot)$ вычисляется за $O(1)$



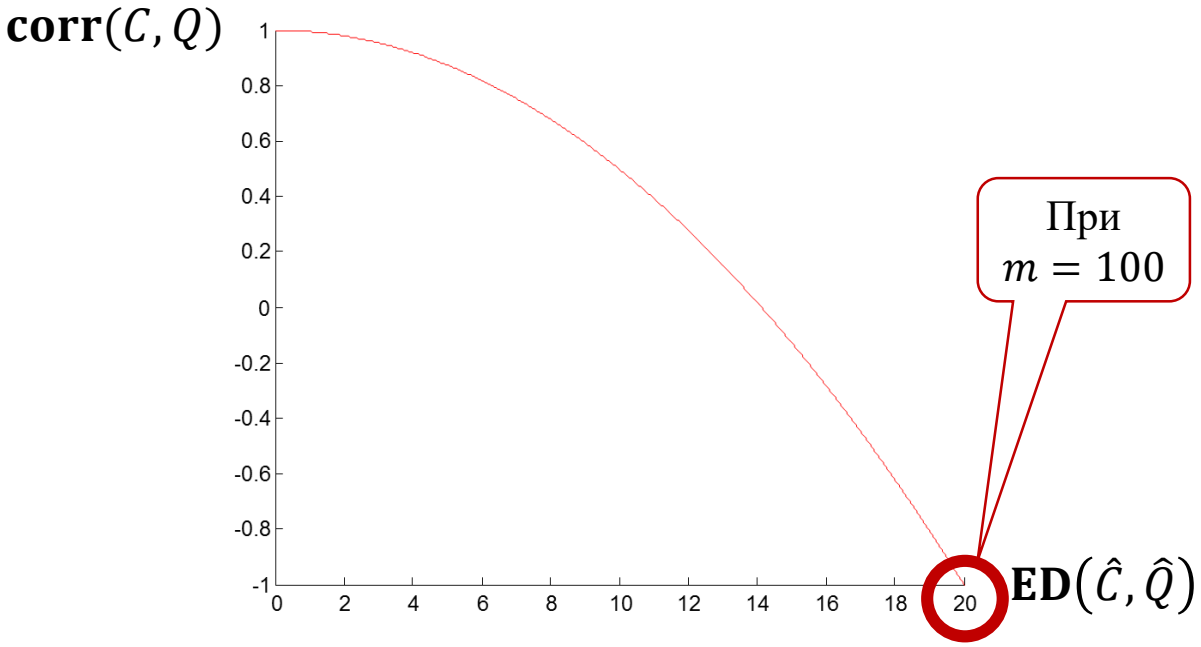
Карл Пирсон
(Karl Pearson)
1857-1936

Шкала Чеддока силы корреляции

+1.0	идеальная положительная
+0.9	очень сильная положительная
+0.7	сильная положительная
+0.4	умеренная положительная
+0.1	слабая положительная
0.0	отсутствие корреляции
−0.1	слабая отрицательная
−0.4	умеренная отрицательная
−0.7	сильная отрицательная
−0.9	очень сильная отрицательная
−1.0	идеальная отрицательная

Z-нормализация связана с корреляцией Пирсона*

- $$\mathrm{ED}(\hat{C},\hat{Q}) = \sqrt{2m\left(1-\mathrm{corr}(C,Q)\right)} = \sqrt{2m\left(1-\frac{\langle C,Q\rangle - m\mu_C\mu_Q}{m\sigma_C\sigma_Q}\right)}$$



* Mueen A., Nath S., Liu J. Fast approximate correlation for massive time-series data. Proc. of the ACM SIGMOD Int. Conf. on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010. P. 171–182. DOI: 10.1145/1807167.1807188

Улучшение алгоритма Brute Force: Нормализация «на лету»

- Можно не делать z-нормализацию на каждой итерации за счет формулы связи ED и corr
- Запрос Q предварительно z-нормализован, значит, $\boldsymbol{\mu_Q = 0}$ и $\boldsymbol{\sigma_Q = 1}$. Тогда

$$\mathrm{ED}(\hat{Q}, \hat{T}_{i,m}) = \sqrt{2m(1 - \frac{\langle Q, T_{i,m} \rangle - m\boldsymbol{\mu_Q}\mu_{T_{i,m}}}{m\boldsymbol{\sigma_Q}\sigma_{T_{i,m}}})} = \sqrt{2m(1 - \frac{\langle Q, T_{i,m} \rangle}{m\boldsymbol{\sigma_{T_{i,m}}}})}$$

- Стандартное отклонение скользящего окна, $\sigma_{T_{i,m}}$, может быть вычислено за один проход за счет применения *кумулятивных (накапливаемых) сумм*:
  - $C_i = \sum_{k=1}^{i} t_k, \quad C2_i = \sum_{k=1}^{i} t_k^2$
  - $S_i = C_{i+m} - C_i, S2_i = C2_{i+m} - C2_i$
  - $\boldsymbol{\sigma_{T_{i,m}}} = \sqrt{\frac{S2_i}{m} - \left(\frac{S_i}{m}\right)^2}$

Алгоритм JITnorm (Just In Time z-normalization)

Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$

Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$; $Q \leftarrow \mathrm{zNorm}(Q)$

2: $Sigma \leftarrow \mathrm{MovStd}(T, m)$

3: **for** $i \leftarrow 1$ **to** $n - m + 1$ **do**

4: $\quad D_Q(i) \leftarrow \sqrt{2\left(m - \sum_{j=1}^{m} \frac{t_{i+j-1} \cdot q_j}{Sigma(i)}\right)}$

5: **end for**

6: **return** D_Q

Вычисление
станд. откл.
с однократным посещением
элементов T

Сложность $O(nm)$

Алгоритм JITnorm (Just In Time z-normalization)

7: **function** MovStd

Вход: временной ряд T длины n, длина подпоследовательности m; $m \ll n$

Выход: массив значений стандартного отклонения $Sigma \in \mathbb{R}^{n-m+1}$

Данные: массивы кумулятивных сумм S, $S2 \in \mathbb{R}^{n+1}$

8: $S \leftarrow \mathbf{0}_{n+1}$; $S2 \leftarrow \mathbf{0}_{n+1}$; $Sigma \leftarrow \mathbf{0}_{n-m+1}$

9: **for** $i \leftarrow 2$ **to** $n+1$ **do**

10: $S(i) \leftarrow S(i-1) + t_{i-1}$; $S2(i) \leftarrow S(i)^2$

11: **end for**

12: **for** $i \leftarrow m+1$ **to** $n+1$ **do**

13: $j \leftarrow i - m$

14: $Sigma(j) \leftarrow \sqrt{\frac{1}{m}\big(S2(i) - S2(j)\big) - \frac{1}{m^2}\big(S(i) - S(j)\big)^2}$

15: **end for**

16: **return** $Sigma$

17: **end function**

$$\sigma_{T_{i,m}} = \sqrt{\frac{1}{m}\sum_{j=i}^{i+m-1} t_j^2 - \left(\frac{1}{m}\sum_{j=i}^{i+m-1} t_j\right)^2}$$

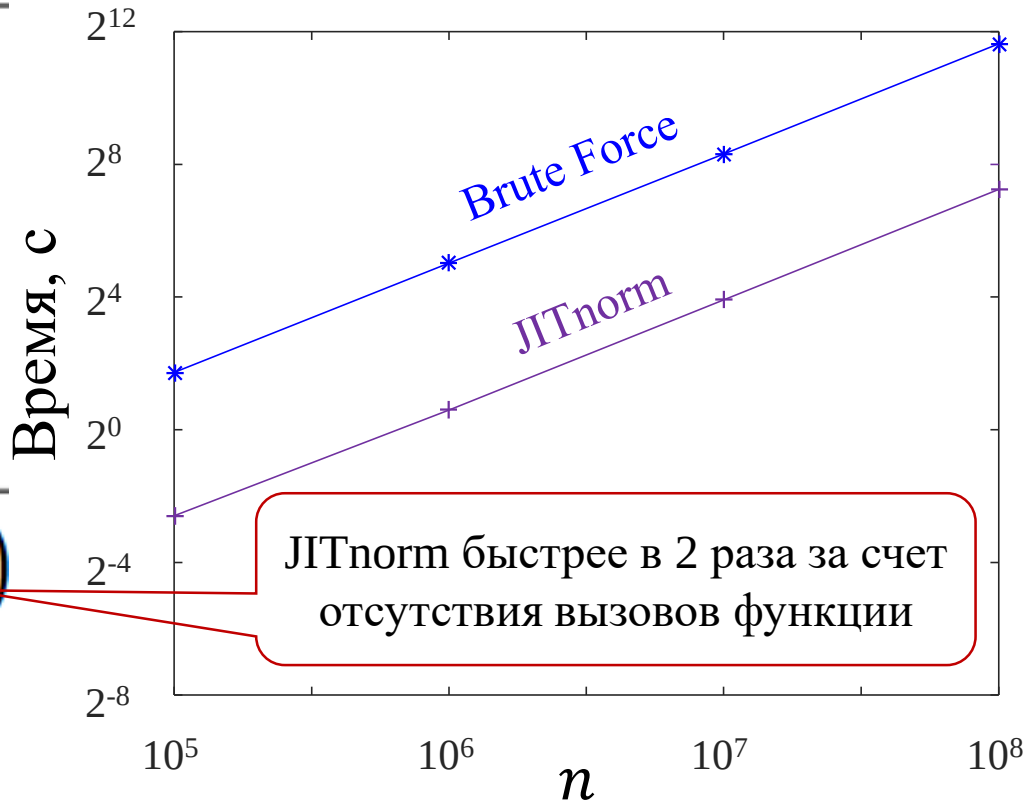
Алгоритм JITnorm (Just In Time z-normalization)

Вход: временной ряд T длины n, запро
Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-}$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$; $Q \leftarrow \text{zNorm}(Q)$
2: $Sigma \leftarrow \text{MovStd}(T, m)$
3: **for** $i \leftarrow 1$ **to** $n - m + 1$ **do**
4: $\quad D_Q(i) \leftarrow \sqrt{2\left(m - \sum_{j=1}^{m} \frac{t_{i+j-1} \cdot q_j}{Sigma(i)}\right)}$
5: **end for**
6: **return** D_Q

Сложность $O(nm)$

Производительность
$m = 100$



JITnorm быстрее в 2 раза за счет отсутствия вызовов функции

Алгоритм MASS1

Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$
Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$
Данные: массив скалярных произведений $QT \in \mathbb{R}^{n-m+1}$,
массив стандартных отклонений $Sigma \in \mathbb{R}^{n-m+1}$

Сложность $O(n \log n)$ **не зависит от** m

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$
2: $Q \leftarrow \mathrm{zNorm}(Q)$; $Q \leftarrow \mathrm{Reverse}(Q)$
3: $Sigma \leftarrow \mathrm{MovStd}(T, m)$
4: $T \leftarrow \mathrm{DblPad}(T)$; $Q \leftarrow \mathrm{DblPad}(Q)$
5: $TQconv \leftarrow \mathrm{Conv}(T, Q)$
6: $D_Q \leftarrow \mathrm{EDNormProfile}(TQconv, Sigma)$
7: **return** D_Q

Применение свертки последовательностей

Улучшение алгоритма JITnorm: Свертка последовательностей

- Можно вычислить $\langle T_{i,m}, Q \rangle$ для всех $i \in [1, n-m+1]$ за $O(n \log n)$, применяя свертку
- Если T и Q рассматривать как последовательности коэффициентов двух многочленов от одной и той же переменной, то свертка T и Q эквивалентна умножению многочленов
- $T = \{t_i\}_{i=1}^{n_T}$, $Q = \{q_i\}_{i=1}^{n_Q}$, ***свертка*** $\boldsymbol{T * Q} = \{(T*Q)_i\}_{i=0}^{n_T+n_Q-1}$, где $(T*Q)_i = \sum_{j=1}^{i} t_j q_{i-j+1}$, причем $t_j = 0$ при $j > n_T$ и $q_{i-j+1} = 0$ при $i - j + 1 > n_Q$
- Пример: $T = (t_1, t_2, t_3, t_4)$ и $Q = (q_1, q_2)$
  - $|T * Q| = 4 + 2 - 1 = 5$
  - $(T*Q)_1 = t_1 q_1$
  - $(T*Q)_2 = t_1 q_2 + t_2 q_1$
  - $(T*Q)_3 = t_2 q_2 + t_3 q_1$
  - $(T*Q)_4 = t_3 q_2 + t_4 q_1$
  - $(T*Q)_5 = t_4 q_2$

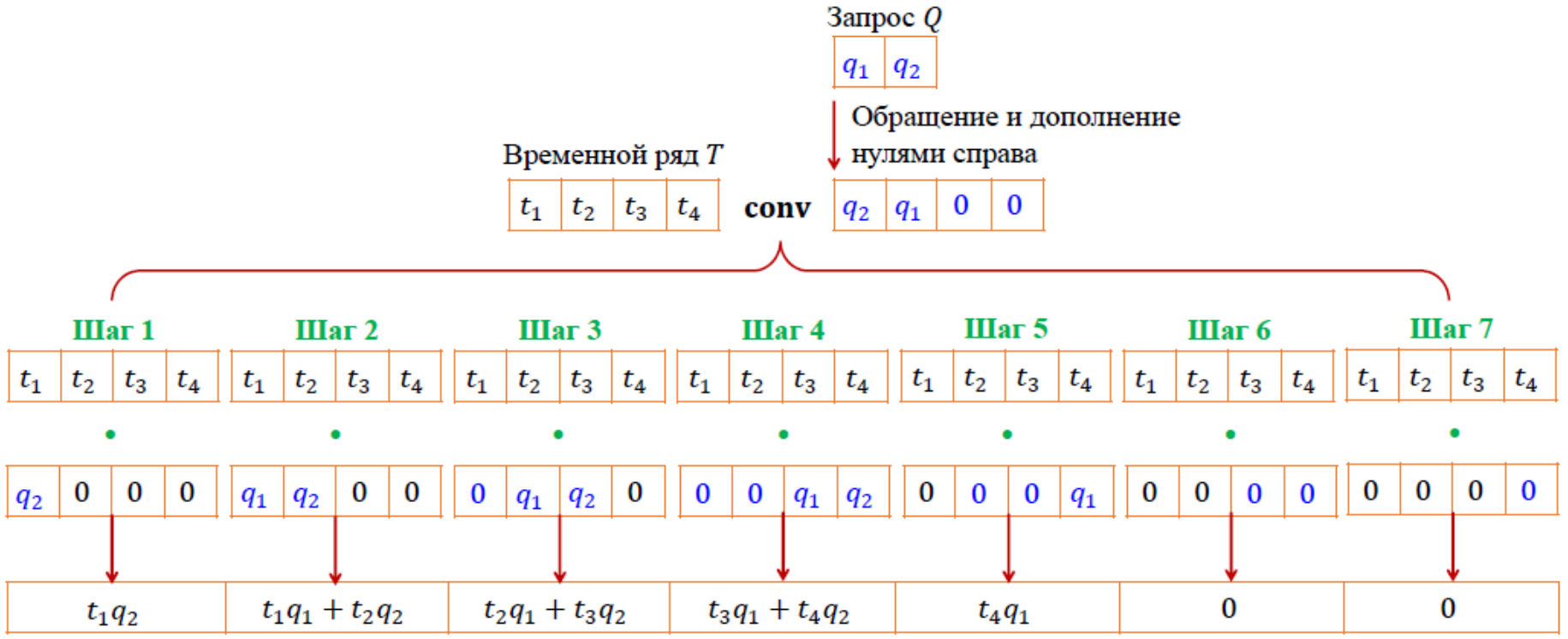
	1	2	3	4	5
$(T*Q)_1$	$t_1 q_1$				
$(T*Q)_2$	$t_1 q_2$	$t_2 q_1$			
$(T*Q)_3$	$t_1 \boldsymbol{q_3}$	$t_2 q_2$	$t_3 q_1$		
$(T*Q)_4$	$t_1 \boldsymbol{q_4}$	$t_2 \boldsymbol{q_3}$	$t_3 q_2$	$a_3 q_1$	
$(T*Q)_5$	$t_1 \boldsymbol{q_5}$	$t_2 \boldsymbol{q_4}$	$t_3 \boldsymbol{q_3}$	$a_3 q_2$	$\boldsymbol{a_4} q_1$

Решающая таблица для вычисления свертки заданных T и Q

Ячейка содержит 0
Элемент равен 0

Улучшение алгоритма JITnorm: Применение свертки

- Для вычисления скалярных произведений выполняется свертка ряда и реверс-версии запроса, которая дополнена нулями справа до длины ряда
- Пример: $T = (t_1, t_2, t_3, t_4)$ и $Q = (q_1, q_2)$,
  $\langle T_{1,2}, Q \rangle = t_1 q_1 + t_2 q_2, \quad \langle T_{2,2}, Q \rangle = t_2 q_1 + t_3 q_2, \quad \langle T_{3,2}, Q \rangle = t_3 q_1 + t_4 q_2$



Улучшение алгоритма JITnorm: Дискретное преобразование Фурье

- Свертка двух последовательностей с помощью двух ДПФ аргументов свертки и одного обратного ДПФ (ОДПФ) результата свертки
- Вычисление свертки $T * Q$
  1. Дополнение T и Q справа нулями до длины $N_{T*Q} = \mathbf{2} \cdot \max(n_T, n_Q)$:
     $T_{pad} \coloneqq \mathrm{DblPad}(T); Q_{pad} \coloneqq \mathrm{DblPad}(Q)$
  2. Выполнение ДПФ для A и B:
     $F_T \coloneqq \mathrm{DFT}(T_{pad}); F_Q \coloneqq \mathrm{DFT}(Q_{pad})$
  3. Вычисление поэлементного произведения результатов ДПФ:
     $F_{T*Q} \coloneqq F_T \circ F_Q$
  4. Выполнение ОДПФ для F_{T*Q}:
     $T * Q \coloneqq \mathrm{IDFT}(F_{T*Q}) = \{(T * Q)_i\}_{i=0}^{0.5 \cdot N_{T*Q}}$



Жан-Батист Жозеф Фурье
(Jean-Baptiste Joseph Fourier)
1768-1830

Улучшение алгоритма JITnorm: Дискретное преобразование Фурье

- $X = \{x_k\}_{k=1}^{n_X}, X_{pad} = \{x_k\}_{k=1}^{N}, N = 2n_X$
- ДПФ: $F = \mathrm{DFT}(X_{pad})$

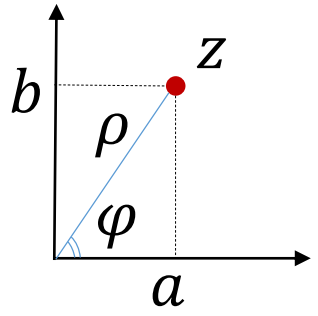
$$f_k = \sum_{\ell=1}^{N} x_\ell \cdot e^{-\boldsymbol{i}\frac{2\pi(k-1)(\ell-1)}{N}}$$

- ОДПФ: $X = \mathrm{IDFT}(X_{pad})$

$$x_k = \frac{1}{N}\sum_{\ell=1}^{N} f_\ell \cdot e^{\boldsymbol{i}\frac{2\pi(k-1)(\ell-1)}{N}}$$

Формы записи комплексного числа

$z = a + \boldsymbol{i}b, \boldsymbol{i}^2 = -1$



$$z = \rho(\cos\varphi + \boldsymbol{i}\sin\varphi),$$
$$\rho = |z| = \sqrt{a^2 + b^2},$$
$$\cos\varphi = \frac{a}{\rho}, \sin\varphi = \frac{b}{\rho}$$

$$e^z = e^a(\cos b + \sin b)$$

Улучшение алгоритма JITnorm: Быстрое преобразование Фурье

- ДПФ и ОДПФ выполняются за $O(n\log n)$ с помощью БПФ (Fast Fourier Transform)
- Открыто К. Гауссом в 1805 г. для интерполяции траекторий астероидов, опубликовано после его смерти
- Алгоритм Гуда—Томаса (1958, 1963) для БПФ в случае $n = n_1 \cdot n_2$, где n_1 и n_2 взаимно просты
- Алгоритм Кули—Тьюки (1965) для БПФ в случае произвольного n; описана реализация на ЭВМ



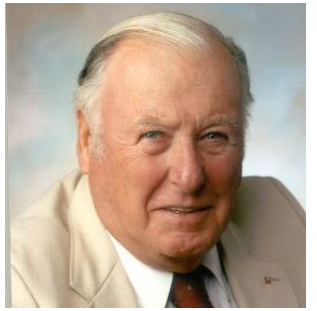
Карл Гаусс
(Johann Carl Friedrich Gauß)
1777-1855



Ирвинг Гуд
(Irving John Good)
1916-2009



Люэлин Томас
(Llewellyn Hilleth Thomas)
1903-1992



Джеймс Кули
(James William Cooley)
1926-2016



Джон Тьюки
(John Wilder Tukey)
1915-2000

Алгоритм MASS1

Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$
Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$
Данные: массив скалярных произведений $QT \in \mathbb{R}^{n-m+1}$,
массив стандартных отклонений $Sigma \in \mathbb{R}^{n-m+1}$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$
2: $Q \leftarrow \mathrm{zNorm}(Q)$; $Q \leftarrow \mathrm{Reverse}(Q)$
3: $Sigma \leftarrow \mathrm{MovStd}(T, m)$
4: $T \leftarrow \mathrm{DblPad}(T)$; $Q \leftarrow \mathrm{DblPad}(Q)$
5: $TQconv \leftarrow \mathrm{Conv}(T, Q)$
6: $D_Q \leftarrow \mathrm{EDnormProfile}(TQconv, Sigma)$
7: **return** D_Q

8: **function** Conv
Вход: временной ряд T длины n, запрос Q длины m
Выход: массив скалярных произведений $TQconv \in \mathbb{R}^{n-m+1}$,
$TQconv(i) = \langle T_{i,m}, Q \rangle$
Данные: массив коэффициентов свертки DP

9: $\mathcal{F}_T \leftarrow \mathrm{FFT}(T)$
10: $\mathcal{F}_Q \leftarrow \mathrm{FFT}(Q)$
11: $DP \leftarrow \mathcal{F}_T \circ \mathcal{F}_Q$
12: $DP \leftarrow \mathrm{IFFT}(DP)$
13: **return** $DP(m..n)$
14: **end function**
15: **function** EDnormProfile
Вход: массив скалярных произведений $TQconv$,
массив стандартных отклонений $Sigma$
Выход: массив расстояний D

16: $D \leftarrow \sqrt{2(m - \frac{TQconv}{Sigma})}$
17: **end function**

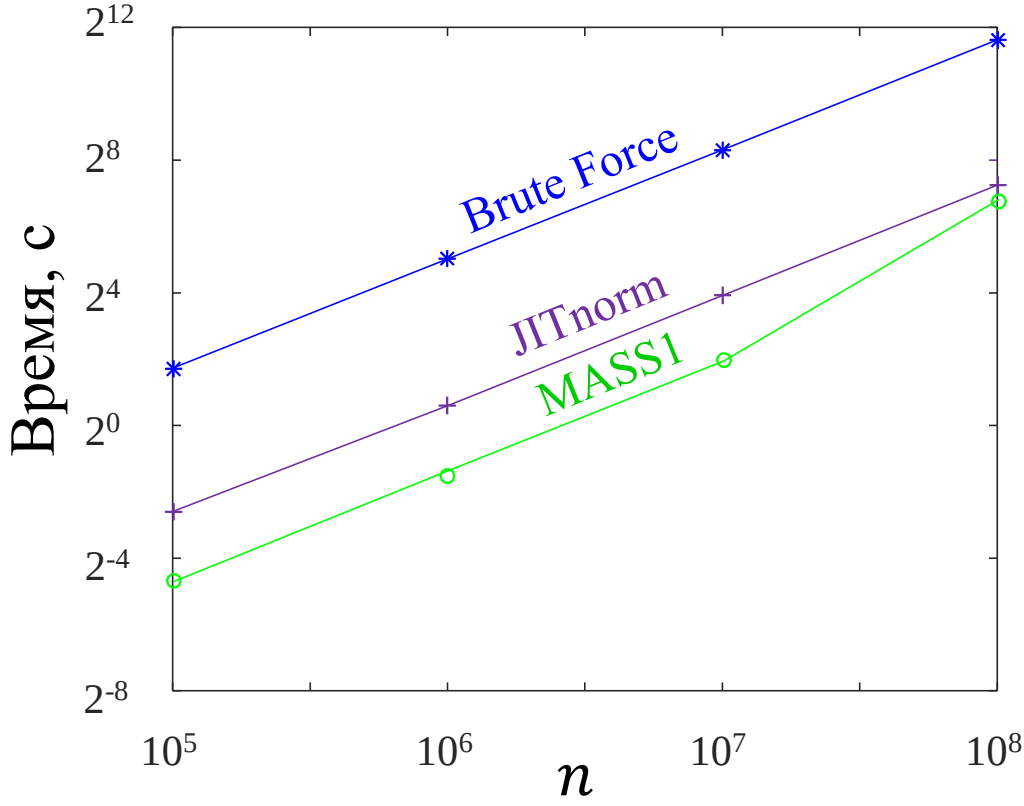
Алгоритм MASS1

Сложность $O(n \log n)$ **не зависит от** m

Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$
Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$
Данные: массив скалярных произведений $QT \in \mathbb{R}^{n-m+1}$,
массив стандартных отклонений $Sigma \in \mathbb{R}^{n-m+1}$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$
2: $Q \leftarrow \textsc{zNorm}(Q)$; $Q \leftarrow \textsc{Reverse}(Q)$
3: $Sigma \leftarrow \textsc{MovStd}(T, m)$
4: $T \leftarrow \textsc{DblPad}(T)$; $Q \leftarrow \textsc{DblPad}(Q)$
5: $TQconv \leftarrow \textsc{Conv}(T, Q)$
6: $D_Q \leftarrow \textsc{EDnormProfile}(TQconv, Sigma)$
7: **return** D_Q

Производительность
$m = 100$



Алгоритм MASS2

Вход: временной ряд T длины n, запрос Q длины m; $m \ll n$

Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$

Данные: массив скалярных произведений $TQconv \in \mathbb{R}^{n-m+1}$,
массив стандартных отклонений $Sigma \in \mathbb{R}^{n-m+1}$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$
2: $Q \leftarrow \textsc{zNorm}(Q)$; $Q \leftarrow \textsc{Reverse}(Q)$; $Q \leftarrow \textsc{Pad}(Q)$
3: $Sigma \leftarrow \textsc{MovStd}(T, m)$
4: $TQconv \leftarrow \textsc{Conv}(T, n, Q, m)$
5: $D_Q \leftarrow \textsc{EDnormProfile}(TQconv, Sigma)$
6: **return** D_Q

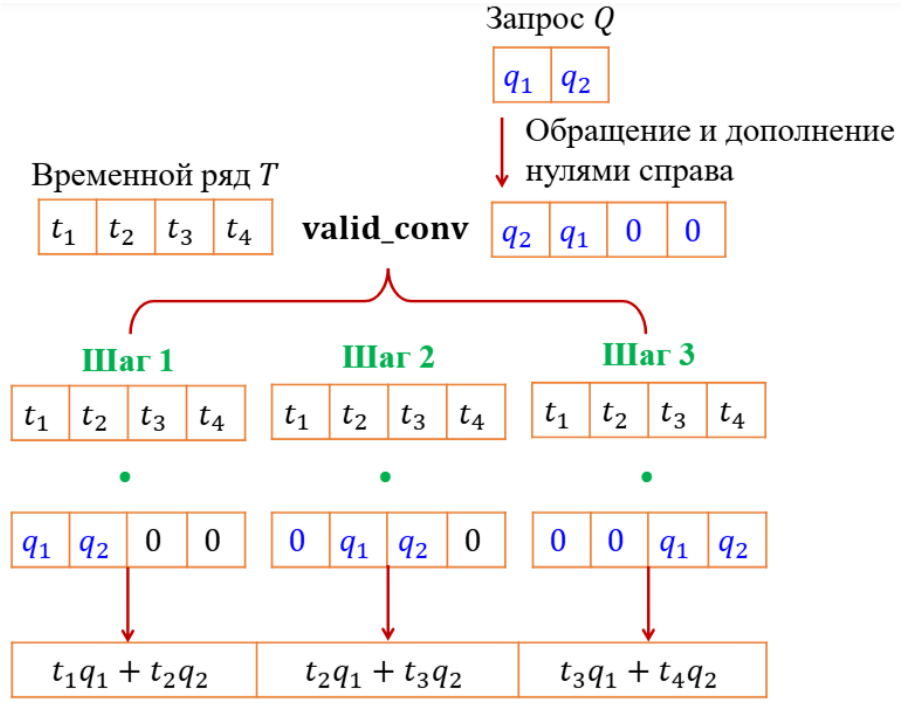
Сложность $O(n\log n)$ **не зависит от m**

К T и Q *не* применяется $DblPad$, т.к. **вычисляется лишь половина свертки**

Алгоритм MASS2: допустимая свертка

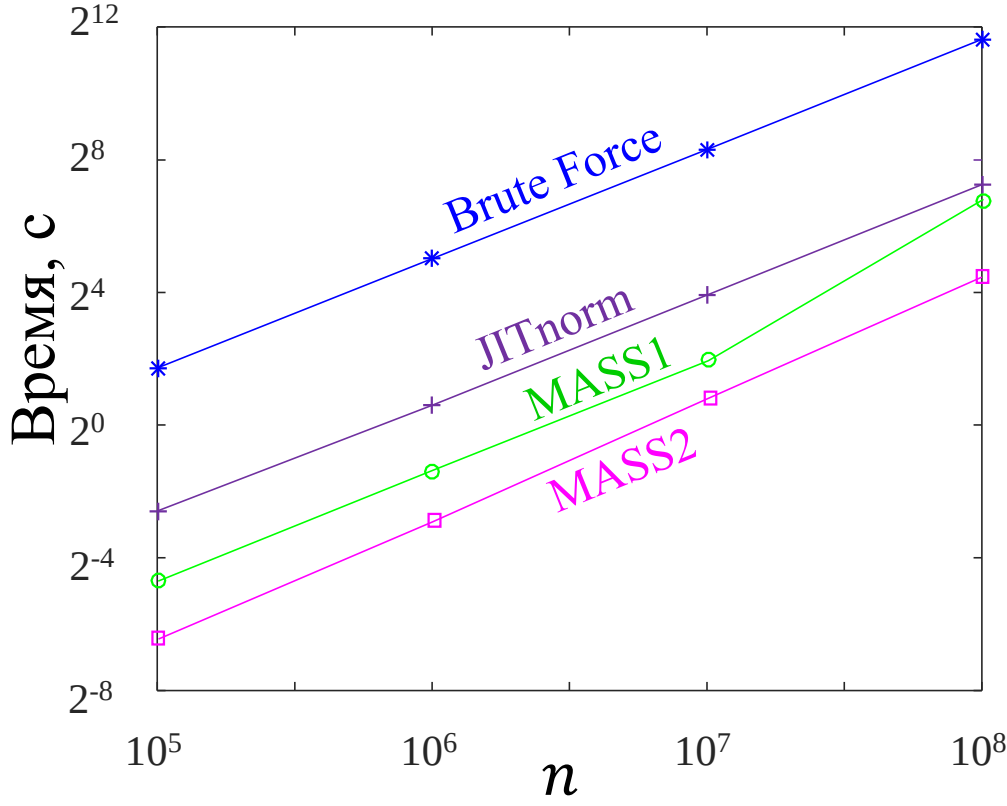
- Дополнение аргументов свертки нулями справа можно уменьшить: ряд T не дополнять, запрос Q дополнять до длины ряда

$$\text{valid_conv}(T, Q) = \text{IFFT}(\text{FFT}(T)^{\circ}\text{FFT}(\text{Pad}(Q)))$$



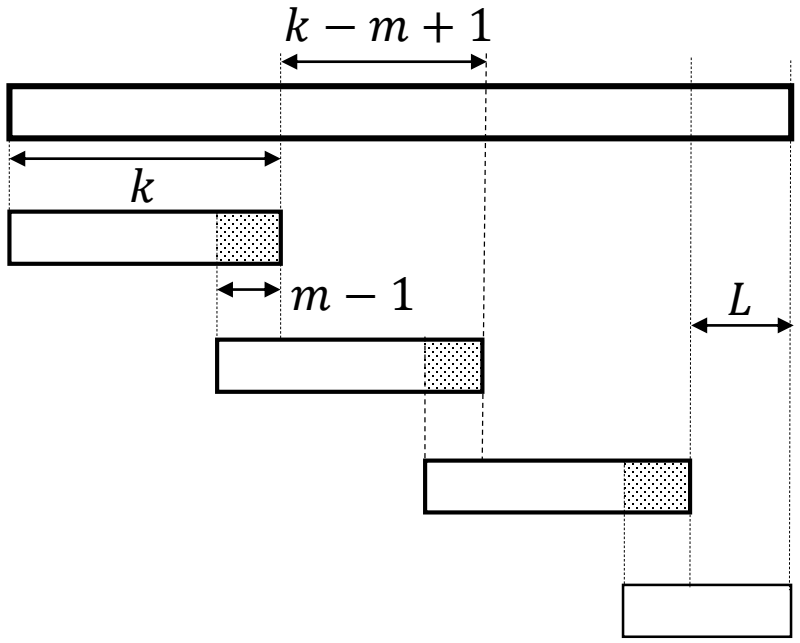
Производительность

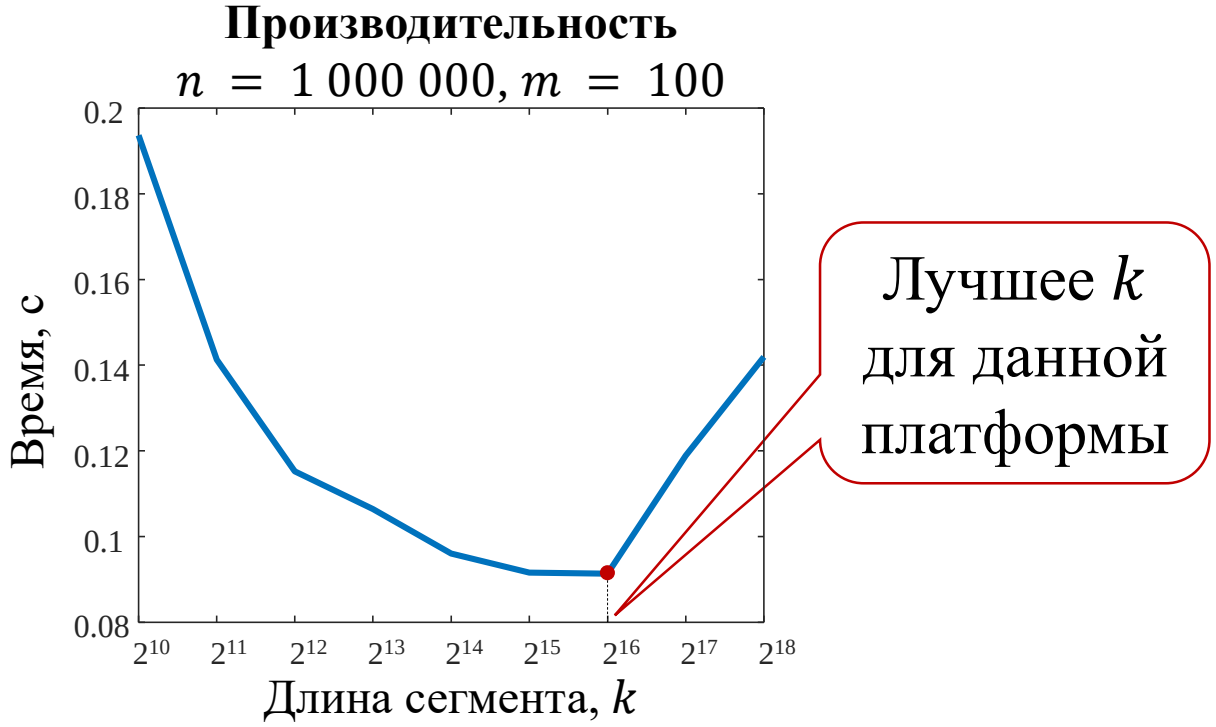
$m = 100$



Алгоритм MASS3: Сегментация ряда

- Если ряд T не помещается в памяти, применяем сегментную обработку
- Длина каждого сегмента (кроме последнего) – степень двойки (аппаратно-зависимый параметр)
- Сегменты перекрываются на $m - 1$ элементов





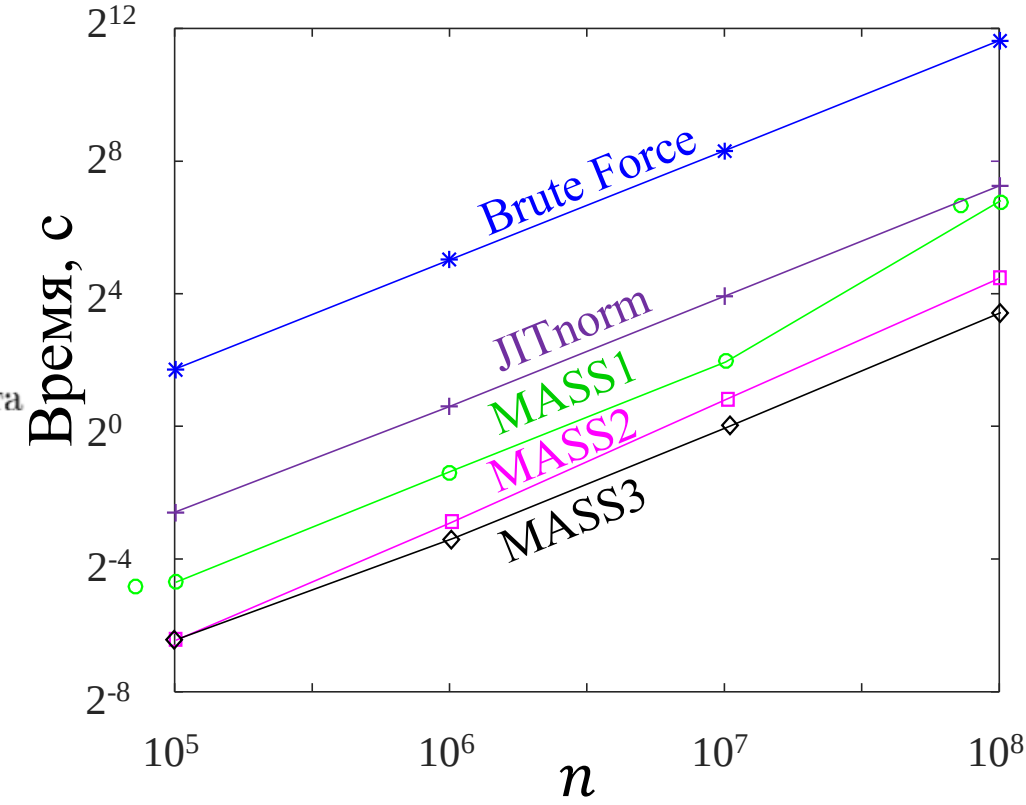
Алгоритм MASS3

Вход: временной ряд T длины n, запрос Q длины m, длина сегмента k;
$m \ll n$, $k \geqslant m$
Выход: профиль расстояния $D_Q \in \mathbb{R}^{n-m+1}$, $D_Q(i) = \hat{\mathrm{ED}}(Q, T_{i,m})$
Данные: вспомогательный ряд-сегмент $T' \in \mathbb{R}^k$,
массив скалярных произведений $TQconv \in \mathbb{R}^{n-m+1}$,
массив стандартных отклонений $Sigma \in \mathbb{R}^{n-m+1}$

1: $D_Q \leftarrow \mathbf{0}_{n-m+1}$; $T' \leftarrow \mathbf{0}_k$
2: $Q \leftarrow \mathrm{zNorm}(Q)$; $Q \leftarrow \mathrm{Reverse}(Q)$; $Q \leftarrow \mathrm{Pad}(Q)$
3: **for** $i \leftarrow 1$ **to** $n - m + 1$ **step** $k - m + 1$ **do**
4: $j \leftarrow i + k - 1$; $T' \leftarrow T_{i,j}$
5: **if** $j > n$ **then** ▷ Обработка последнего сегмента
6: $j \leftarrow n$
7: $T'_{1,k-1} \leftarrow \mathbf{0}_{k-1}$; $T'_{1,j-i} \leftarrow T_{i,j-i+1}$
8: **end if**
9: $Sigma \leftarrow \mathrm{MovStd}(T', m)$
10: $TQconv \leftarrow \mathrm{Conv}(T', j - i + 1, Q, m)$
11: $D_Q(i..j - m + 1) \leftarrow \mathrm{EDnormProfile}(TQconv, Sigma)$
12: **end for**
13: **return** D_Q

Производительность

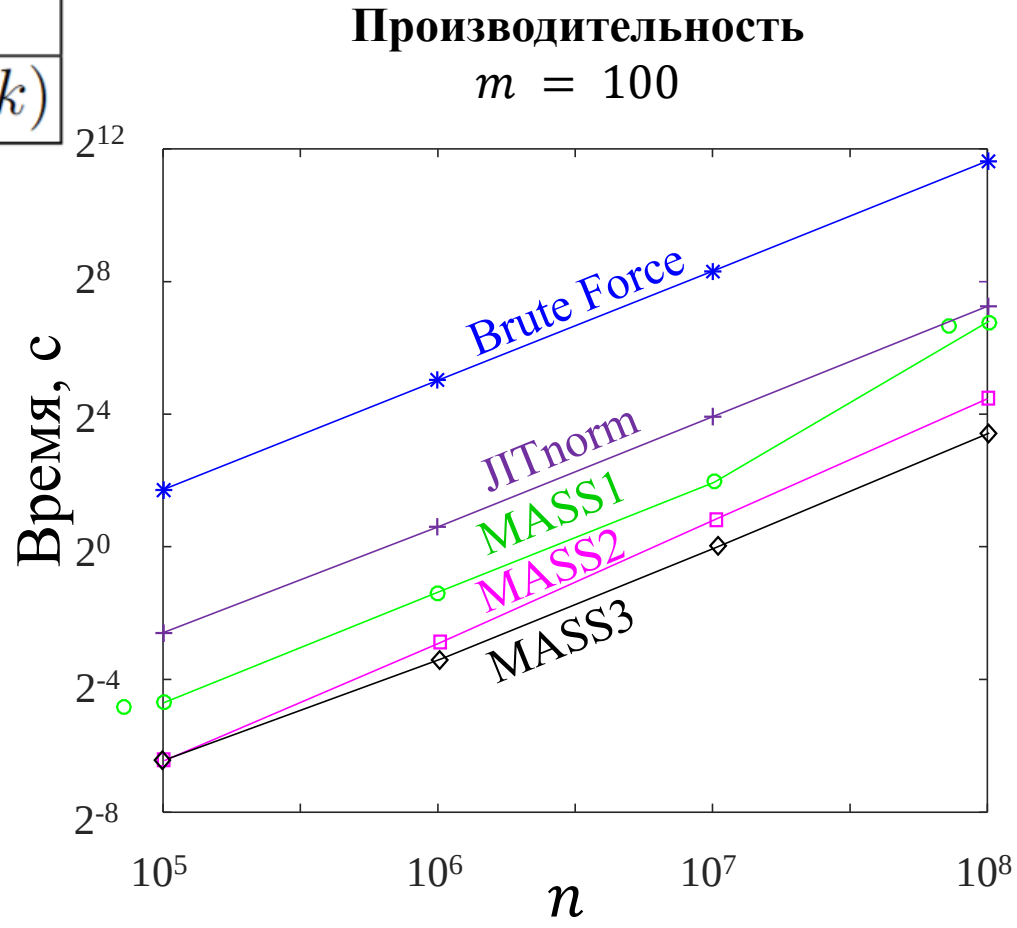
$m = 100$



Алгоритмы MASS1, MASS2, MASS3

Алгоритм	JITnorm	MASS1	MASS2	MASS3
Сложность	$O(nm)$	$O(n \log n)$	$O(n \log n)$	$O(\frac{n-k}{k-m} k \log k)$

- MASS4 – ДКТ вместо БПФ
- MASS3 – наиболее быстрый алгоритм
  - Поиск по образцу
  - Кластеризация
  - Поиск аномалий
  - Вычисление МП ряда



Литература

Цымблер М.Л. Интеллектуальный анализ временных рядов: учебное пособие. Челябинск: Издательский центр ЮУрГУ, 2026. 312 с. ISBN 978-5-696-05766-8

Глава 3. Поиск по образцу, раздел 3.1

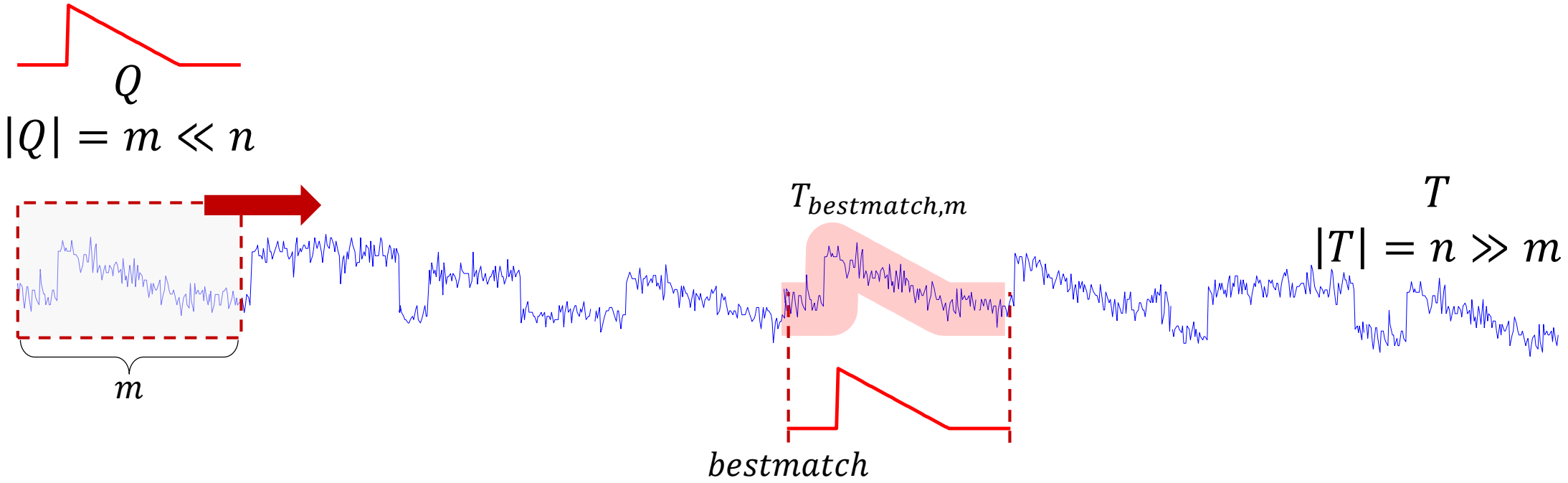
Поиск по образцу на основе динамической трансформации времени



Возьми себе в образец героя древних времен, наблюдай его, иди за ним вслед, поравняйся, обгони – слава тебе!

А.В. Суворов

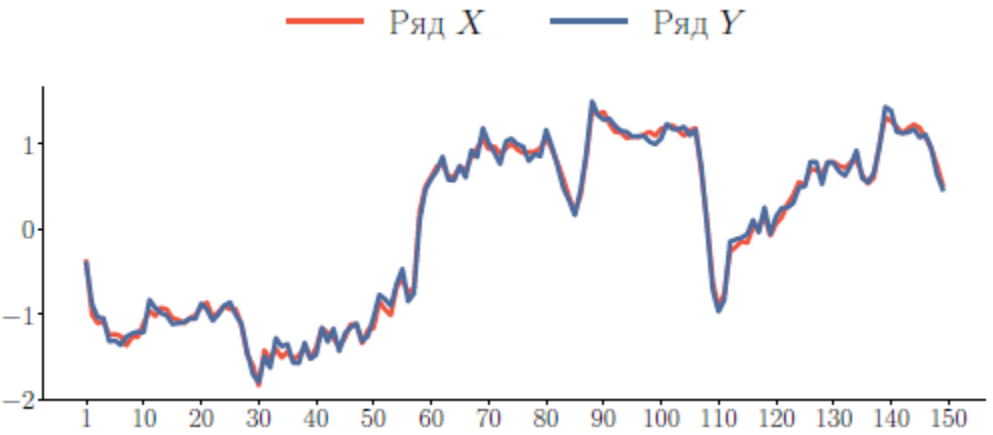
Поиск по образцу (subsequence matching/similarity search)



В ряде T найти подпоследовательность $T_{bestmatch,m}$, наиболее похожую на запрос Q:

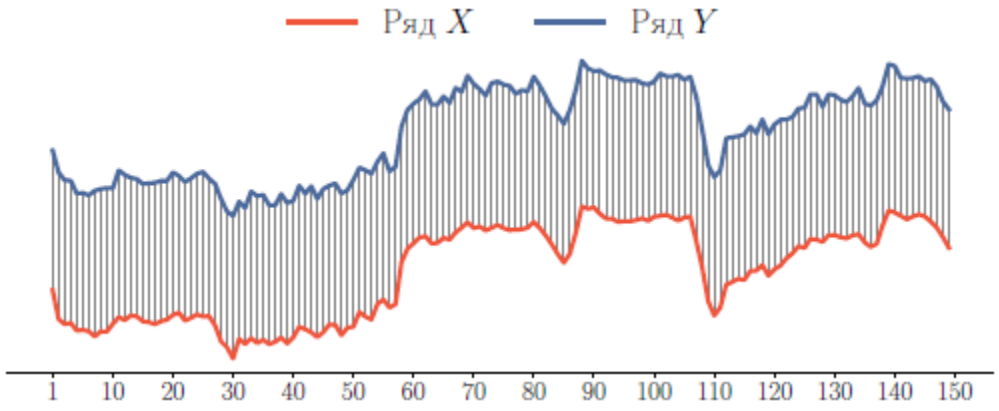
$$\forall T_{i,m} \in S_T^m \;\; \mathrm{Dist}\left(T_{bestmatch,m}, Q\right) \leq \mathrm{Dist}\left(T_{i,m}, Q\right)$$

DTW-расстояние учитывает сдвиги по времени



$\mathrm{DTW}(X, Y) = \mathbf{0.838}$

$\mathrm{ED}(X, Y) = \mathbf{0.925}$

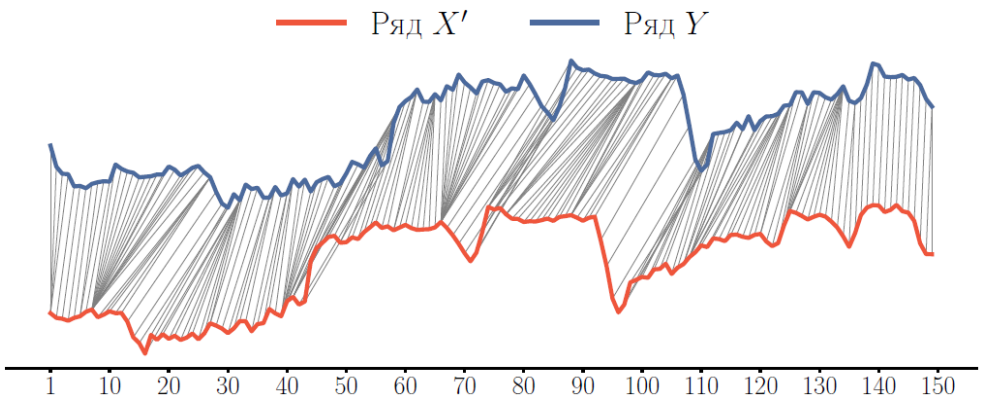


Ряд X' получен сдвигом ряда X

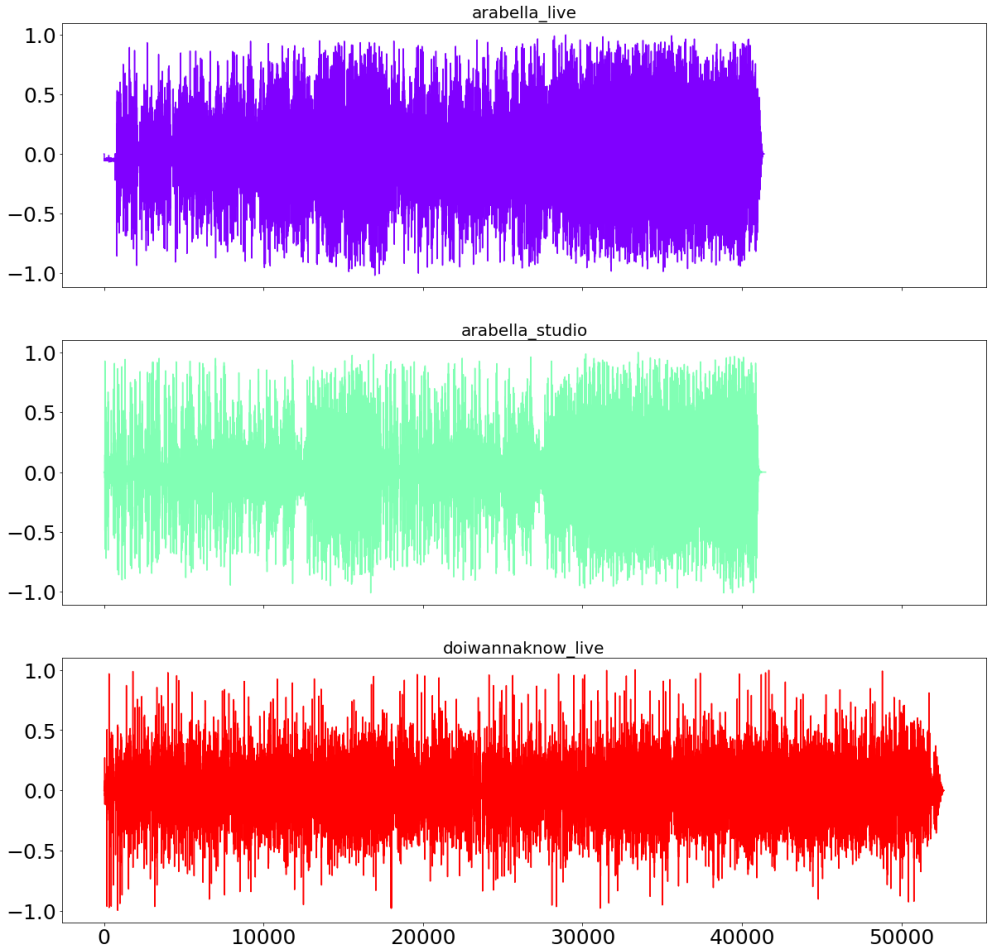


$\mathrm{DTW}(X', Y) = \mathbf{1.941}$

$\mathrm{ED}(X', Y) = \mathbf{9.513}$



Применение DTW: распознавание речи





Arabella (studio)



Arabella (live)



Do I Wanna Know (live)

ED (норм.)	Arabella (live)	Do I Wanna Know (live)
Arabella (studio)	0.043	0.038

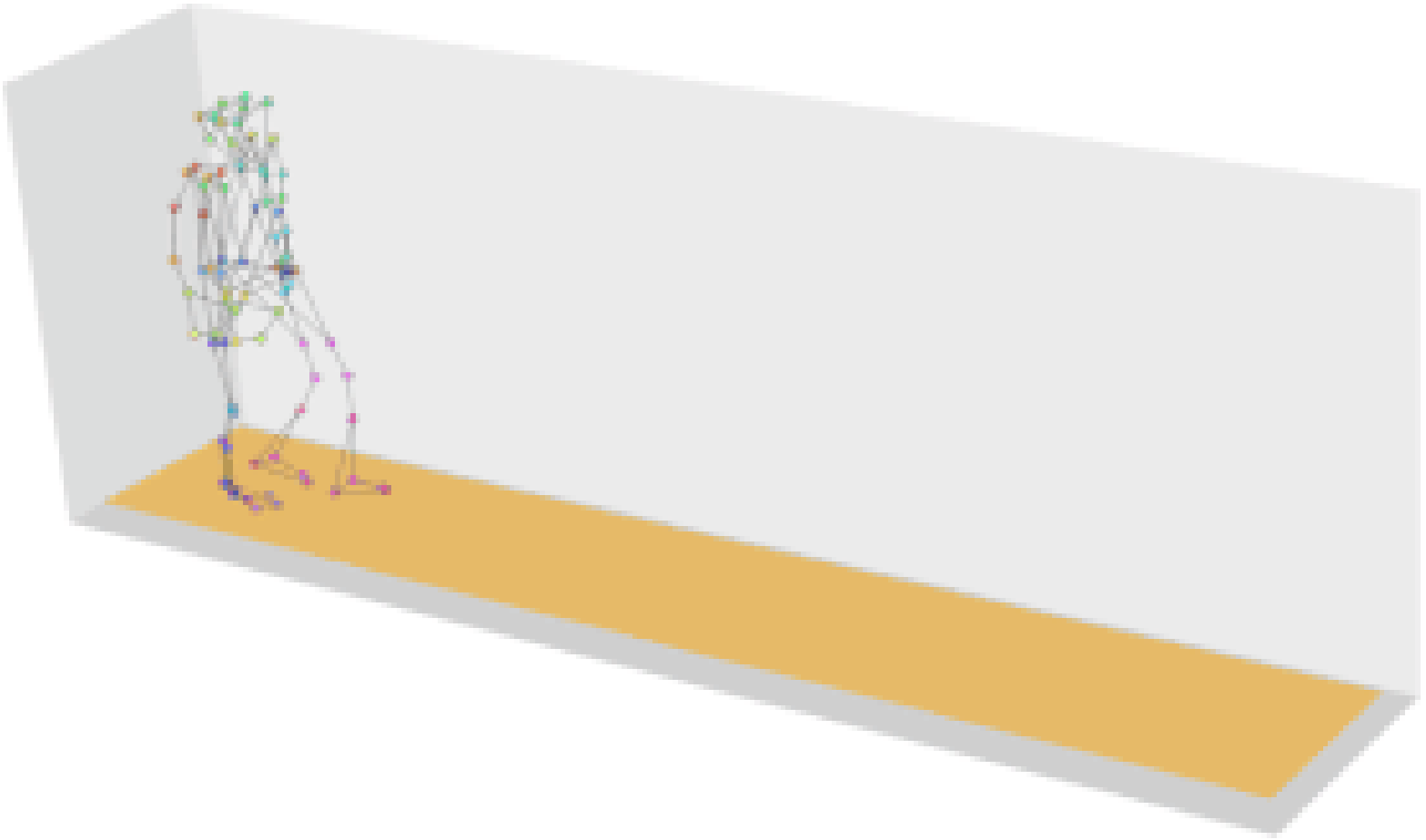
ED не отличает
разные записи разных песен!

DTW отличает
разные записи одной песни!

DTW (норм.)	Arabella (live)	Do I Wanna Know (live)
Arabella (studio)	0.82	1

Mora P. Dynamic Time Warping: Explanation and extensive testing on audio and tabular data. URL

Применение DTW: биометрия



- Две записи походки одного человека с помощью системы захвата движения
- Скорость в попытках разная, но DTW помогает понять, что траектории конечностей имеют большое сходство

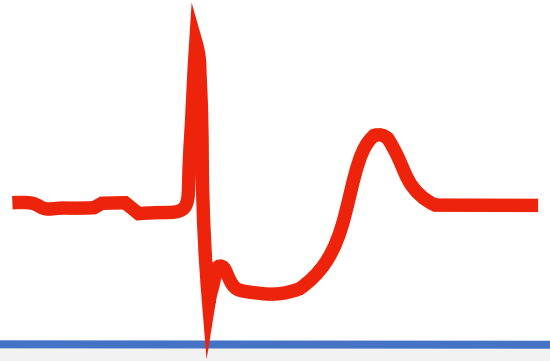
Применение DTW: медицина



Преждевременное сокращение желудочков



Инфаркт



Гиперкалиемия

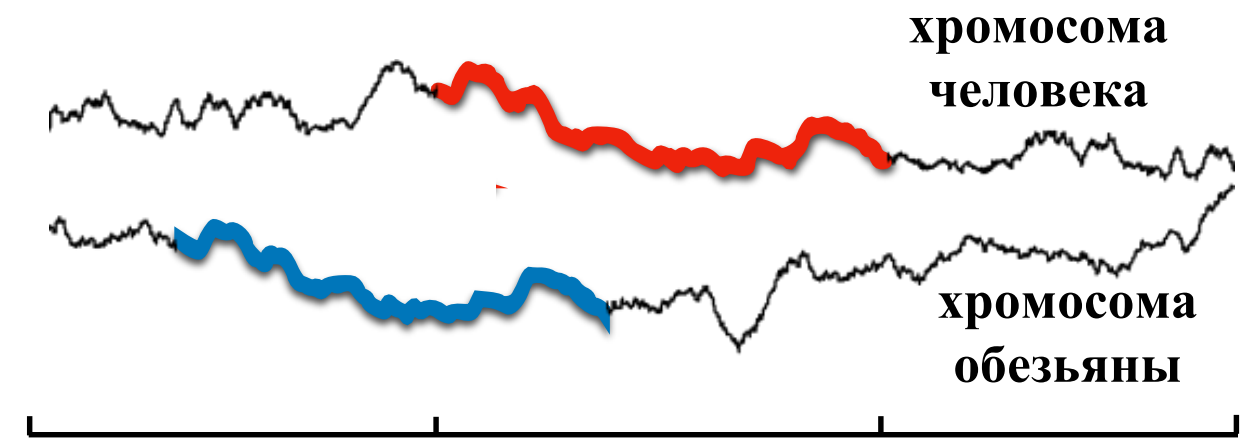


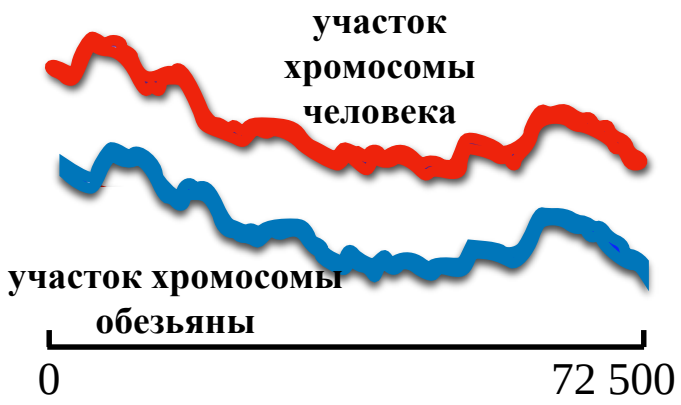
Применение DTW: генетика*

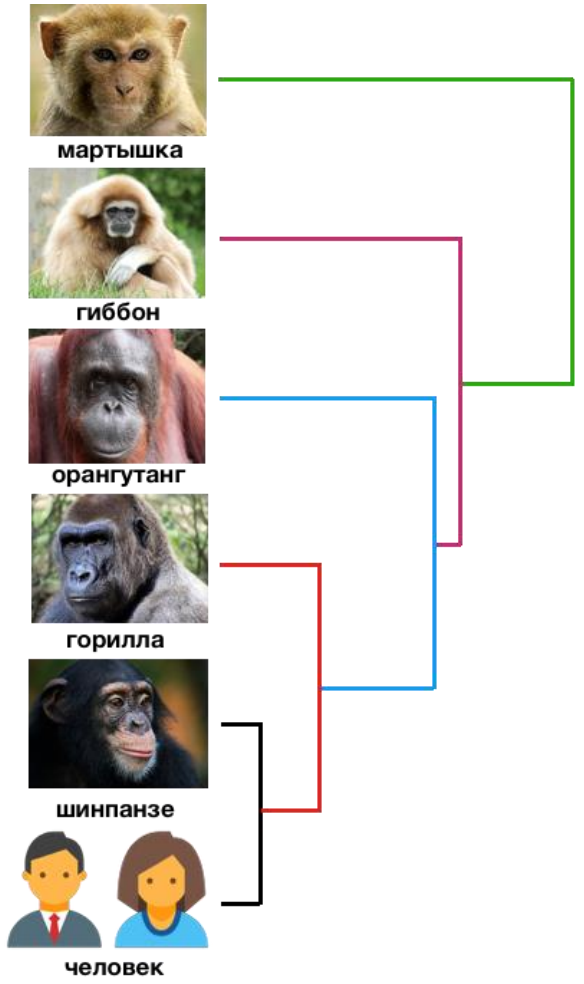
Хромосома человека:
GTCAAT…AAGAGATTTG
Хромосома обезьяны:
GGCAAT...ACAGATTTGA

Трансформация цепочки ДНК во временной ряд

$t_1 := 0$
for $i \in 1..|DNAstr|$ **do**
  case $DNAstr$ **of**
  A : $t_{i+1} := t_{i+2}$
  G : $t_{i+1} := t_{i+1}$
  C : $t_{i+1} := t_{i-1}$
  T : $t_{i+1} := t_{i-2}$
  end



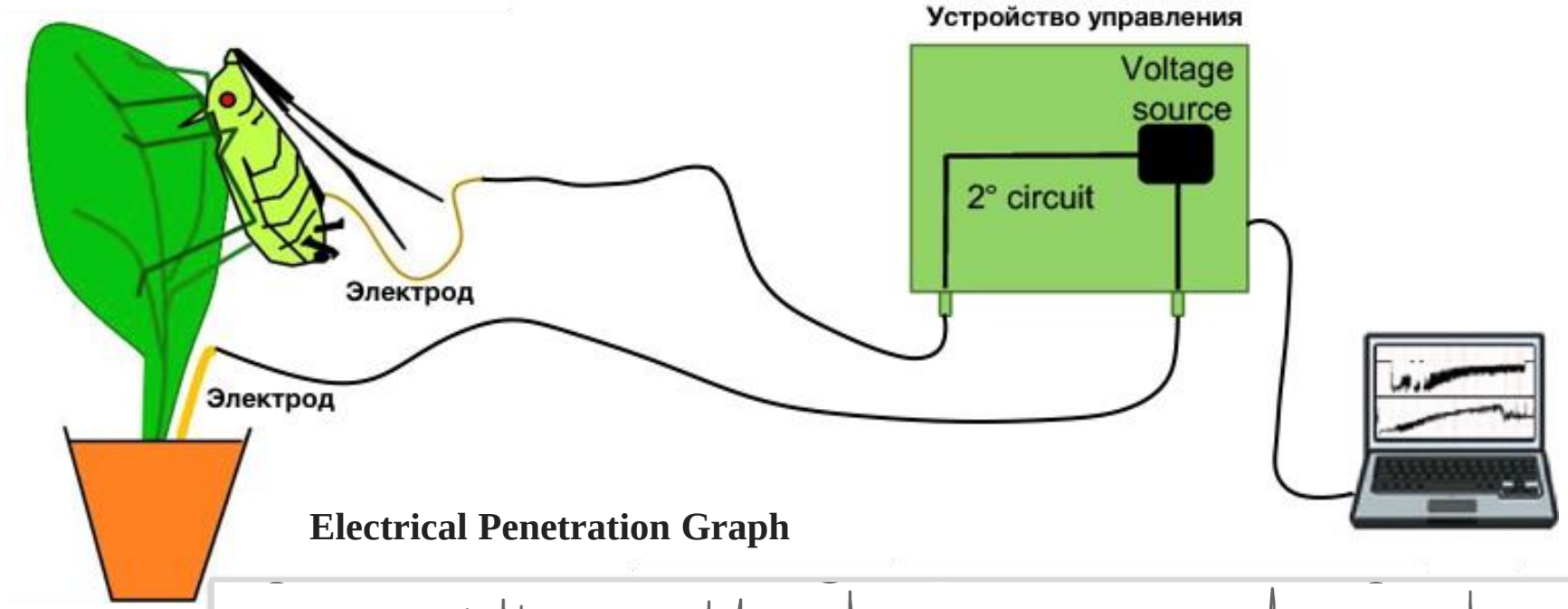




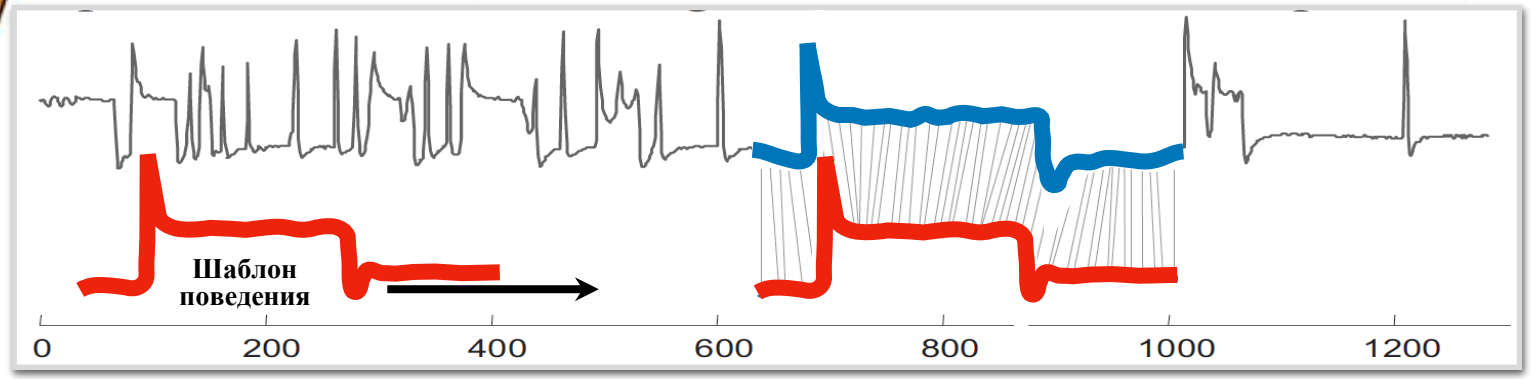
* Rakthanmanon T. *et al.* Addressing big data time series: Mining trillions of time series subsequences under Dynamic Time Warping. ACM TKDD. 2013. 7(3). 10. https://doi.org/10.1145/2500489

Применение DTW: биология



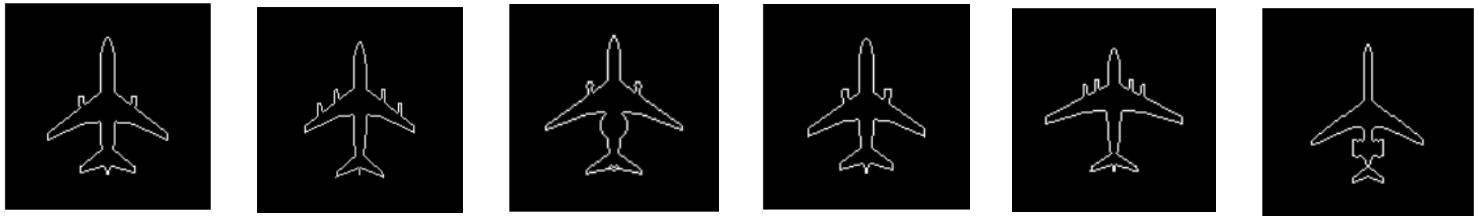


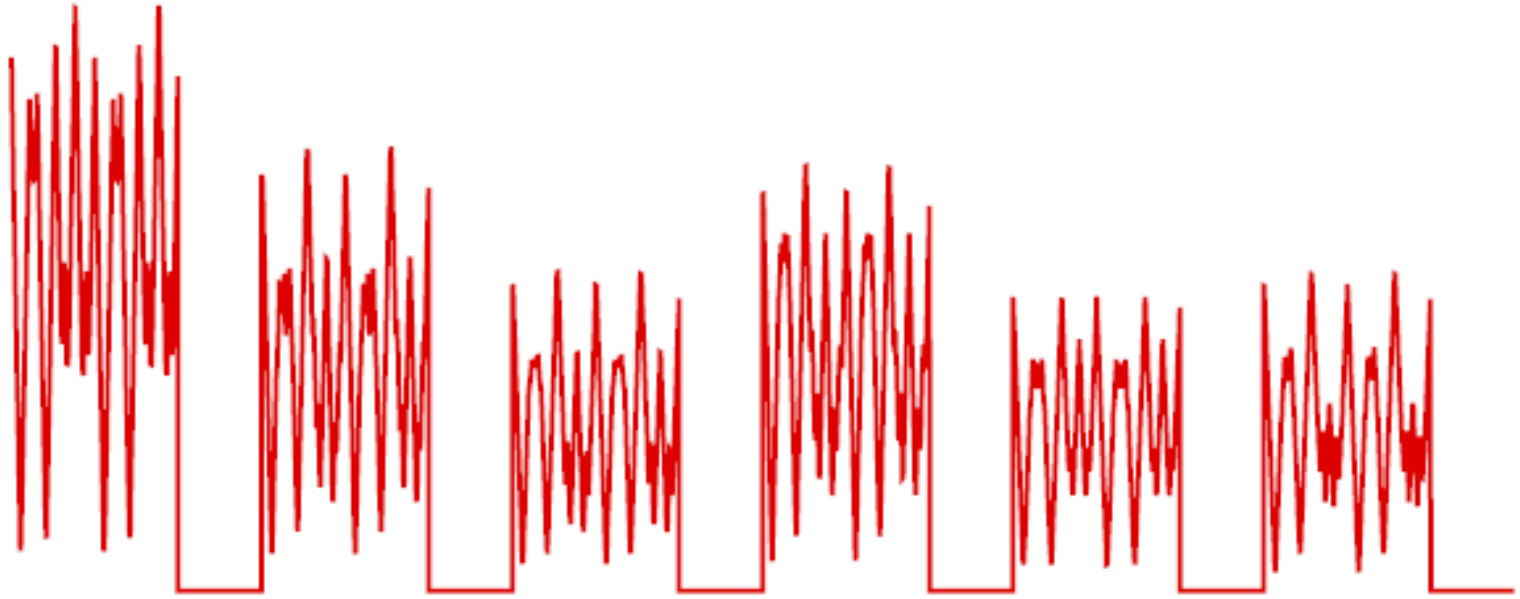
Electrical Penetration Graph



Применение DTW: военное дело

Словарь контуров

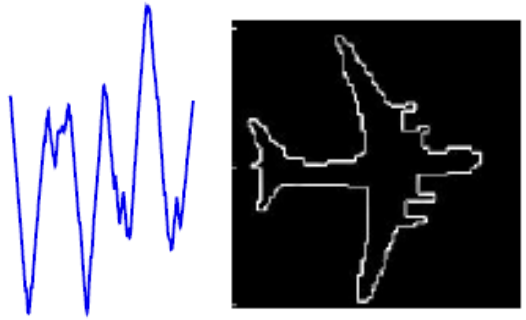




Аэрофотосъемка



Образец поиска



Алгоритм cDTW (constraint DTW)

Вход: ряды Q, C длины m, ограничение r; $0 \leqslant r \leqslant m-1$
Выход: расстояние DTW от Q до C
Данные: матрица локальной стоимости $d \in \mathbb{R}^{m \times m}$,
матрица глобальной стоимости $D \in \mathbb{R}^{(1+m)\times(1+m)}$

1: $D(0, \cdot) \leftarrow +\infty_{1+m}$; $D(\cdot, 0) \leftarrow +\infty_{1+m}$; $D(0, 0) \leftarrow 0$
2: **for** $i \leftarrow 1$ **to** m **do**
3: **for** $j \leftarrow \max(1, i-r)$ **to** $\min(m, i+r)$ **do**
4: $d(i, j) \leftarrow \mathrm{Cost}(q_i, c_j)$
5: $D(i, j) \leftarrow d(i, j) + \min\big(D(i-1, j-1), D(i-1, j), D(i, j-1)\big)$
6: **end for**
7: **end for**
8: **return** $\sqrt{D(m, m)}$

Вычислительная сложность $\boldsymbol{O(mr)}$

Наивный поиск на основе DTW

Вход: временной ряд T длины n, запрос Q длины m, $m \ll n$;
радиус полосы Сакоэ Тибы r, $0 \leqslant r \leqslant m-1$

Выход: наиболее похожая подпоследовательность $C_{\text{bestmatch}}$,
расстояние до запроса bsf

1: $bsf \leftarrow +\infty$
2: **for** $i \leftarrow 1$ **to** $n - m + 1$ **do**
3: $dist \leftarrow \text{cDTW}(Q, T_{i,m}, r)$
4: **if** $dist < bsf$ **then**
5: $bsf \leftarrow dist$; $C_{\text{bestmatch}} \leftarrow T_{i,m}$
6: **end if**
7: **end for**
8: **return** $\{C_{\text{bestmatch}}, bsf\}$

Вычислительная сложность
$\boldsymbol{O(nmr)}$

Неравенство треугольника не работает,
$\text{DTW}(Q, C_i)$ нельзя применить
в качестве нижней границы

Нижняя граница (lower bound)

- Симметричная неотрицательная функция **LB**: $\mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ со сложностью меньше $O(m^2)$

$$\forall T_{i,m}, \forall Q\colon \mathbf{LB}\left(Q, T_{i,m}\right) \le \mathrm{DTW}^2(Q, T_{i,m})$$

$$\forall T_{i,m}, \forall Q\colon \mathbf{LB}\left(Q, T_{i,m}, r\right) \le \mathrm{cDTW}^2(Q, T_{i,m}, r)$$

- Минимизируем не ED и DTW, а их квадраты
  - ускорение без изменения относительного порядка
  - корень извлекается перед выдачей ответа пользователю
  - дальнейшие оптимизации (алгоритм UCR-Suite)
- Контрапозиция: если $\mathbf{LB}\left(Q, T_{i,m}\right) > bsf$, то $\mathrm{DTW}^2\left(Q, T_{i,m}\right) > bsf$, т.е. $T_{i,m}$ отбрасываем без вычисления $\mathrm{DTW}^2\left(Q, T_{i,m}\right)$

Поиск на основе DTW с нижней границей

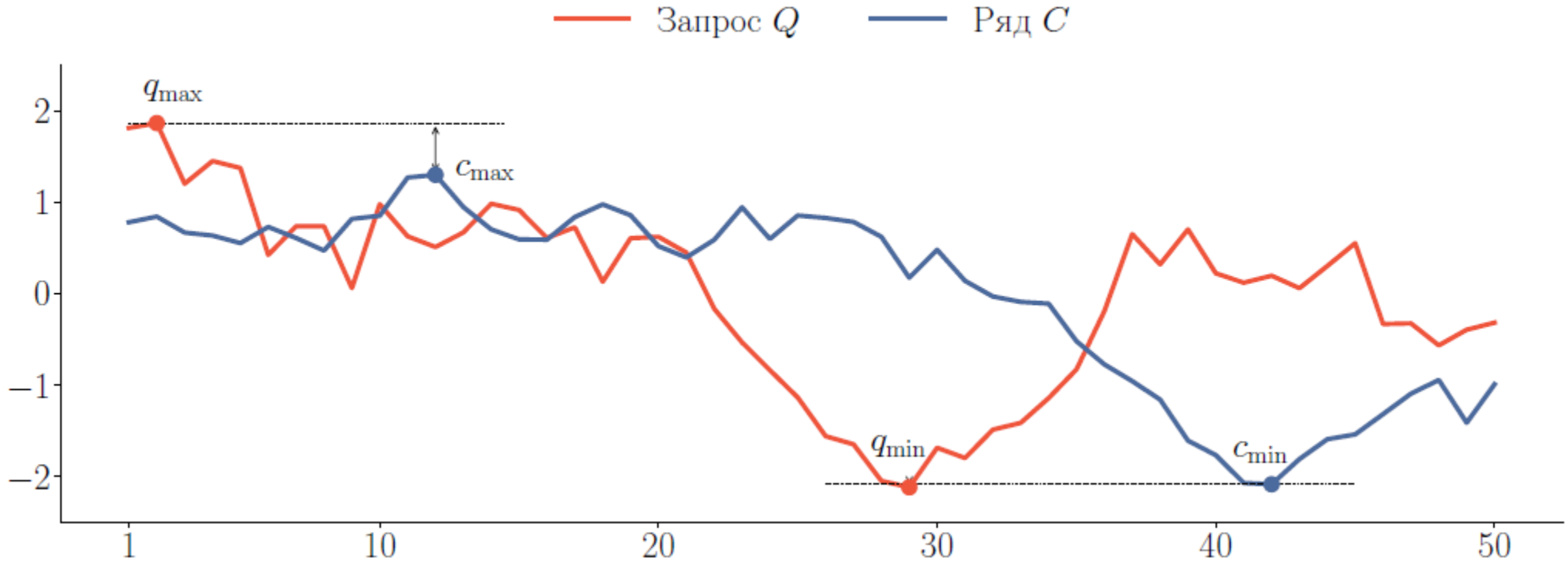
Вход: временной ряд T длины n, запрос Q длины m, $m \ll n$;
радиус полосы Сакоэ Тибы r, $0 \leqslant r \leqslant m-1$

Выход: наиболее похожая подпоследовательность $C_{\text{bestmatch}}$,
расстояние до запроса bsf

1: $bsf \leftarrow +\infty$
2: **for** $i \leftarrow 1$ **to** $n - m + 1$ **do**
3: **if** $\text{LB}(Q, T_{i,m}) < bsf$ **then**
4: $dist \leftarrow \text{cDTW}^2(Q, T_{i,m}, r)$
5: **if** $dist < bsf$ **then**
6: $bsf \leftarrow dist$; $C_{\text{bestmatch}} \leftarrow T_{i,m}$
7: **end if**
8: **end if**
9: **end for**
10: **return** $\{C_{\text{bestmatch}}, \sqrt{bsf}\}$

Вычислительная сложность
$\boldsymbol{O(nmr)}$,
но работает существенно быстрее наивного алгоритма за счет отбрасывания

Нижняя граница LB_{Kim}*





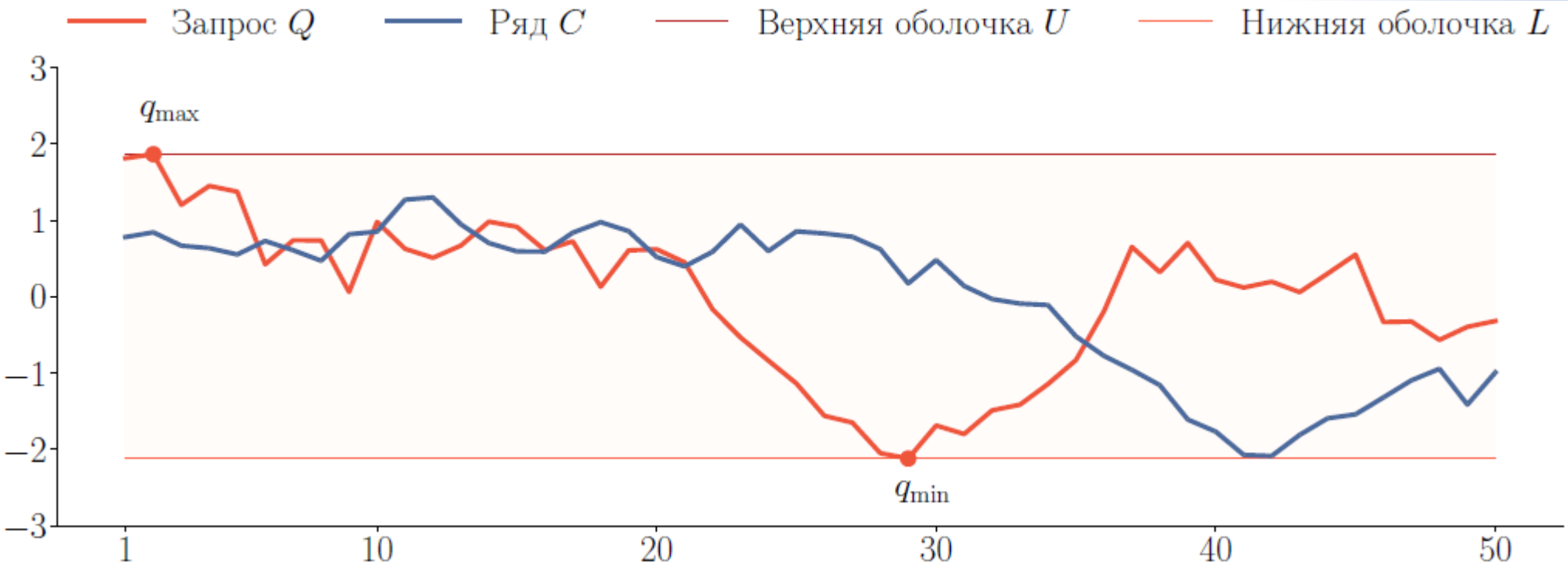
Санг-Вук Ким
(Sang-Wook Kim)

$$LB_{Kim}(Q, C) = (q_{max} - c_{max})^2 + (q_{min} - c_{min})^2$$

Сложность
$O(m)$

*Kim S., *et al.* An index-based approach for similarity search supporting time warping in large sequence databases. Proc. of the 17th Int. Conf. on Data Engineering, ICDE 01, April 2-6, 2001, Heidelberg, Germany, pp. 607-614. DOI: 10.1109/ICDE.2001.914875

Нижняя граница LB_{Yi}*





Байоюнг-Ки И
(Byoung-Kee Yi)

$$\mathrm{LB_{Yi}}(Q,C) = \sum_{i=1}^{m} \begin{cases} (c_i - q_{\max})^2, c_i > q_{\max} \\ (c_i - q_{\min})^2, c_i < q_{\min} \\ 0, \quad q_{\min} \le c_i \le q_{\max} \end{cases}$$

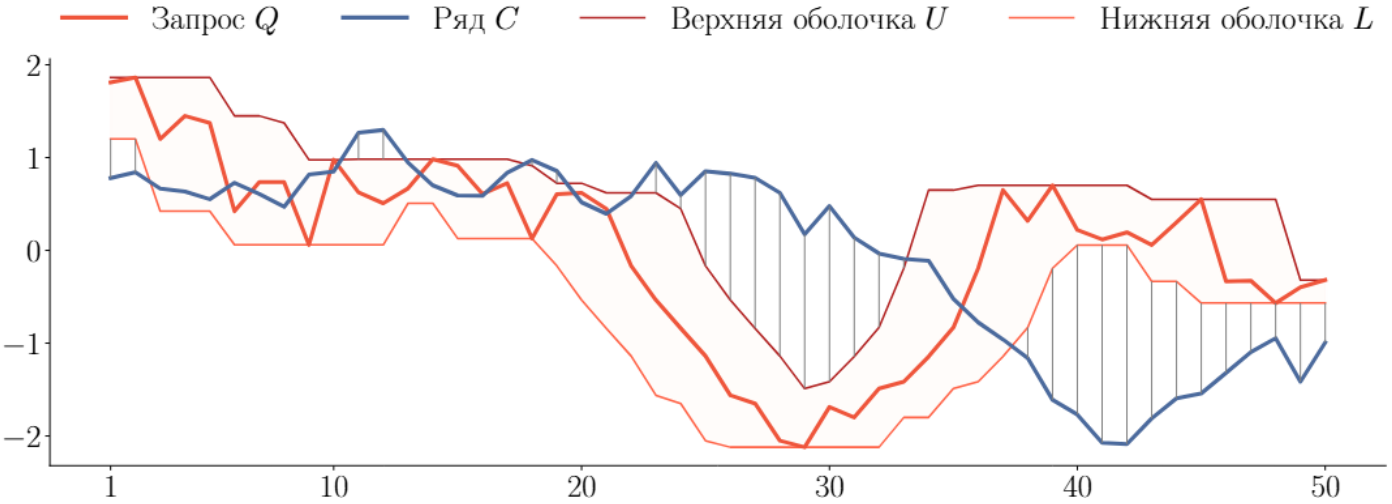
Сложность
$O(m)$

* Yi B. et al. Efficient retrieval of similar time sequences under time warping. ICDE 98. DOI: 10.1109/ICDE.1998.655778

Нижняя граница $\mathbf{LB_{Keogh}}$*



Эймон Кио
(Eamonn Keogh)



Квадрат евклидова расстояния от подпоследовательности C до ближайшей оболочки запроса Q

Верхняя оболочка запроса: U

$$u_i = \max_{\max(1,i-r)\le k\le \min(m,i+r)} q_k$$

Нижняя оболочка запроса: L

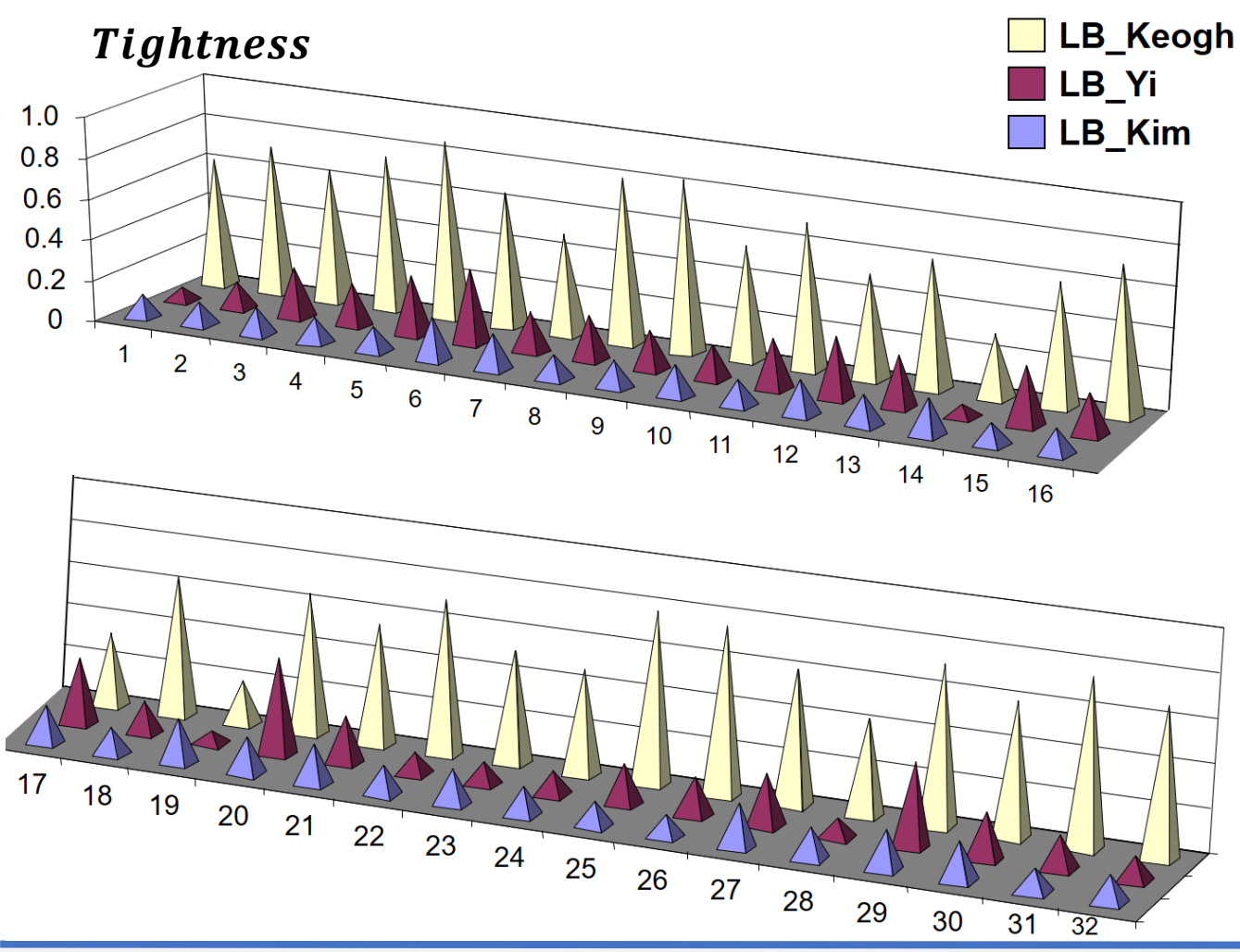
$$\ell_i = \min_{\max(1,i-r)\le k\le \min(m,i+r)} q_k$$

$$\mathbf{LB_{Keogh}}(Q,C,r) = \sum_{i=1}^{m} \begin{cases} (c_i - u_i)^2, c_i > u_i \\ (c_i - \ell_i)^2, c_i < \ell_i \\ 0, \quad u_i \le c_i \le \ell_i \end{cases}$$

NB! $\mathbf{LB_{Keogh}}$ **работает только в случае равных длин запроса и подпоследовательности:** $|Q| = |C|$

* Keogh E.J. Exact Indexing of Dynamic Time Warping. VLDB 2002. pp. 406–417. DOI: 10.1016/B978-155860869-6/50043-3

Сравнение нижних границ: Tightness (близость к DTW)

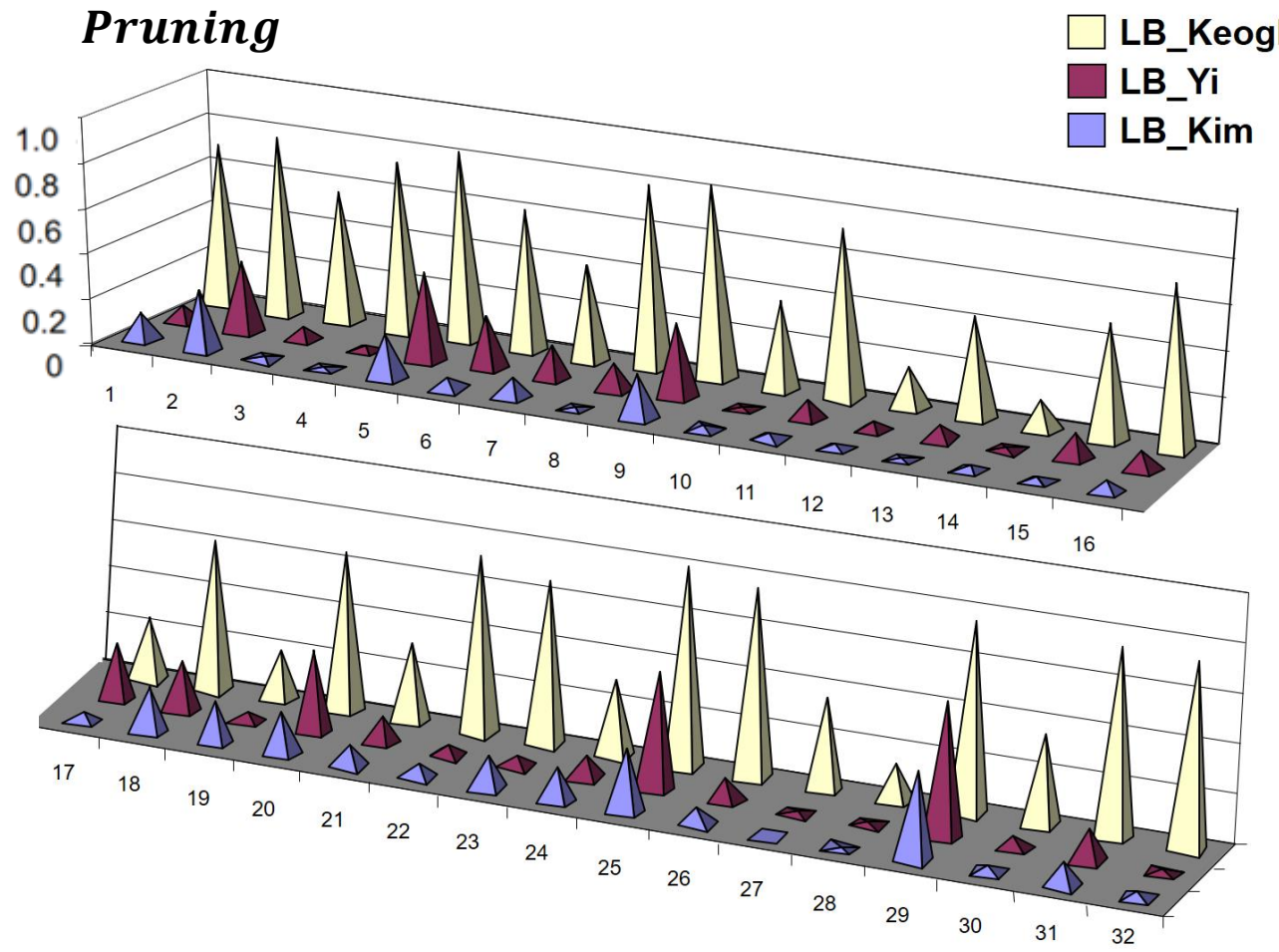


- Взяли 32 временных ряда из различных предметных областей
- В каждом ряде взяли 50 случайных подпоследовательностей наиболее типичной длины 256
- Для каждой пары подпоследовательностей Q, C вычислили $\mathrm{LB}(Q, C)$ и $\mathrm{DTW}(Q, C)$ и усреднили
- В итоге вычислили метрику

$$\boldsymbol{Tightness} = \frac{\mathbf{LB}}{\mathbf{DTW}}$$

($0 \leq Tightness \leq 1$, большее значение лучше)

Сравнение нижних границ: Pruning power (объем отбрасывания)



- Взяли 32 временных ряда из различных предметных областей
- В каждом ряде взяли 50 случайных подпоследовательностей наиболее типичной длины 256
- Каждую подпоследовательность взяли как запрос для поиска наиболее похожей подпоследовательности среди остальных и усреднили число случаев где НЕ вычисляли DTW
- В итоге вычисли метрику

$$\boldsymbol{Pruning} = \frac{\text{число случаев где НЕ вычисляли } \mathbf{DTW}}{|S_T^m|}$$

($0 \leq Pruning \leq 1$,
большее значение лучше)

Применение нескольких нижних границ

Вход: временной ряд T длины n, запрос Q длины m, $m \ll n$;
радиус полосы Сакоэ—Тибы r, $0 \leqslant r \leqslant m-1$
Выход: наиболее похожая подпоследовательность $C_{\text{bestmatch}}$,
расстояние до запроса bsf

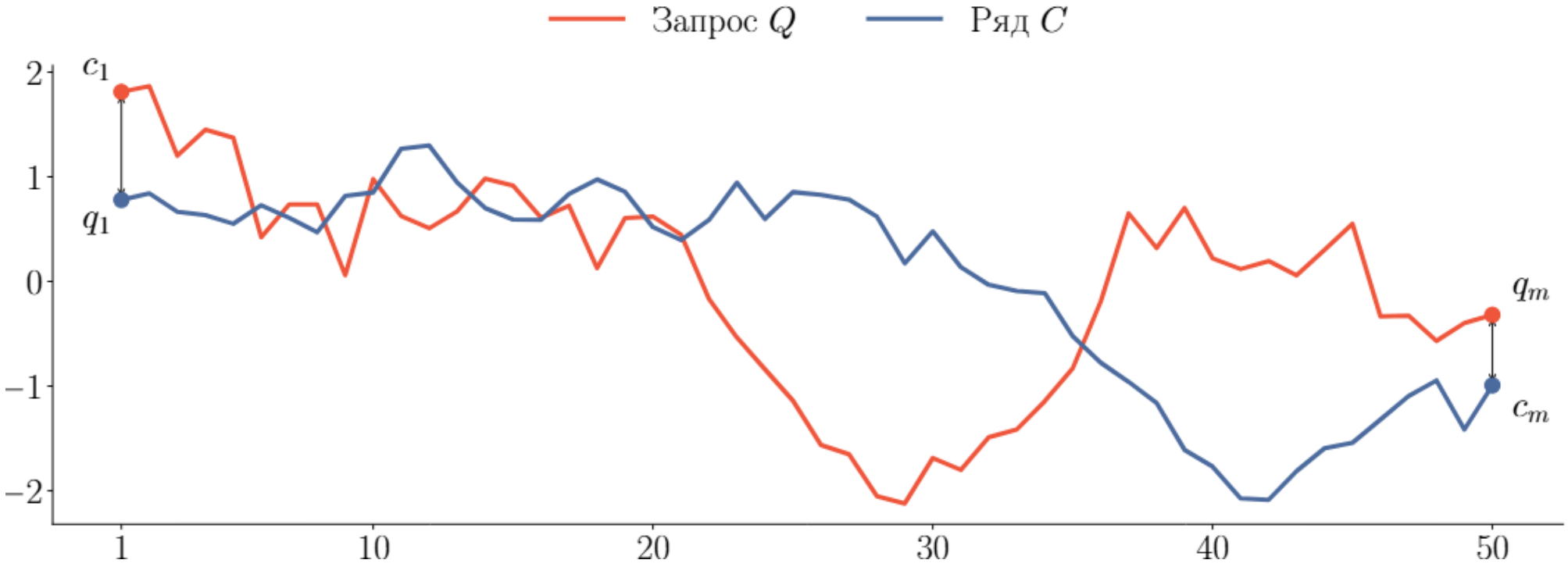
```
1: bsf ← +∞
2: for i ← 1 to n − m + 1 do
3:     if LB_1(Q, T_{i,m}, r) < bsf then
4:         if LB_2(Q, T_{i,m}, r) < bsf then
5:             if LB_3(Q, T_{i,m}, r) < bsf then
       ▷ Можно добавить другие нижние границы: if LB_k(Q, T_{i,m}, r) < bsf then ...
6:                 dist ← cDTW^2(Q, T_{i,m}, r)
7:                 if dist < bsf then
8:                     bsf ← dist; C_bestmatch ← T_{i,m}
9:                 end if
10:            end if
11:        end if
12:    end if
13: end for
14: return {C_bestmatch, √bsf}
```

- Постепенное увеличение вычислительной сложности и близости применяемых границ
- Комбинирование границ (max значение из нескольких)
- Ранняя остановка вычислений
- Динамическая настройка границ
- Предварительная z-нормализация запроса и подпоследовательности
- Вычислительные эксперименты

Алгоритм UCR-Suite

- Наиболее быстрый алгоритм поиска по образцу на основе DTW
- Ключевые особенности
  - Z-нормализация запроса и подпоследовательности
  - Каскад $LB_{Kim}FL$, LB_{Keogh}, $LB_{Keogh}EC$
  - Оптимизация вычислений
    - Ранний останов вычисления ED и LB_{Keogh}
    - Ранний останов вычисления DTW
    - Более раннее отбрасывание DTW с помощью LB_{Keogh}
    - Переупорядочивание раннего отбрасывания

UCR-Suite, каскад: Нижняя граница $\mathrm{LB_{Kim}FL}^*$





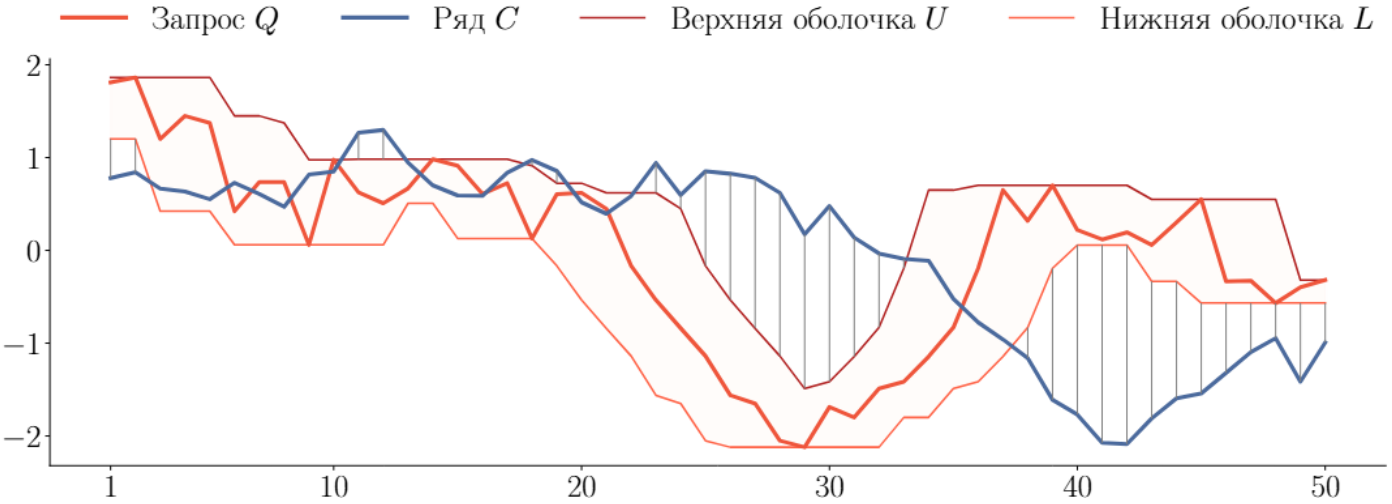
Эймон Кио
(Eamonn Keogh)

$$\mathrm{LB_{Kim}FL}(Q, C) = (q_1 - c_1)^2 + (q_m - c_m)^2$$

Сложность
$O(1)$

* Rakthanmanon T. *et al.* Addressing big data time series: Mining trillions of time series subsequences under Dynamic Time Warping. TKDD. 2013. Vol. 7, No. 3. P. 10. DOI: 10.1145/2500489

UCR-Suite, каскад: Нижняя граница $\mathbf{LB_{Keogh}}$



Квадрат евклидова расстояния от подпоследовательности C до ближайшей оболочки запроса Q

Верхняя оболочка запроса: U

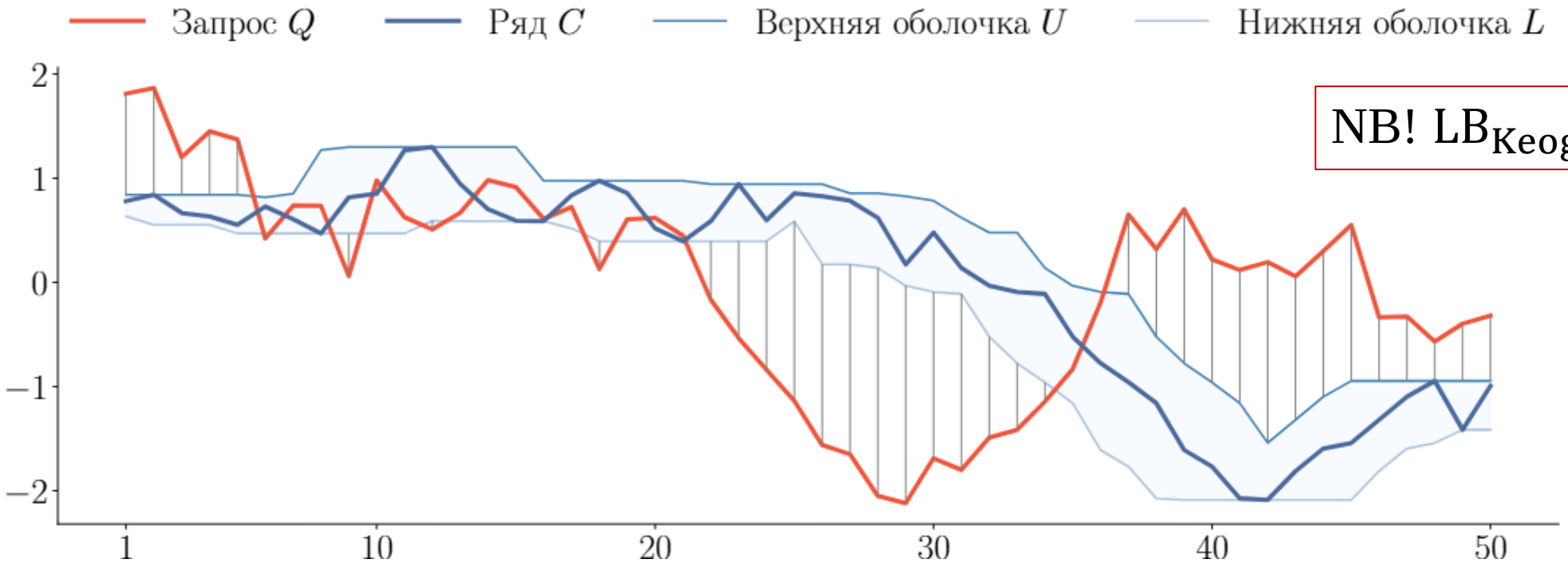
$$u_i = \max_{\max(1,i-r)\le k\le \min(m,i+r)} \hat{q}_k$$

Нижняя оболочка запроса: L

$$\ell_i = \min_{\max(1,i-r)\le k\le \min(m,i+r)} \hat{q}_k$$

$$\mathbf{LB_{Keogh}}(Q,C,r) = \sum_{i=1}^{m} \begin{cases} (\hat{c}_i - u_i)^2, \hat{c}_i > u_i \\ (\hat{c}_i - \ell_i)^2, \hat{c}_i < \ell_i \\ 0, \quad u_i \le \hat{c}_i \le \ell_i \end{cases}$$

UCR-Suite, каскад: Нижняя граница $\mathbf{LB_{Keogh}EC}$

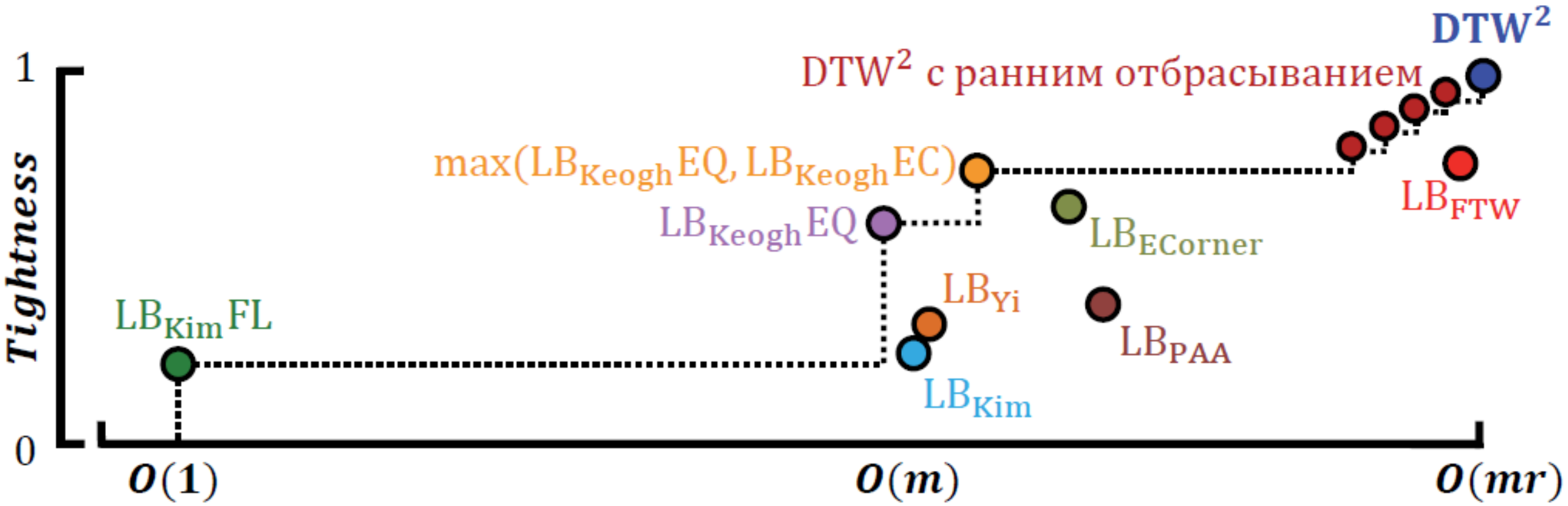


NB! $\mathrm{LB_{Keogh}EC} \neq \mathrm{LB_{Keogh}EQ}$

- Оболочка строится вокруг подпоследовательности-кандидата, а не вокруг запроса, Q и C меняются местами (аббр. Envelope around the Candidate):

$$\mathbf{LB_{Keogh}EC}(Q, C, r) = \mathrm{LB_{Keogh}EQ}(C, Q, r)$$

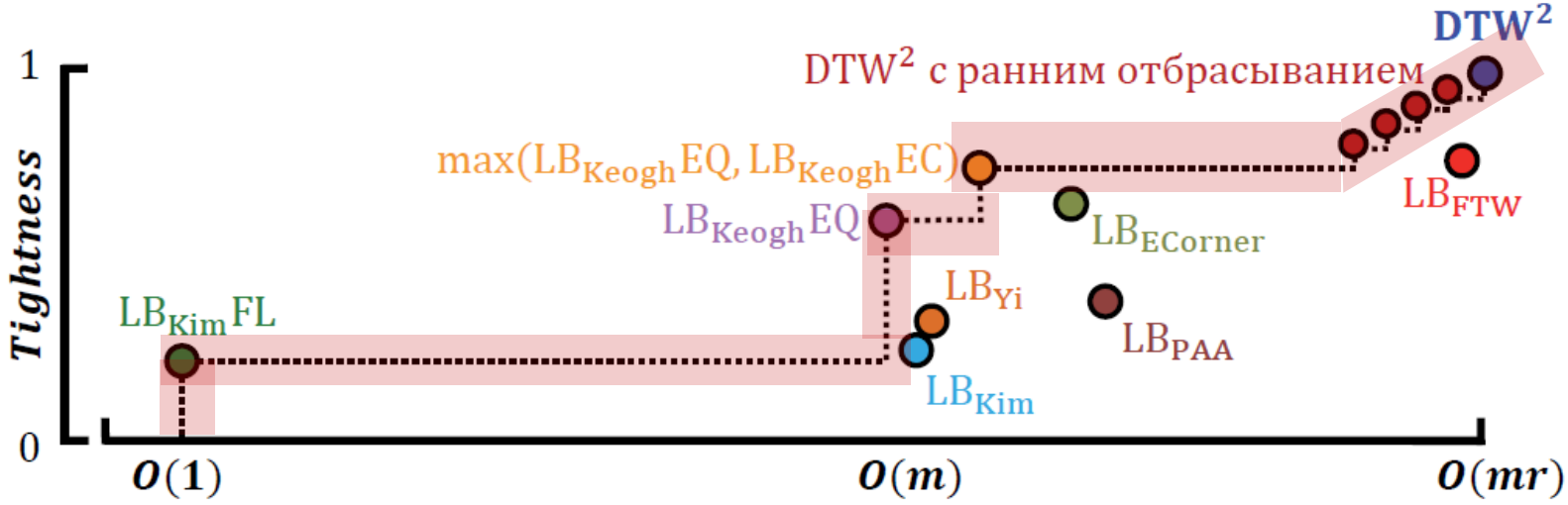
UCR-Suite, обоснование каскада нижних границ



* Rakthanmanon T. *et al.* Addressing big data time series: Mining trillions of time series subsequences under Dynamic Time Warping. TKDD. 2013. Vol. 7, No. 3. P. 10. DOI: 10.1145/2500489

UCR-Suite: каскад нижних границ

- Все в линейке ····· каскадом от менее к более вычислительно затратным
- Отказ от любой нижней границы в каскаде замедляет вычисления минимум вдвое
- Применение каскада позволяет достичь $\boldsymbol{Prune = 0.99999}$*





Большой каскад / Гос. музей-заповедник «Петергоф»

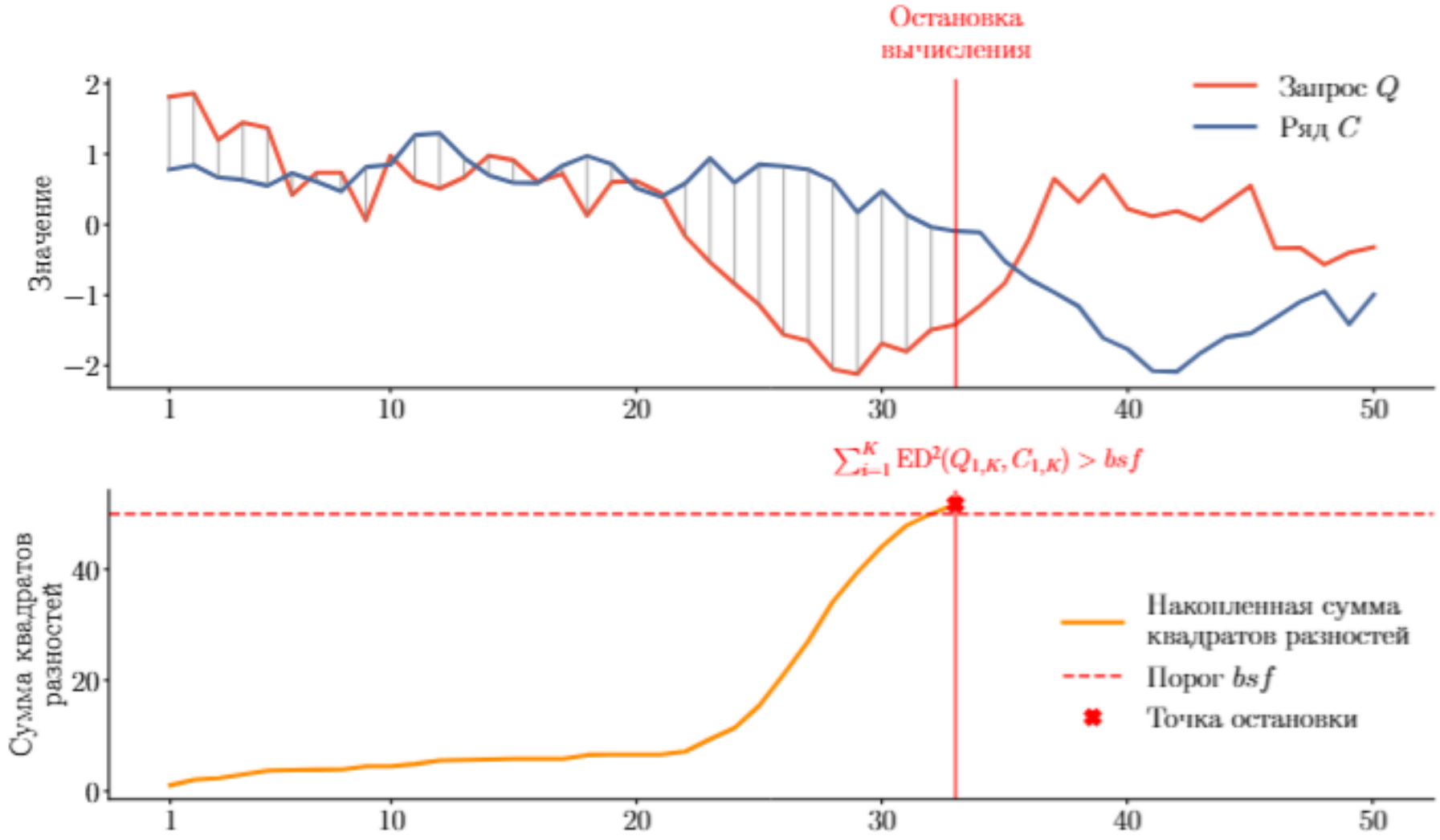
* Rakthanmanon T. *et al.* Addressing big data time series: Mining trillions of time series subsequences under Dynamic Time Warping. TKDD. 2013. Vol. 7, No. 3. P. 10. DOI: 10.1145/2500489

Оптимизация: ранняя остановка вычислений

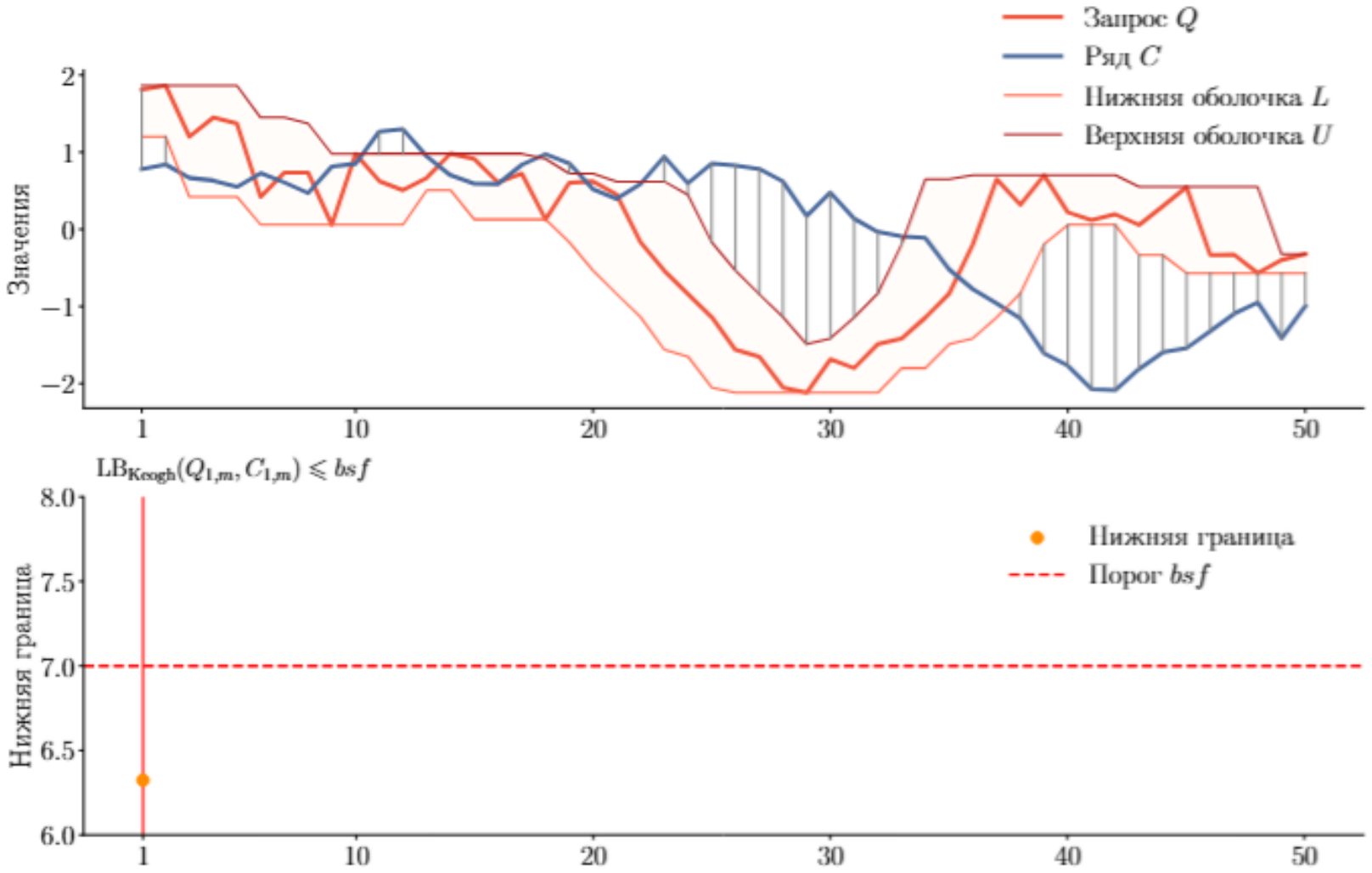
Прекратить вычисления, если текущий результат больше bsf

1. Ранняя остановка вычисления ED и $\mathrm{LB}_{\mathrm{Keogh}}$
2. Ранняя остановка вычисления DTW
3. Более раннее отбрасывание DTW с помощью $\mathrm{LB}_{\mathrm{Keogh}}$
4. Переупорядочивание раннего отбрасывания

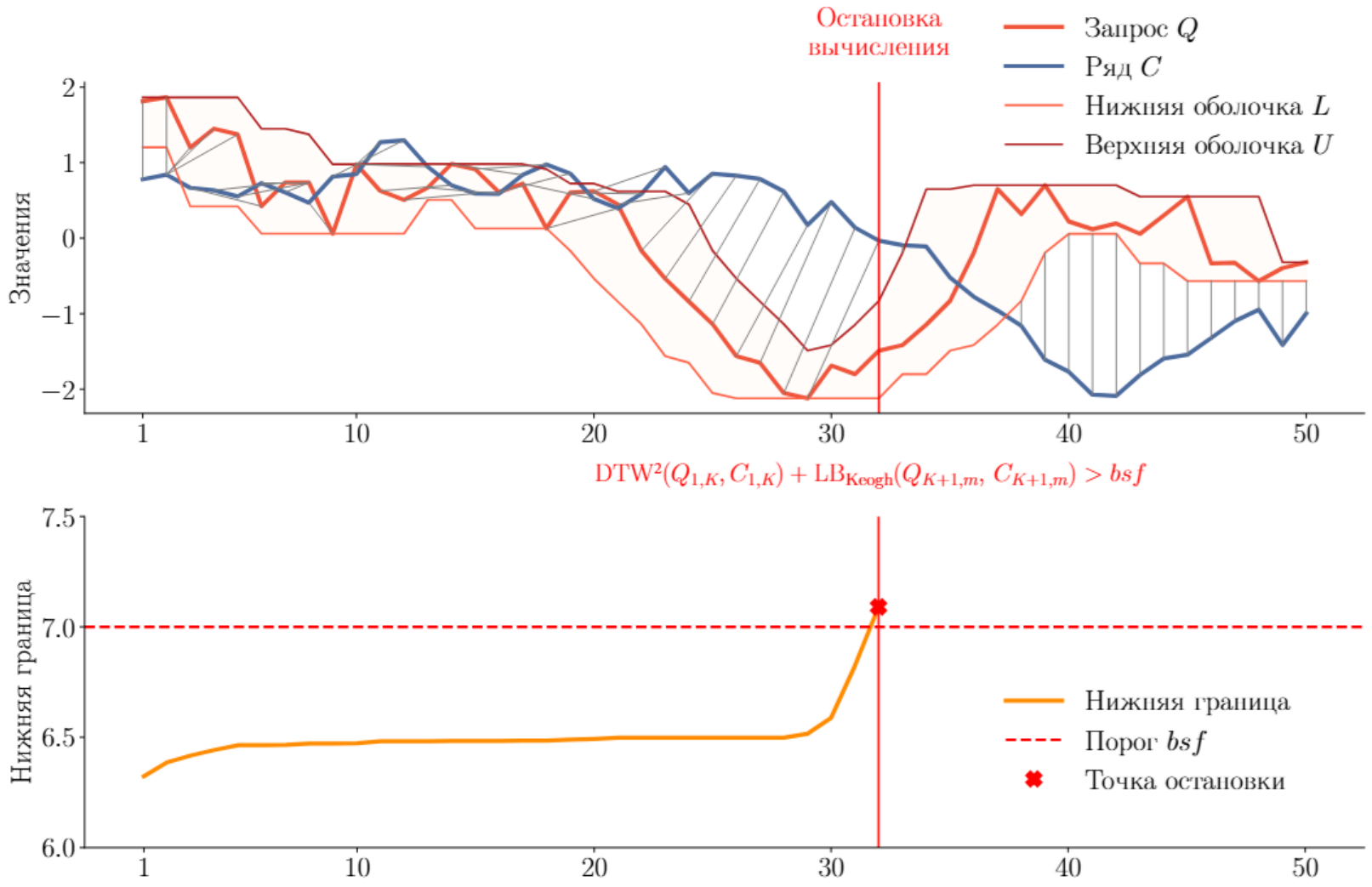
Ранняя остановка вычисления ED и LB_{Keogh}



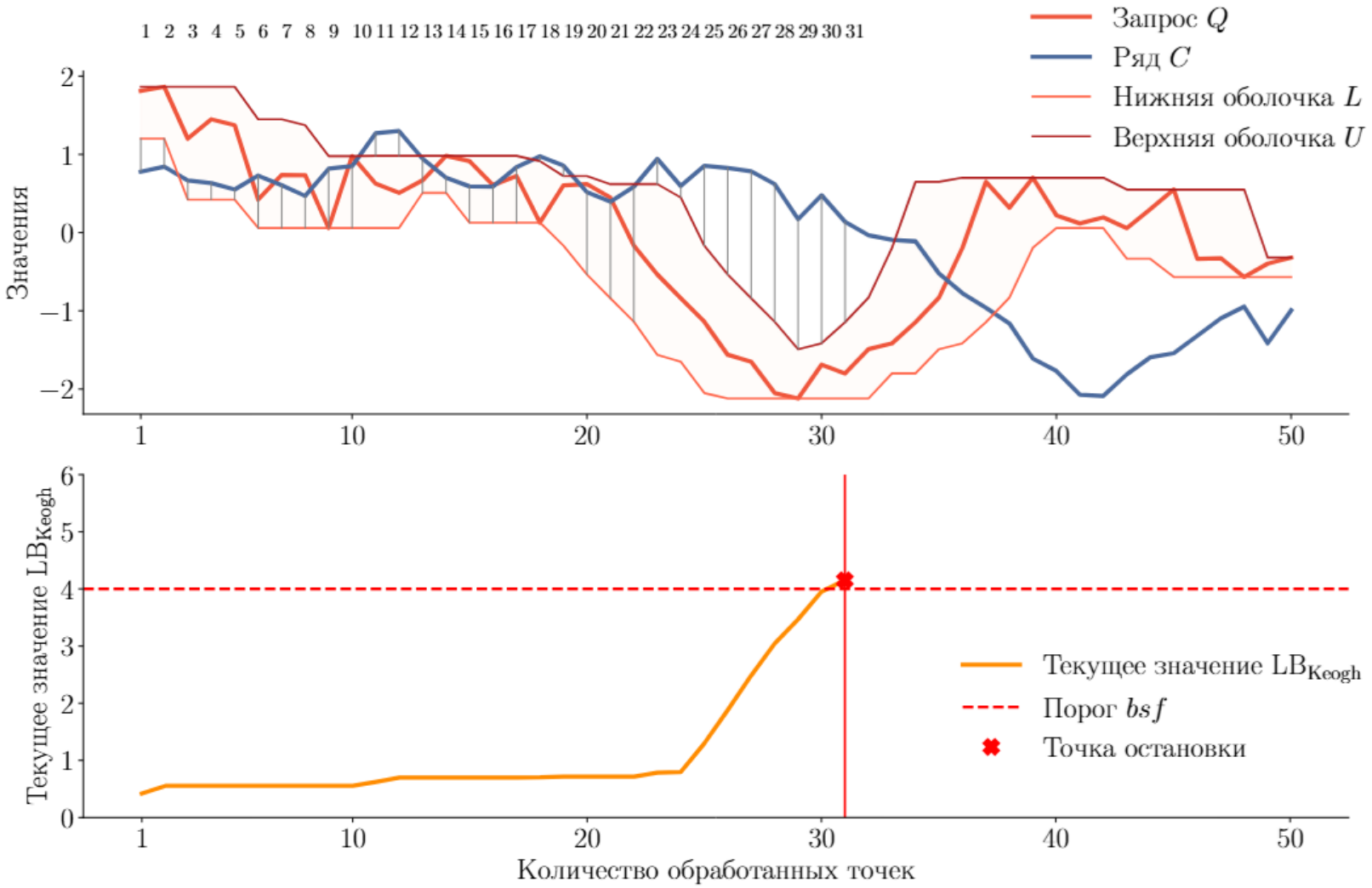
Ранняя остановка вычисления DTW (1/2)



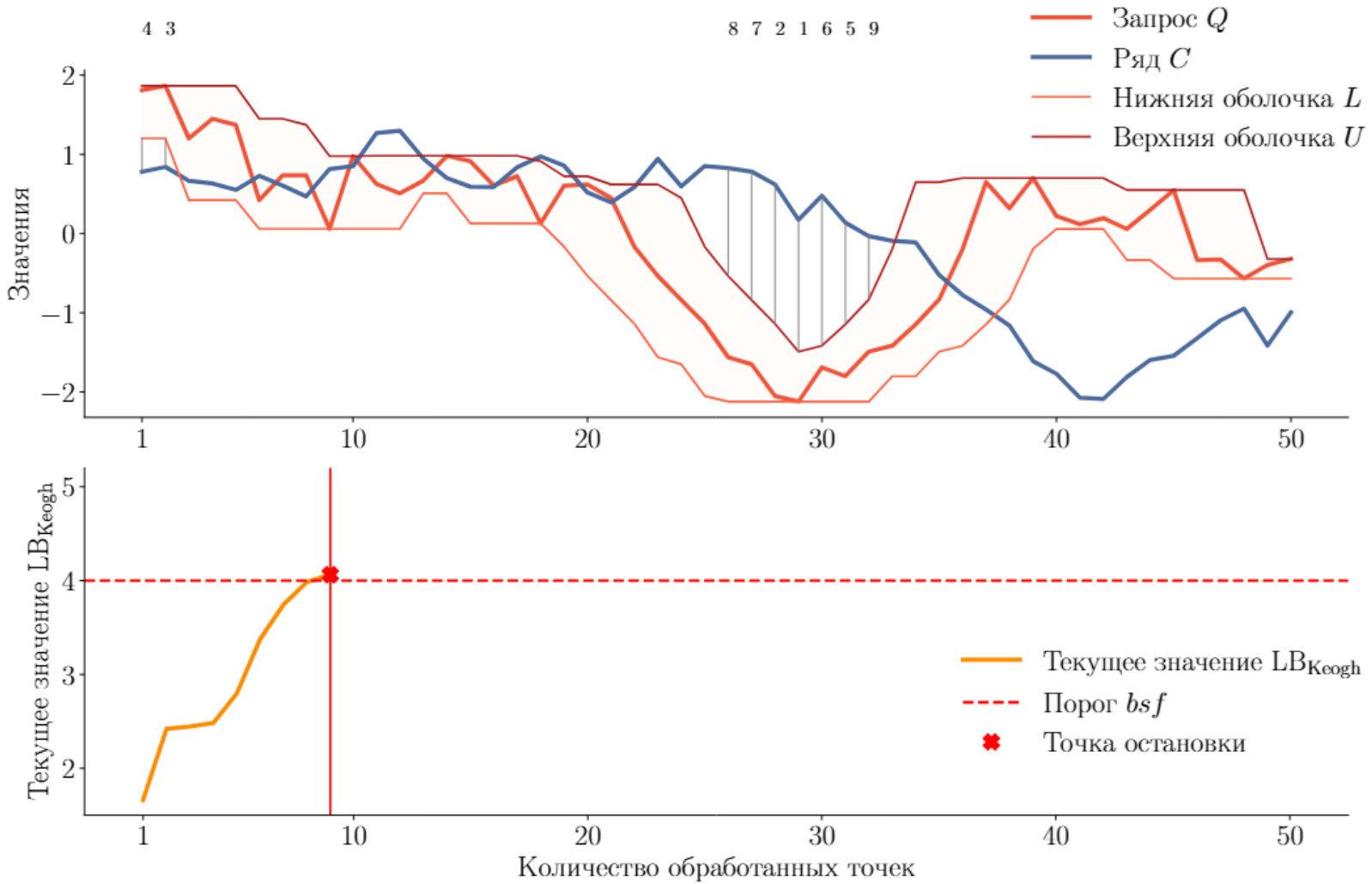
Ранняя остановка вычисления DTW (2/2)



Переупорядочивание вычислений и их ранняя остановка (1/2)



Переупорядочивание вычислений и их ранняя остановка (2/2)



Литература

Цымблер М.Л. Интеллектуальный анализ временных рядов: учебное пособие. Челябинск: Издательский центр ЮУрГУ, 2026. 312 с. ISBN 978-5-696-05766-8

Глава 3. Поиск по образцу, раздел 3.2